pii · personal information index

Architecture and Development Manual

Marcelo Correa

Version 0.2.0 · build 2026-11-04 10:27 UTC · commit `4249d5b2fa85`

The internals: engine, composition, stores, gates

Contents

1 Design goals

This chapter answers a single question: what is pii *for*, and which constraints follow from that purpose whether or not anyone likes them. Every later chapter describes a mechanism; this one describes the pressure that shaped it. The purpose is narrow. pii exists so that a claim about work a model did — "the spec is implemented", "the tests pass", "the document is current" — can be checked by someone who was not present, from records the system kept while it worked. Everything else in the workspace is downstream of that.

The book is deliberately organised around failure. Each of the five chapters that follow names the mechanism *and* the defect that made the mechanism necessary, because in this repository the two are not separable: the exit rule in composition exists because a refusal silently vanished, the health record exists because a status was being recomputed from 41,150 events on every call, and the queue's lease exists because one dead claim wedged an entire lane. A design goal that has never cost anybody anything is a slogan. The ones below have all been paid for.

1.1 The four goals

1. **Evidence over narration.** A run leaves a journal of transitions; the stores keep counters and headers. A statement that cannot be reconstructed from those records is treated as unknown rather than believed.
2. **Composition over configuration.** A graph is assembled from nodes and graphs that already exist. There is one implementation of "implement a spec" and several ways to reach it, never two implementations that can drift apart.
3. **Refusal over guessing.** An unknown configuration key, an unserved model id, a port a node does not declare, a capability the graph does not hold, a pin that no longer matches the file: each refuses by name, at the earliest moment it can be detected.
4. **Bounded growth.** Every store, journal, artefact directory and source file has an owner and a ceiling. An unattended host must not fill its disk, its memory or its review capacity while nobody is looking.

The four are not independent. Evidence is what makes a refusal meaningful — a refusal that leaves no record is indistinguishable from a crash. Composition is what makes evidence cheap, because a called graph journals under the caller's run id instead of inventing a second history. Bounded growth is what keeps the first three affordable over months rather than days.

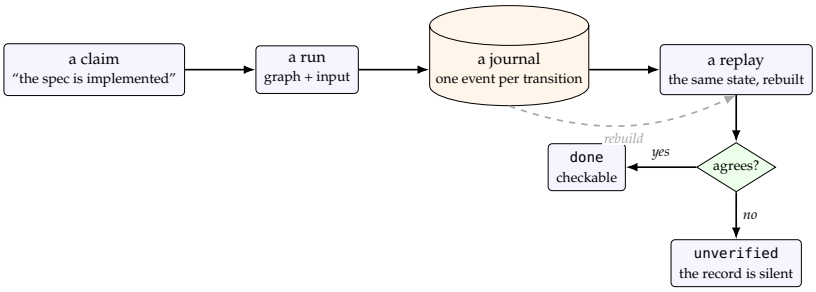


Figure 1.1: The chain a verdict has to survive: the dashed edge is what makes it a test rather than an essertion, because the state is rebuilt from the journal and compared with the record the write path maintained.

1.2 Evidence over narration

A shell script leaves a transcript, and a transcript is not evidence. It says what was printed, not which decision was taken at which port, and it cannot distinguish “the command succeeded” from “the command was never run”. The engine therefore journals a *state transition* per node: the node, the port it left on, the value it produced, the hash of that value, and a monotonic sequence number. From that record three things become possible that a transcript cannot support: a run can be resumed from the position the journal says is next, a refusal can be attributed to the hook that raised it, and health can be rebuilt rather than trusted.

The third of those is the one that was got wrong first, and the measurement is worth stating. Health used to replay every journal of every run on every call. On this host, with 1,895 runs and 41,150 events in the store, that took 82, 85 and 92 milliseconds across three consecutive calls: linear in history, on one of four async workers, with no cap. At ten times the history it is roughly nine tenths of a second of a blocked worker per health call, and health is the surface every watchdog reaches for first. The fix was not a cache with a time-to-live. It was the observation that the answer is a *status*, and a status should be maintained at the moment it changes, not recomputed from first principles. The record is now written by the write path; the replay survives as the test that the record still agrees with the events.

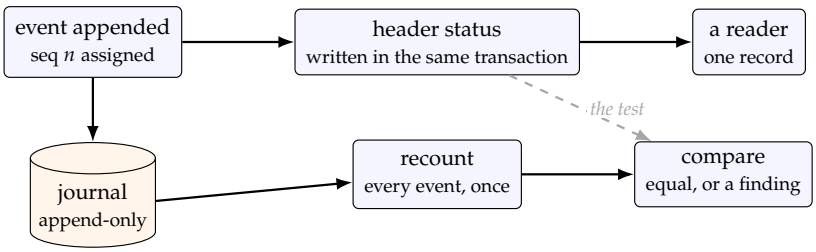


Figure 1.2: Two readers of one truth — the fast path reads a maintained record, the slow path recomputes from the journal — kept together because a counter with no recomputation behind it is an assertion with extra steps.

The record can be interrogated directly, which is the practical test of whether it is evidence at all rather than a log:

```
pii run trace <run_id>              # every transition, in sequence order
pii run explain <run_id>            # why the run ended as it did
pii graph lint graphs/queue.toml    # the same rules a run start applies,
    ↪ over one file
```

There is a second, quieter consequence. Because the journal is the primary artefact, a terminal verdict must not be a summary of intentions. The engine determines a run's output by the last finished node that has no outbound data edge on the port it took — the node whose value nobody consumed. If a final node left on a non-`ok` port carrying an error, the run is a failure even though a graph author might have expected "it reached the end". The record decides, not the shape of the picture.

1.3 Composition over configuration

The second goal is stated as a rule about implementations, not about files: one behaviour, one place. A graph that needs to implement a spec calls the graph that implements specs. It does not carry a copy of it with different node ids, because a copy is a second implementation and the two will diverge the first time only one of them is fixed.

This is not hypothetical in this repository. The development queue used to carry its own `dispatch` coder and its own `judge`, doing inline what `graphs/spec.toml` already did, and the two drifted in exactly the way the rule predicts: the queue's copy ran the row's criteria while the spec graph ran the spec's, so a row could fail while the work it described was green. Row 364 removed the copy and replaced it with one pinned `call`. The graph that resulted is shorter than the one it replaced.

Composition also has a naming obligation that configuration does not. In `graphs/night.toml` there was a node whose id was `sdd` whose kind was `call` with `graph = "queue"`. An operator reading the night would conclude that SDD ran. It did not; the queue ran, and the queue is row-by-row work rather than the SDD process. Row 373's requirement is not "add an sdd graph" but "make the name true": the node called `sdd` now calls `sdd`. A composition whose names do not describe what is composed is worse than no composition, because it is read by people who will not check.

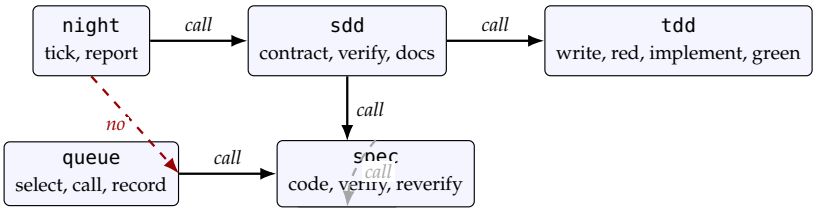


Figure 1.3: What composes into what, and the edge that had to be removed — the red dashed edge is the old `sdd` node pointing at the queue, a composition that ran and meant something else.

The cost of this goal is stated plainly because it is real: to know what `graphs/queue.toml` does, a reader must also read `graphs/spec.toml`, and trust that the pin names the revision they read. There is no single file that answers "what happens when the queue runs". That is the price

of having one implementation instead of two, and Chapter 5 pays it in detail.

1.4 Refusal over guessing

The third goal is the one that produces the most text in the repository, because every refusal carries a machine-readable rule code. A refusal is not a log line that says “invalid graph”; it is a named rule, a named node, and — where a capability is missing — the name of the missing capability. The reason is operational: at three in the morning the difference between `graph.call.widen` and `V1` is the difference between reading one line of code and reading the loader.

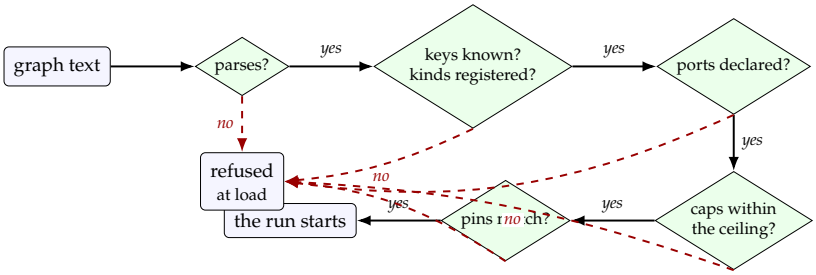


Figure 1.4: Refusal happens before execution and in a fixed order, so the message is the first thing actually wrong rather than the last thing that failed, and nothing on the top row has run yet.

Rule code	Raised by	What it names
`graph.call.widen`	`fragment/call.rs`	the call, the capability, the graph that does not hold it
`graph.call.cycle`	`fragment/call.rs`	the chain of graphs, in order
`graph.call.depth`	`fragment/call.rs`	the call and the depth cap of 4
`graph.call.exit`	`fragment/call.rs`	the callee with no `exit`, or an `exit` naming no node
`graph.call.id`	`fragment/call.rs`	the node id the call would duplicate
`graph.call.key`	`fragment/call.rs`	the one unknown key on the call
`fragment.pin.mismatch`	`fragment/call.rs`	expected and actual sha256
`fragment.pin.drift`	`fragment/splice.rs`	the fragment path and both hashes
`fragment.block.invalid`	`fragment/splice.rs`	a blocking fragment at a pre-surface
`fragment.autonomy.forbidden`	`fragment/check.rs`	the node that would own a timer, store or budget
`fragment.nesting.forbidden`	`fragment/check.rs`	the fragment that attaches or calls
`graph.grant.ceiling`	`toml_load.rs`	the graph or node, and the ceiling

Rule code	Raised by	What it names
`V0`–`V11`	`validate.rs`	the rule, then the node

The codes share a property that matters more than their spelling: they are stable, greppable, and each one maps to exactly one place in the source. That is what allows a test to assert a refusal by name instead of asserting that "an error occurred", which is the assertion that lets a wrong refusal pass.

1.5 Bounded growth

The fourth goal is the least glamorous and the one that decides whether the other three survive a year of unattended operation. Three ceilings are visible from the outside:

- **Source files.** A file may have at most 450 effective lines — non-blank, non-comment. The limit is enforced by a gate step, not by review, and the fix when it is hit is an extraction into a module that owns the concept. `daemon/sys_status.rs`, `store/sys_status.rs` and `daemon/model_roles.rs` all exist because a file crossed the line and the concept inside it deserved a name.
- **Artefacts.** A trajectory directory has an aggregate byte bound and a file-count bound, and the oldest finished runs are pruned past them. The defaults are 512 MiB and 50,000 files; both are configuration keys rather than constants, because the right number depends on the host.
- **History.** Journals are append-only by design, so growth is managed by retention and compaction (`pii store compact`, `pii store prune`) rather than by rewriting events. An event that is deleted is a claim that can no longer be checked, which is why the operation is explicit and why the daemon must be stopped for it.

The ceilings are visible from the operator's side too, which is how a bound becomes a habit rather than a rule:

```
pii store stats                   # tables, sizes, and what is growing
pii store prune                   # retention, with the daemon stopped
bash quality-gate.sh --summary    # one line per gate step, including the
    ↪ file budget
```

Bounded growth is also why the engine has a step limit, a wall-clock limit and a token budget rather than trusting nodes to be reasonable. A graph that loops is not an error in this system; a graph that loops forever is, and the difference is a number in the graph's `[limits]` table.

1.6 The cost, stated as a ledger

Every goal above is bought with something. The table is the honest version of the trade, and each row is developed in the chapter named in the last column.

Choice	What it buys	What it costs	Chapter
Journal every transition	replay, attribution, rebuilt counters	write volume; a write-behind buffer that must be flushed at the right moments	3
Inline calls at load time	one implementation, no serialisation, one journal	a reader must follow a pin to know what runs; the callee becomes part of the caller's name space	5
Capabilities as a ceiling	an escalation must be written where it is reviewed	a graph that needs one more capability must be edited at two levels	4
Named refusals	a failing test can assert the reason	more strings to keep stable than a single "invalid"	4
450-line files	a concept has one home	more modules, more `mod` lines, more files to open	2
Maintained counters	constant-time health	a write path that must never forget to update them; a rebuild test that must exist	3

The last row is the one that recurs. Almost every defect described in this manual is a variant of the same shape: a fast path added for a good reason, with no slow path left to check it. The maintained counter, the inlined graph, the pinned fragment and the bounded file all trade a computation for a record. The discipline that keeps the trade honest is that the computation is kept somewhere as a test.

1.7 The vocabulary, fixed early

Five words carry most of the meaning in this book, and each one is used in a narrower sense than its everyday meaning. They are stated once here so that later chapters can use them without redefinition.

node A `kind` plus configuration. It does one thing, declares the ports it can leave from, and is the only unit the scheduler dispatches.

port A named outcome, not a boolean. `select` has four; a coder has two; a judge has two or three. An edge names the port it leaves from.

graph A table of nodes, a table of edges, an `entry`, a `caps` list, optional `exit` and optional `[limits]`. It is data, not code.

run One execution of a graph with one input. It has an id, a journal, a header and exactly one terminal disposition.

call A node kind that is replaced, at load time, by the nodes of another graph. It is not a subprocess and not a remote run.

fragment A graph file under `graphs/fragments/` with a `[fragment]` header, attached to a host at a named surface and woven in at load time.

pin The sha256 of a file's bytes, written into the file that includes it. A pin that does not match is a refusal, not a warning.

store A redb database with named tables. Every store has a single writer and an owner crate.

Two of these definitions do quiet work. *Port* being a name rather than a boolean is what makes a graph read as a sentence: `red.ok` to `not-red` says something a `true` branch cannot. And *pin* being a hash of bytes rather than a version number is what makes a review meaningful: a version can be reused, a byte string cannot change without the hash changing.

1.8 Five defects that shaped the design

The goals above are abstractions. The table below is the concrete version: five rows, each of which changed a mechanism, with the symptom as an operator would have seen it.

Row	Symptom	What changed
353	A claimed queue row never completed, and every other row stopped being claimed. A restart did not help, because a row that is `Running` blocks the queue by design.	The claim became a lease: a row is requeued when the run that claimed it is terminal or gone, and never while it is alive or suspended. A boot pass re-offers suspended runs without waiting for an event.
364	A row stayed `Running` although the called graph had refused. Nothing in the journal said why, because nothing after the refusal had run.	A host edge may name a callee's refusal port; the load marks that refusal node to return its verdict on `ok` instead of failing the run.
370	`pii health` took 82–92 ms and grew with history; at ten times the store it would block a worker for most of a second.	The totals became a record maintained by the write path; the replay stayed as the test that the record agrees with the events.
372	The engine could not express "this test must fail", so a RED could not be proven and a test written after the code was indistinguishable from one written before it.	The judge accepts `expect = "fail"` and gains a third port, `unrunnable`, for a criterion that could not run at all.
373	A node named `sdd` called `queue`. The night claimed to run SDD and ran row-by-row work.	The name was made true: the node calls the `sdd` graph, and the pin moved in the same commit.

Read as a set, the five share a shape. In each case the system was doing something reasonable and the *record* was wrong: the row said `Running`

when it was parked, health said nothing about how it was obtained, the night's node said `sdd` when it was the queue. The fix is never to make the code more clever; it is to make the record say what happened, and then to leave a test that fails if the record and the events disagree.

1.9 What pii is not

Three negative statements prevent most misreadings of the rest of the book.

- **It is not a workflow engine with an iteration primitive.** Composition is static: pieces are spliced at load time, so a graph cannot say "do this call once per criterion". Everything that needs repetition today is one node looping internally — a judge running four commands — which is precisely why the judge has to know how to run criteria at all.
- **It is not a distributed system.** A call is inlined into the caller; there is no remote invocation, no queue between graphs and no serialisation boundary. The only subprocess in the picture is an agent, and the engine treats it as a node that may block, not as a peer.
- **It is not a scripting language.** A graph has no expressions beyond a small reference dialect (`$.input.x`, `$.nodes.n.field`) evaluated by the node that declares it. There are no local variables, no function calls and no conditionals outside a `route` or `gate` node. The restriction is the point: a graph that cannot compute can be checked by reading it.

1.10 How the claims in this book were checked

Every mechanism described here was read in the source and, wherever it could be, exercised. The distinction matters because this repository has paid three times for a check that passed while the live path was broken: a check that hand-wrote the lines it was meant to observe, a check fed a source constant instead of the message the live path delivers, and a verification harness that was itself wrong three times over. Each one certified a real defect as fixed.

The rule that follows from those three is stated in the workspace's agent guide and applies to prose as much as to tests: a check's input must come from the live path, and every rule that skips or suppresses something needs a near-miss case that must still take the normal path. The table below is the version of that rule applied to the claims in this book.

Claim	How it was checked	The near miss
A call returns through every node with an unwired `ok` port	read in `fragment/splice.rs`, then traced on `graphs/spec.toml`'s tail	a callee whose tail is one declared exit still returns on that node
A wired refusal returns instead of failing the run	the unit tests in `fragment/call_output.rs` assert the inner node gains `returns`	a refusal no host wires still fails the run
A value naming a longer id is untouched	`rewrite_str` refuses to match `$.nodes.sddx` against `$.nodes.sdd`	a bare `$.nodes.sdd` is rewritten
Health no longer scans journals	the requirement's test compares a rebuilt record with a replay	an absent record rebuilds rather than reporting zeros
A judge can be expected to fail	ports change only when `spec` and `expect` are both present	a judge without `expect` keeps two ports

Where a claim could not be exercised — the daemon's behaviour under a supervisor that kills it mid-agent, for instance — this book says so in the text rather than rounding it to a certainty. The editor's rule for the volume is the same as the gate's: a number with no measurement behind it is not allowed to look like one.

1.11 What the reader should expect

The chapters that follow are ordered by dependency rather than by importance. The workspace (Chapter 2) explains which crate owns which behaviour and how to find a behaviour from a symptom. The engine (Chapter 3) explains what a run is: its states, its tick loop, its journal, its recovery. Node kinds (Chapter 4) explain the vocabulary a graph is written in — what a node may declare and what it may do. Composition (Chapter 5) explains how graphs become each other's parts, and contains the one rule in this system that surprises experienced readers: a call returns through every node that declares an `ok` port and has no outgoing `ok` edge, which is not the same set as the node the callee names in `exit`.

That rule is worth flagging here because it is the clearest example of the manual's method. It was not designed; it was discovered, by a queue row that stayed `Running` for a night while the graph it called had already refused. The next four chapters describe the system that produced that behaviour, and the reasoning that fixed it without inventing a second inclusion mechanism.

2 The workspace

This chapter answers where a behaviour lives. pii is a Cargo workspace of twenty-three members — twenty-two crates and `xtask` — and the split is by *responsibility that changes for its own reasons*, not by layer. The practical consequence for a reader is that a symptom names its own crate: a refusal raised while a graph is being read comes from `pii-graph`; a run that never finishes comes from `pii-server`'s engine; a duplicated implementation is found by `xtask`, not by a reviewer. The chapter ends with a table that maps a symptom to the file a reader should open first, because that is the question a new contributor actually asks.

Two rules keep the workspace from decaying, and both are enforced mechanically rather than by convention: a file has a budget of 450 effective lines, and a copy of an idea is treated as a defect even when both copies are correct today. The second rule is the unusual one, and it is why several crates contain a file whose only job is to be the single home of a concept.

2.1 Membership

Crate	What it owns	A file you will open
`pii-abi`	The vocabulary: ids, ports, run status, journal events, capability bits, the wire types shared by everything. It depends on no other pii crate.	`src/journal.rs`, `src/cap.rs`
`pii-core`	The few root concepts a node needs to run: the context, the capability set, the filesystem scope, the budget, the clock, the state directory resolver.	`src/ctx.rs`, `src/cap.rs`
`pii-observe`	The closed event vocabulary and the sinks that receive it. Names live in one module so a metric cannot be spelled two ways.	`src/names.rs`
`pii-store`	Every redb table and its write paths: runs, queue, approvals, heals, memory, skills. Deliberately knows nothing about graphs.	`src/queue.rs`
`pii-graph`	The graph spec, the loader, static validation, the splice, the scheduler, the journal writer, the tick engine.	`src/fragment/splice.rs`, `src/sched/`
`pii-server`	The engine bridge, the daemon, the node kinds, the transports, the JSON-RPC dispatch table.	`src/engine/runs.rs`
`pii-proto`	The client/server protocol and its method names.	`src/run.rs`
`pii-uds`	The unix socket the CLI and the daemon speak over.	—

Crate	What it owns	A file you will open
`pii-cli`	The `pii` binary: argument parsing and one module per command group.	`src/cmd_run.rs`
`pii-worker`	The agent worker: argv construction, approval modes, in-flight limits.	—
`pii-model`	Model providers and the model id vocabulary.	—
`pii-net`	The network surface: egress grants, dial and accept.	—
`pii-memory`	Entities, tiers, the hub the memory nodes read.	—
`pii-learn`	Lessons, candidates, promotions, incidents.	—
`pii-tools`	The tool surface an agent may call, and its sandbox.	—
`pii-channels`	Outbound channels: Discord, MQTT.	—
`pii-tui`, `pii-web`, `pii-mcp`	Three operator surfaces over the same protocol.	—
`pii-wasm`, `pii-hal`, `pii-evolve`	Optional capability surfaces, each behind its own feature.	—

The table is worth reading twice, because the absences are as deliberate as the presences. `pii-store` has no graph types in it: a table stores an event, not a `RunState`. That is what allows the store to be opened by a lint registry, a test fixture or a repair tool without dragging the scheduler's semantics along. Conversely `pii-graph` does not contain a node kind: the built-in seven (`transform`, `route`, `emit`, `wait`, `timer`, `refuse`, `require`) live beside the registry, and everything that touches a model, a container, a store or a channel is registered by `pii-server`. A crate that could run a graph on its own would need none of those, which is what makes the graph crate testable with a stub registry.

2.2 The dependencies that are allowed

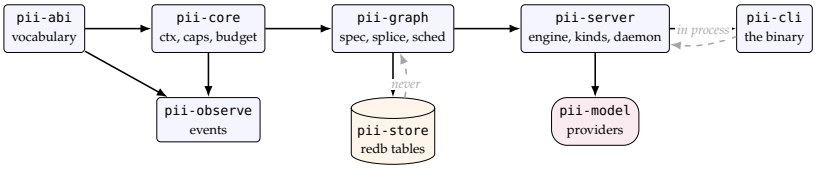


Figure 2.1: The dependency direction, left to right; the dashed edge marked "never" is the rule that keeps the store reusable: no table may name a graph type; the CLI's edge is worth noticing because it is why an operator running `pii run start` needs no daemon.

Read from each crate's own manifest, the edges are these:

Crate	Depends on (pii crates only)
`pii-abi`	none
`pii-observe`	none
`pii-core`	abi, observe
`pii-store`	abi, observe
`pii-proto`	observe
`pii-graph`	abi, core, observe, store
`pii-model`	abi, core, observe, net
`pii-tools`	abi, channels, core, model, observe, proto, uds
`pii-server`	abi, core, store, observe, graph, proto, model, memory, hal, wasm, learn, tools, net, worker, channels
`pii-cli`	abi, core, store, graph, observe, learn, memory, proto, server, uds, worker, tools, tui

Two of these edges are worth a sentence each. `pii-graph` depends on `pii-store`, which looks like an inversion until you see what it uses it for: the journal writer takes a `Store` and appends events in one transaction, so the crate that decides *what* to record is also the crate that records it, and the ordering between a run's header and its events is decided in one place. The reverse edge is what must never exist, and does not: `pii-store`'s manifest lists two pii crates, neither of them the graph.

The second edge is `pii-cli` to `pii-server`. The binary embeds the daemon in-process for the commands that need one, which is why `pii run start` works in a fresh shell with no socket in sight, and why an operator debugging a stuck run can attach to the same process that accepted it. The cost is that the CLI links the largest crate in the workspace; the benefit is that there is one code path for "start a run", not a client path and a server path that can drift.

2.3 One behaviour, followed across the crates

Abstractions about layering are easier to trust with a concrete trace. Take the refusal this manual returns to repeatedly: a host graph calls a callee whose capability list is wider than the call grants. The operator sees one line. The line is produced four crates away from where the decision was taken.

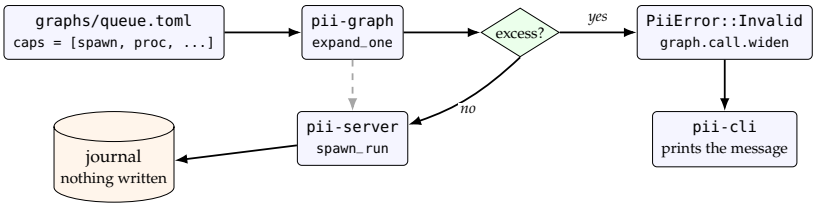


Figure 2.2: A load-time refusal; the graph is refused before a run exists, so the store stays untouched and the message an operator reads is the loader's own text — not a status derived from a run that failed later.

The trace in words. `pii-server`'s `spawn_run` is given a `GraphSpec`, which means the text has already been read and expanded by `pii-graph`. Expansion happens in `fragment/call.rs`: for each `call` node it reads the callee, checks the pin against the bytes, checks the callee's node capabilities against the caps the call grants, and refuses on the first excess. The error is a `PiiError::Invalid` whose message begins `graph.call.widen` and names the node and the capability. `pii-server` does not inspect it;

`pii-cli` prints it. And because the refusal happened during load, no run header was written: the operator sees a failed start, not a failed run, and there is no half-run to clean up.

The direction of that flow is the point of the crate split. A decision taken in the loader is not re-litigated by the engine, and the engine has no code path that could accept a graph the loader rejected. Every caller that runs or checks a graph goes through the same entry (`load_host_with_graphs`), so "check passed" and "run started" cannot disagree.

2.4 Inside `pii-server`

The largest crate is organised by concern, and three directories carry almost all of the behaviour this book describes.

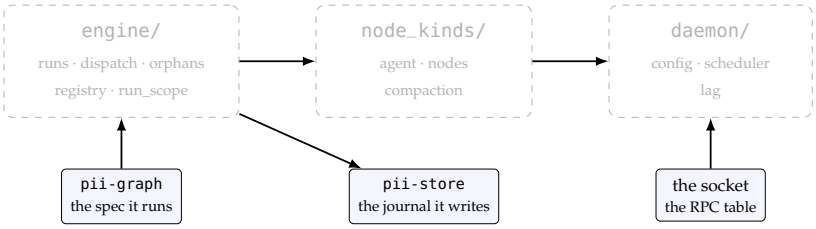


Figure 2.3: The three directories inside `pii-server` this manual spends its time in: `engine/` bridges a graph and the outside world, `node_kinds/` is the vocabulary, and `daemon/` owns the process.

`engine/` is the bridge. `runs.rs` accepts a graph and returns immediately after the header and the `Started` event are durable; `dispatch.rs` is the JSON-RPC table the socket reads; `orphans.rs` is boot recovery for agents a dead daemon left running; `run_scope.rs` builds the capability set and the filesystem scope for a run; `registry.rs` assembles the node registry, including a lint-only variant whose stores are in memory; and `refusal.rs` records a quarantine refusal as a decision rather than as a log line.

`node_kinds/` is deliberately not a flat directory of thirty files. `agent/` holds the turn loop and its tools; `nodes/` holds the kinds that touch the world, and `nodes/coder/` holds the parts of the coder that were extracted when the file crossed the line budget. `compaction.rs` is not a node kind at all — it is agent support used by the turn loop — and lives here because that is the only caller.

The daemon owns the process rather than the work: `config.rs` parses one TOML file with `deny_unknown_fields` on every section, `lag.rs` recovers runs a lagging bus left behind, and the scheduler owns the timer wheel. A run outliving a restart is not the daemon's problem; it is the journal's, which is why recovery lives in `engine/orphans.rs` and in `pii-graph`'s replay rather than in the daemon's start-up code.

2.5 Inside `pii-graph`

The graph crate is where a file becomes a runnable object, and its module names are the chapter titles of the next three chapters.

- `spec.rs` — the types: `GraphSpec`, `NodeSpec`, `Edge`, `EdgeKind`, `JoinPolicy`, `GraphLimits`.
- `toml_load.rs` — text to types, with the grant ceiling applied as the graph is built.
- `validate.rs` — the static rules, `V0` to `V11`, each with its own message.
- `fragment/` — inclusion: `call.rs` expands `call` nodes, `splice.rs` weaves `[[attach]]` fragments, `call_output.rs` rewrites host values and wires refusals back.
- `sched/` — readiness, edge states, dead propagation, dispatch of one node.
- `run/` — the tick loop, resume by replay, retry of a single node.
- `journal.rs` — the write-behind journal writer and its sequence numbers.

The order of that list is the order of a load, and it is worth keeping in mind when a message seems to come from the wrong place. A port error is raised by `validate.rs` after the splice, not by the loader, which is why `V1` names the node and not the file.

2.6 The two budgets

The first budget is arithmetic. `xtask/src/lint_files.rs` defines `MAX_EFFECTIVE_LINES = 450`, where "effective" excludes blank and comment lines, and the gate runs it over the workspace. Exceeding it fails the gate; the accepted fix is an extraction, not a waiver, and there is no per-crate override. Three extractions are visible in the tree as files that exist for no other reason:

- `daemon/sys_status.rs` and `store/sys_status.rs` — the same record from two sides: the daemon's view of system status and the store's persisted form, split when the health work of row 370 pushed a file over the line.
- `daemon/model_roles.rs` — the role-to-model mapping, extracted from the config parser.
- `nodes/coder/` — the coder's runtime selection, output handling, skills and container path, each in its own file.

The second budget is conceptual and harder to enforce: one idea gets one implementation. A copy is a defect even when both copies are correct today, because the only question that matters is what happens the first time one of them is fixed. Two rows in this repository exist because the rule was broken and then noticed:

- Row 359, "delete the copies": implementations that had been duplicated across crates were removed rather than kept in sync by hand.
- Row 371, "the idle daemon leaves no files": the lint registry used to open scratch databases at literal paths, which failed outside one specific checkout and silently dropped node kinds from the registry — so lint reported "unknown kind" for graphs that were fine. The fix moved the scratch stores into memory, which removed both the file and the failure mode.

That second one is a good example of the workspace's recurring shape: the bug was not in the rule, it was in the copy of the environment the rule assumed. A lint that needs a writable directory is a lint that behaves

differently on a host where the directory is not writable, and the difference shows up as a false refusal rather than as an error about the directory.

2.7 The gate

There is one step list, `gate-steps.txt`, and both the local gate and the CI-shaped gate read it. Each line is a label and a shell command; a step passes only if the command exits zero, and every step runs even after an earlier failure, so one run reports every problem rather than the first. The list currently has twenty steps, among them:

Step	What it refuses
`fmt`	any file not formatted by rustfmt
`clippy`	any warning, with `-D warnings`
`build`	a binary that does not link; later tests spawn the real `pii`
`test (lib/bins/tests), test (doc)`	a unit, integration or doctest failure
`lint-files`	a file over 450 effective lines
`lint-scratch`	scratch state written outside `_tmp`
`lint-observability`	an event name or metric outside the closed vocabulary
`reachability-check`	a public item nothing reaches
`deps-check`	a dependency outside the allowlist, or a forbidden edge
`net-check`	a network call outside the declared surface
`autonomy, plan-verify, item-ids`	prose drift in specs, plans and row ids

The gate has two modes that read the same list. The routine mode runs in the working tree, with a warm cache, and is what a developer runs while working. The fresh mode exports `HEAD` with `git archive` into a throwaway directory and runs the same steps there:

```
cargo run -q -p xtask -- fresh-gate           # export HEAD, run gate-steps.txt in a clean tree
cargo run -q -p xtask -- fresh-gate --worktree # same, from the worktree instead of HEAD
bash quality-gate.sh                          # routine mode, warm cache, current tree
bash quality-gate.sh --summary                # one line per step
```

The instruction that goes with it is blunt and was learned the hard way: when the two modes disagree, believe the fresh one. A gate can be green in a tree that has an untracked file, a stale target directory or a modified manifest, and red in a clean clone of the same commit. The fresh mode is the release boundary; the routine mode is a convenience.

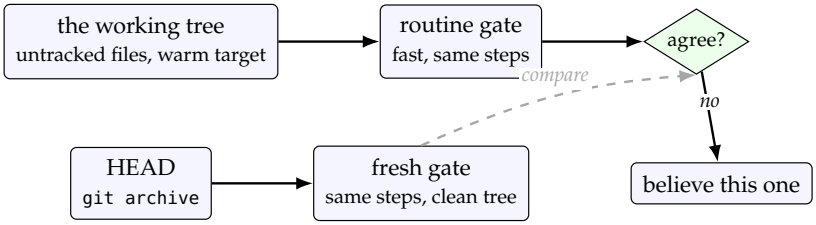


Figure 2.4: The two modes of one gate, which read the same step list on purpose, because a step added to one and not the other would report a different truth depending on how it was invoked.

2.8 Four binaries

The workspace produces four executables, and the split is by audience rather than by function.

- `pii` (`pii-cli`) — the operator's entry point, and the only one most users run. It embeds the daemon for commands that need one, and speaks to a running daemon over the unix socket for the rest.
- `pii-mcp` — the MCP server, so an editor or another agent can call the same tools the turn loop calls.
- `pii-web` — a small HTTP surface over the protocol, for reading runs and graphs from a browser.
- `pii-evolve` — the driver for the evolution loop, which proposes changes from recorded trajectories.

Everything else is a library. The optional surfaces — the TUI, the web viewer, the MCP server, the WASM plugin host, the hardware abstraction — sit behind their own features so that a build for a host without GPIO does not compile GPIO code. That is also why the gate lints the real GPIO backend without ever running it: code that is compiled but never exercised rots silently, and a lint is the cheapest thing that can still be true about it.

2.9 The coupling that is documented rather than removed

One edge inside `pii-server` is worth naming because it explains a design decision that looks unfinished. The JSON-RPC dispatch table in `engine/dispatch.rs` calls into `engine/methods.rs` for roughly ten methods — worker list and reload, approval list and resolve, heal list and get, cron list, channel list, memory search/put/consolidate, system reload, run resume. That is a real dependency from the engine-bridge concern into the daemon-application concern, and it is the reason a full split into `pii-engine`, `pii-nodes` and `pii-daemon` has not been done.

The alternative was a crate dependency edge that says the bridge depends on the application, which is exactly the kind of edge the workspace spends its `deps-check` step preventing. The decision recorded in the module header is to keep the coupling visible in one file, document it there, and defer the split to a reviewed change rather than pretend the boundary is clean. That is the general pattern in this workspace: where a rule cannot be satisfied yet, the violation is written down where the next reader will trip over it, not smoothed over in a comment elsewhere.

2.10 Finding a behaviour from a symptom

The question a new contributor actually asks is not "which crate owns X" but "I saw Y, where do I look". The table is the answer, and every path in it is a real file.

Symptom	Open first
A graph will not load, with a rule code	`crates/pii-graph/src/validate.rs` for `V*`; `fragment/call.rs` for `graph.call.*`; `fragment/splice.rs` for `fragment.*`
A call refuses on capabilities	`fragment/call.rs`, the `widen` checks: one for the call's grant, one per callee node
A queue row stays `Running`	`crates/pii-store/src/queue.rs` for the status transitions; `graphs/queue.toml` for the edges that decide whether a verdict ever reaches `record`
A run never finishes and its journal stops mid-graph	`crates/pii-graph/src/run/engine.rs` (the tick loop) and `sched/state.rs` (`readiness`, `propagate_dead`)
A run resumed after a restart re-ran work	`run/resume.rs`: a node with no terminal event is re-run by design; look for the `NodeInterrupted` marker
Health is slow	`engine/conn.rs` for the heavy-method set; `daemon/identity.rs` for the record it should be reading
An agent left running after a crash	`engine/orphans.rs`, and the `coder.orphan_reaped` event it journals
A configuration key seems ignored	`daemon/config.rs`: every section is `deny_unknown_fields`, so an ignored key is a load error, not a default

Two habits make that table faster to use. The first is to grep for the rule code rather than the prose: every refusal string is a literal that appears in exactly one place, so `grep -rn "graph.call.widen" crates/` lands on the check. The second is to read the graph file before the code. Most "engine" bugs in this repository have been edge bugs — a port with no edge, an edge to a node that can never be ready — and the graph is a page long.

```
grep -rn 'graph.call.widen' crates/           # which check raised it
grep -rn 'V1:' crates/pii-graph/src/validate.rs # the port rule and its
    ↪ message
pii graph lint graphs/queue.toml              # the same rules, over one
    ↪ file
pii graph show graphs/spec.toml               # nodes, ports and edges
    ↪ after expansion
pii run trace <run_id>                        # the journal, in sequence
    ↪ order
```

The third and fourth lines are the ones to reach for first when a graph is in question. `pii graph lint` runs the loader and the static validator and prints every warning as well as every error, and the warnings are where a missing `err` edge shows up: a declared port with no outgoing edge is a warning, not an error, because a graph may legitimately stop on it. `pii graph show` prints the graph after calls are expanded — which is the only honest way to see what will actually run for a graph that composes.

3 The engine

This chapter answers what a run is. It covers the five states a run can be in, what happens between the moment a graph is accepted and the moment its terminal is durable, how the scheduler decides which nodes may run together, what the journal records and when it is flushed, and how a run that was interrupted is resumed rather than restarted. The engine is the part of pii that must not be interesting: it starts a run, walks the graph, and writes down what it decided. Its complexity is almost entirely in the two properties that follow from that — a run must not hang, and a run must not lose work it already did.

Both properties cost something. The no-hang property is why every arm of the tick loop returns and why a node that can never become ready fails the run by name instead of waiting. The no-loss property is why the journal is write-behind rather than write-through, why the terminal event is appended before the status is flipped, and why the resume path replays the whole journal instead of trusting a saved position.

3.1 The record of a run

A run is identified by a `RunId` and described by a `RunHeader` that is written when the run is accepted. The header is small on purpose: it is read by every list command and by every watchdog, so it must be one record and not a fold over the journal.

Field	Meaning
`run`	the run id; the key every other table uses
`graph`	the graph name, or the hash of its body when it has no name
`status`	one of the five states in the next section
`seq`	the next journal sequence number to be written
`dur_ms`	wall-clock duration; zero while running
`revision`	the build revision of the binary that accepted the run
`process_start_ms`	when that process started, so a restart is visible
`created_ms`	when the run was accepted

The last two fields exist because of an operational question that a single timestamp cannot answer: “was this run started by the process that is running now”. Without `revision` and `process_start_ms`, a run that fails after a deploy is indistinguishable from a run that failed before it, and a bug that was fixed looks like it is still there. Rows written before the field existed decode with an empty revision, and are counted as unattributed rather than as a deploy.

3.2 Five states

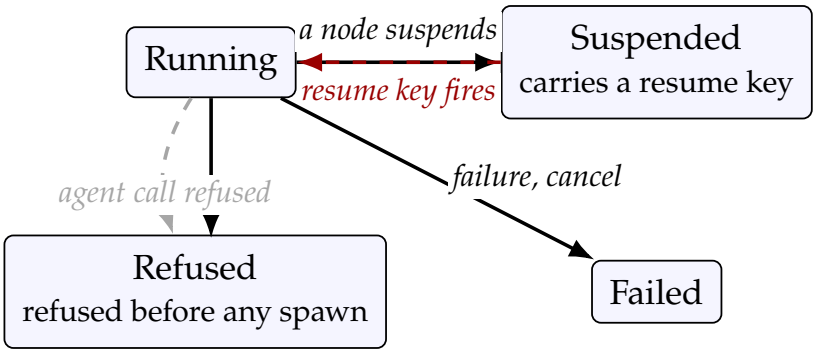


Figure 3.1: The whole life cycle; there is no *Pending* and no *Cancelled* state: a run exists only once it has been accepted, and a cancelled run is closed as *Failed* with `cancelled:` in its error text, which is what the record has always meant.

- **Running** — executing, or waiting to be resumed by a caller that already holds a key. It is also the state a cancelled run is left in if the process dies mid-cancel, which is deliberate: recovery finds it.
- **Suspended** — the run parked itself on a resume key. A timer, for instance, never blocks a thread; it journals a `Suspended` event and returns the key the scheduler will fire.
- **Succeeded** — the run reached a terminal disposition with a value.
- **Failed** — the run ended in an error: a node failed terminally, a limit was exceeded, a join could never be satisfied, or an operator cancelled it.
- **Refused** — an agent call was refused before any process was spawned. It is distinct from *Failed* because nothing was attempted: a refusal is a decision, and the store records it as one.

The absence of a *Cancelled* status is not an oversight. Cancellation has two forms, and neither needs a sixth state. A run that is not executing is closed as *Failed* with `cancelled: <reason>` in its error, which is what an operator reading the record needs to know. A run whose node *is* executing is refused by default — the daemon answers `run.cancel_refused` and names the node — because cancelling would abandon work half-done. The explicit kill form accepts that, waits up to thirty seconds for the node to leave flight, and still leaves the run open if it has not: a run is never closed while its agent may still be running.

3.3 Starting a run

`spawn_run` returns in under fifty milliseconds, and everything that must be true before the caller gets an id is done on the calling thread. The order matters, because each step can refuse and the refusal has to leave a coherent record.

1. The runtime is checked. A start from a plain thread is refused by name rather than panicking on a run thread, which would crash-loop a supervisor.
2. The run is registered as queued, with its own cancel token, so a cancel that arrives before the pool picks it up is not lost.

3. An optional `scope_root` in the input confines the run's write scope to one directory. An invalid root is rejected here, before anything runs, rather than producing a permissive run.
4. The context is built with the graph's `[limits]` as a real budget — tokens, tool calls, micro-dollars and wall clock — and with the capability set the run's scope allows.
5. The header and the `Started` event are written and flushed, so the id the caller receives is already durable.
6. A quarantine check may refuse the run; the refusal is recorded as a deny decision before the error is returned.
7. The work is submitted to the run pool. The closure checks whether the run was cancelled while queued and returns without executing if so.

The fifth step is the one with a visible consequence: because the header is durable before the caller returns, a run id can be used immediately — to subscribe, to cancel, to read the trace — without a race against the pool.

```
pii run start queue --input '{"workdir":"/projects/pi-improved","base
    ↪ ":"<40-hex>"}'
pii run ls --status Suspended          # what is waiting, and on what
pii run get <run_id>                   # header: graph, status, duration
pii run trace <run_id>                 # the journal, in sequence order
```

3.4 The tick loop

The loop is one function, and its shape is the answer to "how does a graph make progress". Each iteration either dispatches a batch, or ends the run. There is no arm that waits.

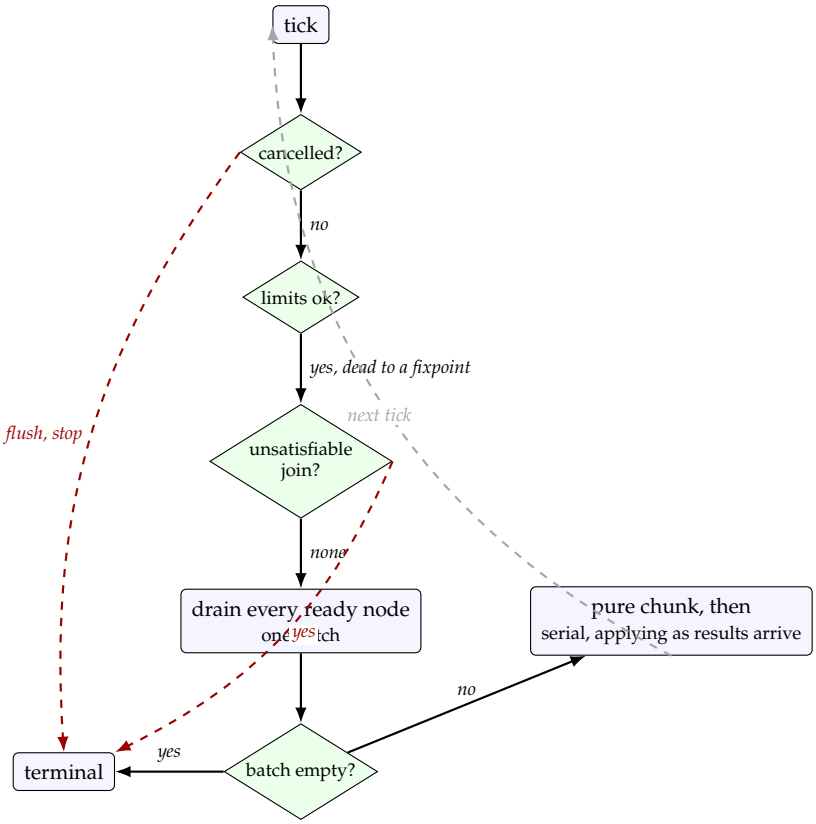


Figure 3.2: One iteration; the two red exits are the no-hang guarantee: a cancellation and an unsatisfiable join both end the run inside the same tick, with a recorded reason.

In order, one iteration does this:

1. **Cancellation.** If the run's token is set, the journal is flushed and the run stops. The flush is not optional: without it a cancel before the write-behind threshold would lose every completed node, and a resume would re-run work whose side effects already happened.
2. **Limits.** The step count and the wall clock are checked first, then the three consumable budgets. A budget exhausted by any node is visible here on the next tick, because nodes share the run's budget rather than holding their own.
3. **Dead propagation.** A node all of whose inbound data edges are dead is itself dead, and so on to a fixpoint. Each newly dead node journals a `NodeDead` event, so a branch that was skipped is distinguishable from a branch that never existed.
4. **Unsatisfiable join.** A join that can never be satisfied fails the run immediately and names the node. This is checked every tick rather than at the end, because a quorum that has already lost too many arms cannot recover.
5. **The batch.** Every node that is ready and not already executed, dead, failed or cancelled is drained into one batch. The batch is journaled as a `TickStarted` event with its width, which is what makes a wide fan-out visible in a trace.
6. **Budget and entry checks at dispatch.** A node whose `loop_budget` is already spent, or a call head entered more than `max_entries` times, fails the run here — before the batch runs, not after.
7. **Inputs frozen.** Each node's input is built from the current edge state and the blackboard, once, before any node in the batch runs.

A node therefore cannot observe a sibling's output from the same tick.

8. **Partition.** Nodes whose effects are parallel-safe run in bounded chunks; nodes that write the filesystem or mutate an external system run one at a time, after the parallel batch.
9. **Apply as results arrive.** Each result is applied to the scheduling state immediately, and race losers are cancelled after *each* result rather than at the end of the batch, so a losing branch does not keep burning a timer.
10. **Restart or end.** A terminal failure recorded mid-batch ends the run now; a suspension ends it as *Suspended*; otherwise the loop iterates.

The choice to drain *all* ready nodes rather than one per tick is what makes the loop's cost proportional to depth rather than to nodes. It also means a graph's fan-out is a real budget question: a hundred pure nodes ready at once will use up to `max_parallel` threads, and the single-element chunk case is dispatched inline precisely because spawning one OS thread per node showed up as retained memory under soak load.

3.5 Readiness, and the four joins

A node is ready when its inbound edges allow it. The rules are ordered, and the order is the part that is easy to get wrong.

1. The entry node is ready on its first pass even if it has inbound edges. A loop's back edge is pending at the start, and without this exception a cyclic graph would deadlock on tick one.
2. Any pending *order* edge makes a node not ready. Order edges gate readiness before data is considered at all.
3. Zero inbound data edges means ready. This must be tested before the join arms, because "all of nothing" is true and would otherwise report the entry as dead.
4. Otherwise the node's join policy decides.

Join	Ready when	Never runs when
`all` (default)	every inbound data edge fired	every edge dead (dead node), or any edge dead while others fired (unsatisfiable)
`any`	at least one edge fired	every edge dead
`any:cancel`	as `any`, and the losers are cancelled once one wins	every edge dead
`quorum:K`	at least *K* edges fired	fewer than *K* can still fire
`allsettled`	every edge settled, fired or dead	every edge dead

The difference between `all` and `allsettled` is the difference between "all my inputs succeeded" and "all my inputs are decided". A node that must proceed whether or not one branch died uses `allsettled`; a node that needs every value uses `all`. The static validator refuses an `any` or `quorum` join with fewer than two inbound data edges — a one-armed race is a typo — but deliberately does not over-enforce a single-armed `allsettled`, which is a legitimate idiom.

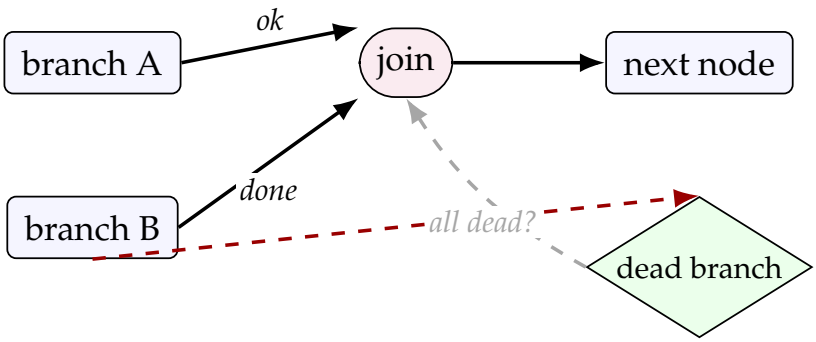


Figure 3.3: Why the join policy is part of the graph rather than a scheduler default; two branches may both feed one node, and whether that node waits for both, for one, or for both to be *decided* is a fact about the graph that no scheduler can infer.

3.6 The journal

The journal is the primary evidence, and it is written by one component with one sequence counter. Every event is appended with a sequence number computed from the header and the pending buffer, the observer is notified immediately, and the event is buffered for a durable write in one transaction with the header.

Event	What it records
`Started`	the graph name and the hash of the run input
`TickStarted`	the batch of nodes drained in one tick, and its width
`NodeStarted`	a node began, at a millisecond offset
`NodeFinished`	the node, the port it took, the value, the value hash, offset and duration
`NodeFailed`	the error, whether it is retryable, and the attempt number
`NodeProgress`	a heartbeat: elapsed time and the child's CPU, so a wedged node (time grows, CPU flat) reads differently from a working one
`NodeCancelled`, `NodeDead`	a race loser, and a node killed by dead-branch propagation
`NodeInterrupted`	an audit marker written on resume for a node the previous attempt left mid-flight; it carries no state transition
`Suspended`, `Resumed`	the resume key, and the hash of the payload that delivered it
`NodeObserved`	an event a node's agent emitted, recorded from the agent's own events rather than from the node's return value
`EffectStarted`, `EffectConfirmed`	an idempotency key, written *before* an effect and confirmed after it
`NodeRetried`	an operator re-ran one node; earlier terminal events are history
`RunSpawned`	a `spawn` node launched a child run
`Finished`	the terminal and the hash of its output

Two of these groups carry more weight than the rest. The `EffectStarted` / `EffectConfirmed` pair is the only defence against an effect that hap-

pened but was not recorded: a key that was started and never confirmed is reported as `effect.unknown`, and a resume refuses the run rather than re-performing an effect whose outcome nobody knows. And `NodeObserved` is how the journal learns what an agent did without trusting the node's return value: the fields are the agent's event fields, copied, not paraphrased.

Write-behind, and the two moments it is flushed

Journaling is buffered. The writer keeps a pending list, flushes when the list reaches its batch threshold or when the sequence number crosses a snapshot boundary every thirty-two events, and flushes unconditionally at three points that matter:

- before a terminal is recorded — the `Finished` event is appended *before* the status is flipped, so a reader that polls the header cannot see a finished run while the terminal event is still in the buffer;
- when a run is cancelled, so completed work survives;
- on resume, before anything new is appended, so the replayed prefix is the durable one.

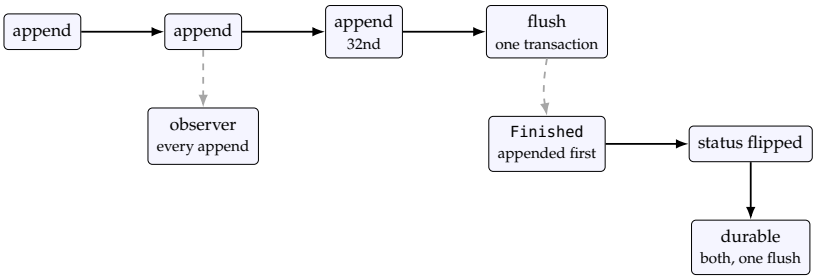


Figure 3.4: Two clocks in the writer: the live view advances on every append, the durable view on a flush, and the terminal path appends before flipping the status so that both land in one transaction.

The failure mode this design accepts is bounded and stated: a process that dies between two flushes loses the buffered events, and the nodes they described will run again on resume. That is why effects are journalled separately and before they happen — the one category where "run again" is not acceptable gets its own protocol.

3.7 How a run ends

When the batch is empty, the engine decides the terminal. Three rules, in order:

1. If a failure was recorded during dispatch, it is the terminal. Nothing overrides it.
2. If any node is neither executed, dead, failed nor cancelled and is not ready, the run fails with `no progress: node '<id>' never became ready`. This is the message an operator sees when an edge is missing, and it names the node rather than the edge, because the edge is usually the thing that is not there.

3. Otherwise the output is the value of the latest finished node that has no outbound data edge on the port it took — the value nobody consumed. If several qualify, the latest by sequence wins.

The third rule has a consequence that surprises graph authors: a node that leaves on a non-`ok` port carrying an error makes the run a failure even if the graph "reached the end". The check is deliberate. A success terminal that carries an error is the shape of a bug that reports green while the work failed, and it is refused at the point where the terminal is built rather than left for a reader to notice.

3.8 Suspension and resume

A node that must wait — a timer, an approval, a peer message — does not block. It returns a suspension: a reason and a key. The run is finished as *Suspended*, and something outside the engine owns the key: the timer wheel fires it, an operator resolves the approval, a peer message arrives.

Resuming is not "continue from where we were", because there is no "where we were" after a restart. It is a replay:

1. The journal is read in sequence order and the scheduling state is rebuilt: which edges fired, which nodes executed, which are dead.
2. A node with a `NodeFinished` is never re-invoked; its value is in the journal and goes back on the blackboard.
3. A node with a `NodeStarted` and no terminal event *is* re-run. This is the documented boundary: the engine knows the node began and does not know whether its side effect happened, so it runs it again unless the effect protocol says otherwise.
4. Before any of that, an `EffectStarted` with no matching `EffectConfirmed` refuses the resume with `effect.unknown`.
5. A confirmed effect whose `NodeFinished` never landed is completed from the journal rather than re-performed.
6. The nodes the journal shows as still suspended are dispatched through the resume entry point rather than a fresh run, so a fired timer resumes to "next" instead of suspending again.

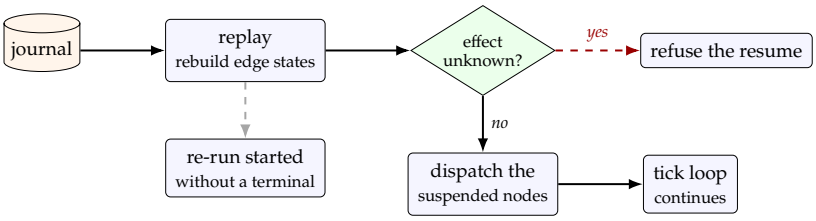


Figure 3.5: Resume is a replay with two exceptions — finished nodes are not re-run, and suspended ones get their own entry point — so everything else is reconstructed from events.

3.9 Recovery at boot

A restart is the normal case, not the exceptional one. Three mechanisms cover it, and each has a distinct failure it was built for. The operator's side of all three is small:

```
pii run resume <run_id>                 # replay the journal and continue
pii run retry <run_id> --node <id>      # re-run one node; its earlier
    ↪ events stay as history
pii run abandon <run_id> --reason '...' # close a run nothing can finish,
    ↪ with a reason on record
pii run cancel <run_id> [--kill]        # refused while a node executes,
    ↪ unless killed
```

Agents left running. A daemon that dies in the middle of an agent leaves the agent with no owner. Before the runs it left *Running* are closed, boot recovery kills each agent it started: a container by its `io.pii` and `pii.run` labels, a host process by the pid journalled while `/proc` still shows that pid running the same program. Every reap is journalled on its run as `coder.orphan_reaped`, and the run's failure text says an agent was killed — so the operator sees why the run ended, rather than an unexplained failure.

Suspended runs with no timer. A timer row used to be deleted before the resume was spawned, so a process that exited in between left a run *Suspended* with nothing that would ever fire. The fixes are small and together they close the hole: the row is deleted only after the resume is spawned, a failed spawn puts the row back or fails the run with `run.resume.failed`, and the daemon calls the existing recovery pass once at boot rather than only after a lagging bus event. A run must never be left suspended with no timer and no event saying so.

Queue claims. A row claimed by a run that then died used to block the queue for ever, because a `Running` row makes `claim_next` refuse to hand out anything. The claim is now a lease: the item records who claimed it and when, and it is requeued to `pending` with a `requeued_after_restart` event when the claiming run is terminal or no longer exists. It is *not* requeued while its run is alive or suspended, which is the near-miss control the fix needed: a suspended run still blocks its row, because its timer will resume it.

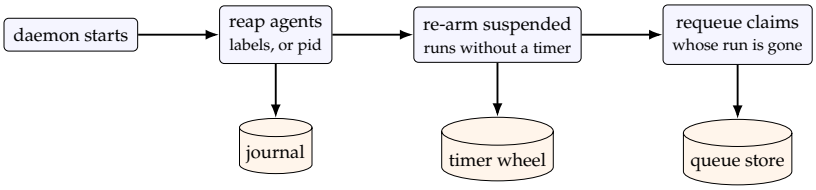


Figure 3.6: Boot, in order: each step writes to the store that owns the fact, and nothing is fixed in memory alone.

3.10 Failure signatures

The table is the practical part of this chapter: what a failure looks like in the record, and what it usually means. Every signature is a real string or event in the code.

What you see	Where	Usually means
`no progress: node 'X' never became ready`	terminal error, status *Failed*	an inbound edge that can never fire; a branch that dead-ended
`join can never be satisfied: node 'X'`	terminal, within one tick	two arms on mutually exclusive ports of one router
`max_steps exceeded`	terminal, status *Failed*	a cycle without a `loop_budget` that validation did not catch, or a node that keeps re-entering
`max_wall_ms exceeded`	terminal	a blocking node with no timeout of its own
`loop_budget exceeded for node 'X'`	terminal, mid-batch	the budget is exactly the number of allowed runs; the failure is the run after it
`call 'X' entered more than max_entries`	terminal, before dispatch	an inner call inside an outer loop, whose count is cumulative for the run
`effect.unknown`	refusal on resume	an effect began and was never confirmed
no `NodeFinished` after a `NodeStarted`	journal tail	the process died mid-node; the node will re-run
a run *Running* with a dead process	header, plus reaps	crash; boot recovery closes it after killing the agent

Two of these deserve a note. `no progress` is the most common message in this repository and almost always means an edge is missing rather than that the scheduler is wrong; the node it names is the one that could not proceed, not the one that failed to feed it. And `loop_budget exceeded` fires on the run *after* the allowed number, so a budget of three produces exactly three executions and then a failure — which is the behaviour a bounded loop wants, and the reason the check is at dispatch rather than at re-entry.

3.11 The limits, as a table

Key	Default	What it bounds
`max_steps`	1000	scheduler iterations, not node runs; a wide graph uses few
`max_wall_ms`	600 000	elapsed time from acceptance
`max_tokens`	unbounded	tokens charged by agent nodes, shared by the run
`max_tool_calls`	unbounded	tool calls across the run; exhaustion routes the agent to its `budget` port
`max_micro_usd`	unbounded	spend, in micro-dollars
`max_parallel`	0	parallel-safe nodes per tick; zero means the host's available parallelism

The defaults are chosen so that an unattended run cannot loop for ever or spend without a ceiling, while a graph that says nothing still behaves like a normal program. A graph that needs longer says so in its own file, where the number is reviewable — which is the whole reason the limits are part of the graph rather than a daemon-wide setting.

Node kinds and ports 4

This chapter answers what a node may declare and what it may do. A node is a `kind` plus configuration, and its *ports* are the outcomes it can hand back to the graph. The ports are part of the contract rather than a detail of the implementation: an edge that leaves a port the node does not declare is refused when the graph is loaded, and a declared port with no outgoing edge is reported as a warning, because that is the shape of a missing failure path. The chapter then covers the ladder that bounds what a node may *do* — three ceilings, checked at three different moments — and ends with the two mechanisms that make a node's outcome meaningful: the `expect` mode of a judge and the effect declaration that decides whether a node may run beside another.

The reason to spend a chapter on this is that a graph is written in this vocabulary and nothing else. There is no expression language to fall back on, no way to compute a port name at run time and no way for a node to invent an outcome. When a graph misbehaves, the question is almost always which port fired and whether the edge that should have carried it exists.

4.1 A node, and the four things it declares

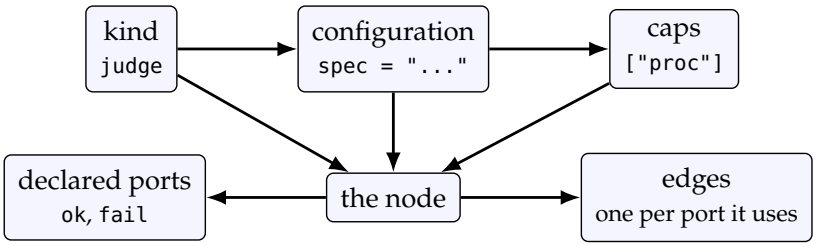


Figure 4.1: A node's four declarations; the factory derives the port set from the kind and the configuration, which is why a judge in fail mode has a different port set from the same judge in pass mode.

- **kind** — one of the registered factories. The registry is assembled per process; the built-in seven are always present, and everything that touches a model, a container, a store or a channel is registered by the server.
- **configuration** — a JSON object. Unknown keys are refused by the factory that reads them, not silently ignored, which is why a misspelled key is a load error rather than a node that behaves like its default.
- **caps** — what the node needs. It is a request, not a grant: the graph must hold it, and the host must hold what the graph holds.
- **join** — how many inbound data edges must fire before the node runs. Default `all`; the alternatives are covered in Chapter 3.

Everything else on a node is optional and bounded: `loop_budget` for a node a cycle may re-enter, `retry` for a node that may fail transiently, `timeout_ms` for a node that spawns a process.

4.2 The ports, by kind

The table is the vocabulary. Every entry was read from the factory's `ports` method, and the last column is the outcome a graph author is usually looking for when a run ends somewhere unexpected.

Kind	Ports	When each fires
`transform`	`ok`, `err`	`ok` with the evaluated value; `err` when the expression cannot be evaluated against the input
`route`	from its `rules`	one port per rule, declared in configuration; at least one is required, and the first matching rule wins
`emit`	`ok`, `err`	writes or appends a file within the write scope
`wait`	`fired`, `timeout`	sleeps in five-millisecond slices, checking cancellation; it blocks a thread and is bounded by its own timeout
`timer`	`fired`, `timeout`	never blocks: it suspends the run on a key the timer wheel owns
`require`	`ok`	passes when its condition holds; otherwise the node fails
`refuse`	none, or `ok`	fails the run by name — unless the load gave it `returns`, in which case it returns a verdict on `ok`
`agent`	`ok`, `err`, `budget`, `fail`	`budget` when the tool-call budget is exhausted; `fail` when the turn loop ends without a result
`coder`	`ok`, `err`	spawns an agent against a work directory; `err` when the agent could not start or exited non-zero
`tool`	`ok`, `err`	calls one tool from the sandbox surface with fixed arguments
`gate`	`allow`, `deny`	consults the approval store; `deny` is a outcome, not a failure
`select`	`ok`, `blocked`, `empty`, `running`	claims the next queue row; `blocked`, `empty` and `running` are three different reasons there is no row
`judge`	`ok`, `fail` (and `unrunnable`)	runs a spec's criteria; `unrunnable` only in fail mode, for a command that could not run at all
`record`	`ok`	writes the arriving verdict to the queue store
`plan`, `enqueue`	`ok`, `err`	asks a planner for a spec, and admits a written spec to the queue
`memory`, `remember`, `recall`	`ok`, `err`	the memory and entity stores
`spawn`	`ok`, `err`, `escalate`	starts a child run; `escalate` is the outcome that asks a human to look
`post`, `mqtt`	`ok`, and `mqtt` also `err`	outbound channels; only the MQTT side declares a port for a failed send

Kind	Ports	When each fires
`sop`	`ok`, or none	a compiled procedure step; the `fail` operation declares no port
`classify`, `hal`, `wasm`, `debug_crash`	`ok`	single-outcome kinds
`heal_detect`	`ok`, `detected`, `flake`, `inconclusive`	the self-healing detector's four verdicts

Two rows deserve emphasis because they are the ones graph authors get wrong. `select` has four ports, and the three non-`ok` ones mean different things: `empty` is "no row is pending", `blocked` is "a row is waiting on another row", and `running` is "a row's run is still alive". Collapsing them into one failure port is the mistake that makes a queue look wedged when it is merely idle. And `refuse` has *no* ports in its ordinary form — not a port that never fires, but an empty set — which is the fact at the centre of Chapter 5.

4.3 One kind, two port sets

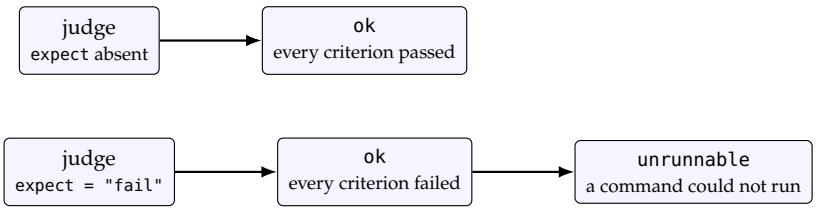


Figure 4.2: The same factory, two contracts; the `unrunnable` port exists only in fail mode, because in fail mode a command that never ran would otherwise be counted as a pass — which is the exact defect the mode was added to prevent.

The fail mode is worth stating precisely, because its whole value is that it cannot be satisfied by accident. A judge with `expect = "fail"` reports `ok` only when *every* criterion fails, and `fail` when any criterion passes. A criterion that could not run at all — a missing binary, exit 127 — is neither, and is reported on `unrunnable`. The default stays `pass`, so every graph written before the mode existed is unaffected.

The reason this matters is the one the TDD graph is named for. "The test fails before the code exists" is the only evidence that the test tests anything; without a fail-mode judge, a test written after the implementation and a test written before it are indistinguishable in the record. The graph that uses it is four nodes and a loop:

```
[[node]]
id   = "write_test"
kind = "coder"

[[node]]
id     = "red"
kind   = "judge"
expect = "fail"

```

```
[[node]]
id   = "implement"
kind = "coder"

[[node]]
id   = "green"
kind   = "judge"
```

The edges are where the loop's meaning lives: `red.ok` leads to `implement`, `red.fail` leads to `not-red` — the test passed before the code existed, so the test proves nothing — and `green.fail` leads to `still-red`. Both are named nodes rather than a generic failure, because an operator reading the trace needs to know which of the two happened.

4.4 The capability ladder

Capabilities descend. A node may hold only what its graph holds, and a graph may hold only what the host grants. Each of the three levels is checked by different code at a different moment, which is why a capabilities problem can produce three different messages.

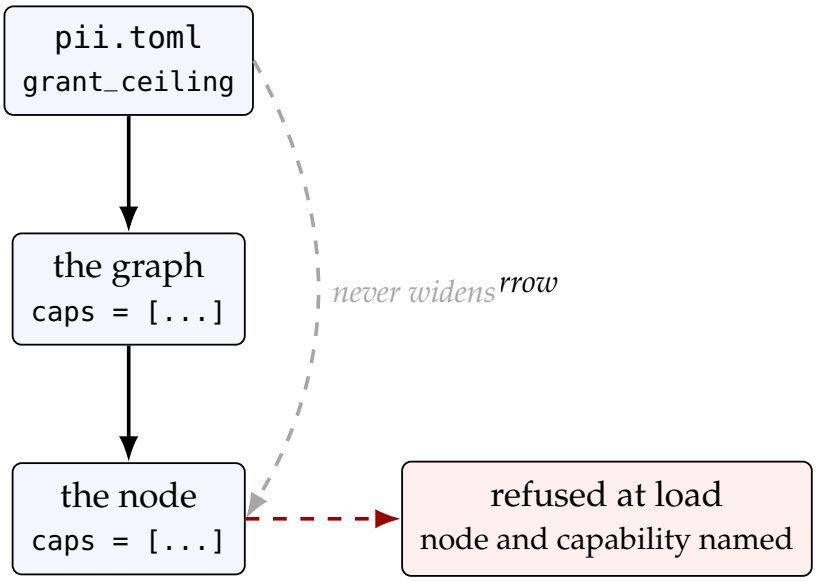


Figure 4.3: Three rungs, one direction; the ceiling is an operator decision, the graph's list is a design decision, and the node's list is a claim about one piece of work; each may only be narrower than the one above it.

Check	Where	Message
graph and node against the ceiling	`toml_load.rs`	`graph.grant.ceiling` names the graph or node and the excess
node against the graph	`validate.rs`, rule `V2`	"node 'X' requires capability 'Y' not granted by the graph"
callee against the call's grant	`fragment/call.rs`	`graph.call.widen` names the node, the capability and the call

The twelve capability bits are stable by contract: their values must not change, because they are compared bitwise across levels. Their names are what appears in a refusal, and a refusal that names one capability is worth more than a refusal that says "denied".

Name	Allows	Name	Allows
`fs_read`	reading within the allow scope	`fs_write`	writing within it
`net`	reaching the network	`proc`	talking to processes
`hw`	touching hardware	`secrets`	reading secret material
`memory_read`	recalling from memory	`memory_write`	writing to memory
`spawn`	starting subprocesses	`post`	posting into a channel
`mqtt_read`	listing and subscribing devices	`mqtt_write`	commanding a device through its allowlist

The practical consequence of the ladder is measured rather than theoretical. On 2026-11-03 a coder inside a graph with `caps = ["spawn", "proc"]` had no write tool at all: the agent started, found no way to write, stopped, and said so. That is the behaviour the ladder wants — a refusal that names the missing capability beats a silent degradation in which the agent improvises with the tools it happens to have. The graph now grants `fs_read` and `fs_write` explicitly, and the comment beside the grant records the measurement.

4.5 Effects: what may run beside what

Every factory declares its effects, and the scheduler reads that declaration to decide whether two nodes in the same batch may run concurrently. The categories are coarse on purpose:

- **pure** and **reads the filesystem** — parallel-safe. These may run in a chunk of up to `max_parallel` threads.
- **writes the filesystem** and **mutates an external system** — sequential. They run one at a time, after the parallel batch, in declaration order.

One exception is written into the validator rather than into the effect table. A `coder` spawns an agent for as long as it likes while its factory declares only "mutates an external system"; declaring "spawns a process" instead would make every coder node non-parallel-safe and change the scheduling of graphs that never had a problem. The validator therefore judges a coder by its *kind*: anything that is a coder, or that declares "spawns a process", must carry a positive `timeout_ms`, and the message is `V10`. A node that blocks on a child process without a bound is the shape of a run that never ends.

The other validator rules that concern a single node are worth having in one place:

Rule	What it refuses or warns
`V0`	duplicate node id, unknown kind, an edge naming a node that does not exist, an entry that is not a node
`V1`	an edge leaving a port the node does not declare
`V2`	a node requiring a capability the graph does not hold
`V3`	a cycle with no `loop_budget` among its members
`V5`	warning: a node unreachable from entry; warning: a declared port with no outgoing edge
`V6`	an `all` join whose arms sit on mutually exclusive ports of one router
`V9`	an `any` or `quorum` join with fewer than two inbound data edges
`V10`	a process-spawning node without a positive `timeout_ms`
`V11`	warning: a coder's `err` edge into a terminal `transform`, which would finish the run as succeeded although the agent failed

`V11` is the warning a graph author should read most carefully, because it describes a run that reports green while the work failed. The terminal-transform shape is tempting: route the error to a node that formats a message, and the graph "ends cleanly". The validator cannot refuse it — a transform is a legitimate last node — so it warns, and the warning is the last cheap moment to notice.

4.6 Choosing a kind

The decision is usually made by asking what the step needs, in this order.

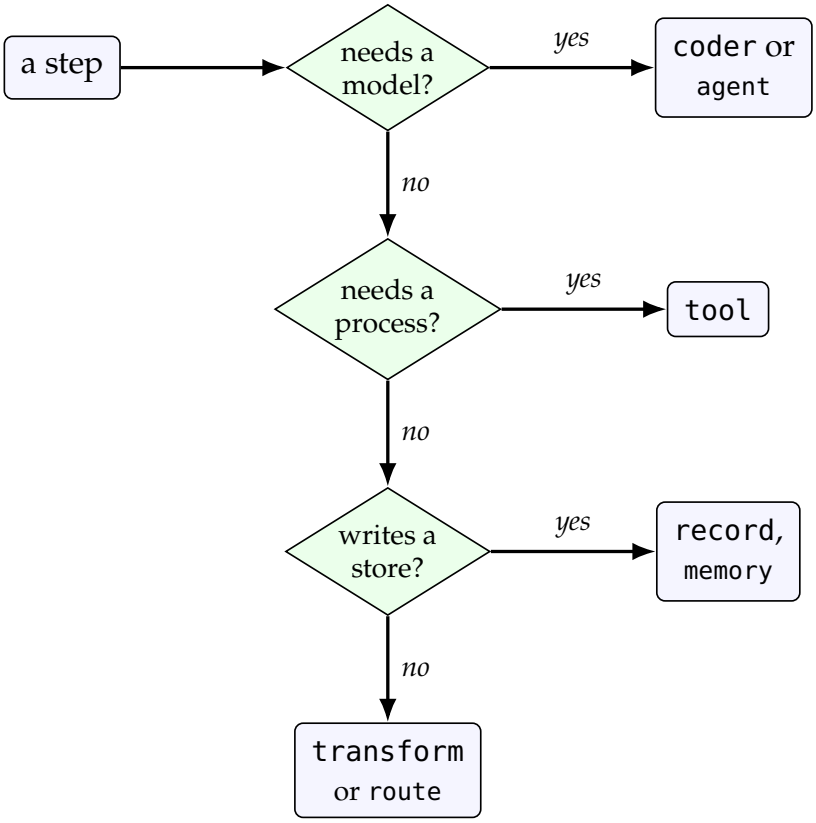


Figure 4.4: The order the checklist uses; the last answer is the important one: a step that needs no model, no process and no store is a `transform`, and writing it as a coder is how a graph acquires a model dependency it never needed.

The checklist that goes with the figure is the one in the graph library's own guide, restricted to the kind question: does an existing graph already do this (call it, do not copy it); does the step need a model (coder or agent); does it need a process (tool); does it write a store (record, memory); otherwise it is a transform. The last line is where most over-engineering is caught. A step that reshapes a value does not need an agent, and giving it one costs a model call, a timeout, a capability and a failure mode.

4.7 Three ways a step waits

A graph is mostly waiting, and the kind of wait decides what a run costs while it waits. The three mechanisms are distinct and are chosen by the kind, not by the author.

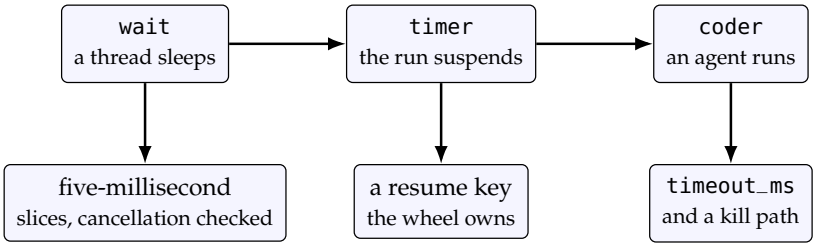


Figure 4.5: The cost of waiting, left to right; the first form holds a thread, the second holds nothing but a journal entry, and the third holds a process that must be bounded and, at boot, reaped.

- **A blocking wait.** `wait` sleeps in slices of at most five milliseconds and checks cancellation between them, so a cancel is observed promptly rather than after the full duration. It occupies a dispatch thread for its whole duration, which is acceptable for seconds and wrong for hours.
- **A suspension.** `timer` never blocks. It returns a suspension with a reason and a key of the form `timer:<node>:<fire>`, the run is journalled as *Suspended*, and the timer wheel owns the key. The run occupies nothing while it waits, which is what allows a nightly graph to sit for hours without holding a thread.
- **A child process.** A coder or agent starts something that runs for as long as it likes, so it must be bounded twice: by `timeout_ms` on the node (enforced by the validator as `V10`) and by the run's wall clock. If the daemon dies first, boot recovery reaps the agent by its container labels or by the pid it journalled, and records `coder.orphan_reaped` on the run.

The choice is not stylistic. A `timer` inside a fragment is refused outright: fragments may not own a scheduler, so a fragment that would wait is refused with `fragment.autonomy.forbidden` rather than silently suspending a host that did not ask for it.

4.8 A worked grant

The queue graph is the shortest honest example of the ladder in use, because every rung is visible in twenty lines. The graph holds four capabilities; the call to the spec graph grants the same four; the coder inside the callee asks for the same four. Nothing is wider than the rung above it, so the load succeeds — and the comment says why the two filesystem bits are there at all.

```
caps = ["spawn", "proc", "fs_read", "fs_write"]   # graph level

[[node]]
id    = "implement"
kind  = "call"
graph = "spec"
caps  = ["spawn", "proc", "fs_read", "fs_write"]   # the call's grant
```

If the call had granted three of the four, the load would have failed with `graph.call.widen` naming the coder inside `spec` and the missing bit. That is the behaviour to want: the graph that is wrong is refused before it runs, and the message names both the node that needed the capability and the list it exceeded.

4.9 Reading a node's failure

A node failure is journalled as `NodeFailed` with the error text, whether it is retryable and the attempt number. Three things follow from that record and are worth knowing when reading a trace:

- A retryable failure with attempts left does not fail the run; the node is attempted again according to its `retry` policy.
- A node that fails terminally marks its outbound data edges dead, which propagates: downstream nodes that depended only on it become `NodeDead` rather than failing themselves. The trace therefore shows one root failure and a set of deaths, which is the shape to look for.
- A node that leaves on a non-`ok` port is not a failure at all unless the port says so. `gate.deny`, `select.empty` and `judge.fail` are outcomes, and a run that ends on one of them ends as a success carrying that value — which is why the terminal rule (Chapter 3) refuses a success terminal that carries an error.

The distinction between "a port that means no" and "a node that broke" is the whole of graph authoring. The kinds in this chapter exist to make the first category expressible, so that the second one stays rare enough to be worth investigating.

Composition 5

This chapter answers how graphs become each other's parts. There are two inclusion mechanisms and they share one property: both decide everything at *load* time. A `call` node is replaced by the callee's nodes; an `[[attach]]` table weaves a fragment into a host at a named surface. Nothing is resolved during a run, nothing crosses a process boundary, and there is no second implementation of "include". That single decision is what makes the rest of the chapter necessary, because it also means the rule that decides where a call returns is a rule about graph shape rather than a rule about invocation.

The chapter spends most of its length on one such rule, because it is the sharpest edge in the system. A call returns through every node that declares an `ok` port and has no outgoing `ok` edge — not through the node the callee names in `exit`. A `refuse` node declares no ports at all, so it is not an exit under that rule, and a callee that ends in a refusal used to dead-end inside the splice: no edge fired, the run stopped, and the caller's queue row stayed `Running` instead of being recorded as parked. Row 364 is the fix, and it is a wiring rule rather than a second mechanism.

5.1 Two inclusions, one shape

	`call` node	**`[[attach]]` fragment**
What is included	another graph, by name	a fragment file under `graphs/fragments/`
Naming	`<call>__<node>`	`<attach>.<node>`, or `<attach>@<target>.<node>` when the attach spans several targets
Entry	a synthesised head `<call>__in`	the fragment's own entry, chained
Pin	`sha256` on the call node	`pin` on the attach
May contain	calls, nested to depth 4	no call, no attach, no timer, no store, no budget
Ordering	declaration order of the calls in the file	`order`, then attach id
Where it runs	wherever its nodes land in the host	a named surface: run start, node before, node after, run end

The two are deliberately not interchangeable. A `call` is the right tool for "this graph is a step in a larger process"; an `attach` is the right tool for "every run of this graph should do this first", or "every node of this kind should be followed by this check". The restriction that a fragment may not call and may not attach is what keeps the weave a single pass: a fragment is work, never a scheduler and never a composition of its own.

5.2 The load, in order

Every caller that runs or checks a graph goes through one function, `load_host_with_graphs`. Its order is fixed, and the order explains which error a bad file produces.

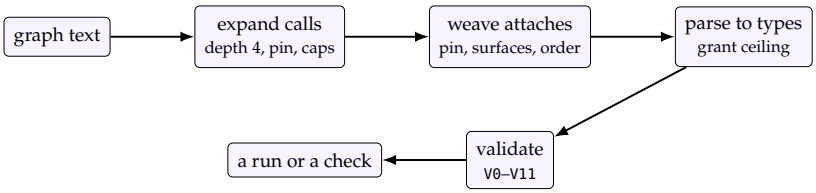


Figure 5.1: The load pipeline; calls are expanded on the *table* before anything is typed, which is why an expanded node's id is a string operation rather than a data structure one, and why `pii graph show` can print a composed graph at all.

1. **Expand calls.** The host text is parsed as a TOML table, and each `kind = "call"` node is replaced in place. This happens first so that a call inside a callee is expanded by the same code, recursively, with a chain of graph names to detect cycles.
2. **Weave attaches.** If the expanded table still has an `[[attach]]` array, the fragments are read, pin-checked, contract-checked and spliced at their surfaces.
3. **Parse to types.** The resulting table is loaded into `GraphSpec`, and the registry's grant ceiling is applied here — the graph and each node are refused if they request more than the host allows.
4. **Validate.** The static rules run over the expanded graph: ports exist, capabilities are within the graph's, cycles are bounded, joins are satisfiable.

The consequence of the order is that a pin error beats a port error. A graph whose callee has changed reports `fragment.pin.mismatch` before anyone learns that the graph also has a typo in an edge — which is correct, because until the pin is honest the shape of the graph is unknown.

```
pii graph lint graphs/queue.toml     # expand, weave, validate; prints
    ↪ warnings too
pii graph show graphs/queue.toml     # the graph after expansion, node by
    ↪ node
pii graph explain graphs/sdd.toml    # the composition, one level at a
    ↪ time
```

5.3 A call, before and after

The clearest way to see the mechanism is to look at the same file twice. Before the load, the queue carries a `call` node and three edges that name it. The node is a placeholder: it has a kind, a graph name, a pin and a caps list, and no ports of its own.

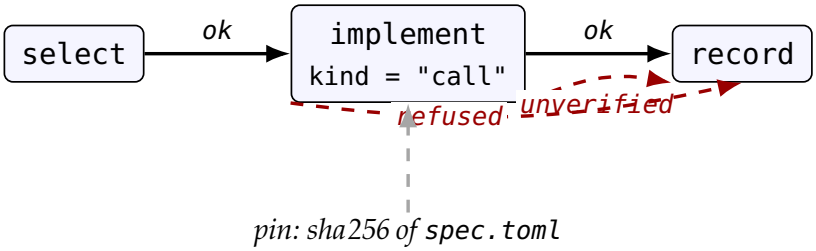


Figure 5.2: Before the load: one node standing in for a graph; the two red edges are the ones row 364 added — a host may wire a callee's refusal *by name*, which is the whole of the refusal mechanism.

After the load the placeholder is gone. Its head is a synthetic node that counts entries, the callee's nodes are present under namespaced ids, and the host edges have been re-pointed: edges that arrived at the call now arrive at the head, and edges that left the call now leave the callee's declared exit.

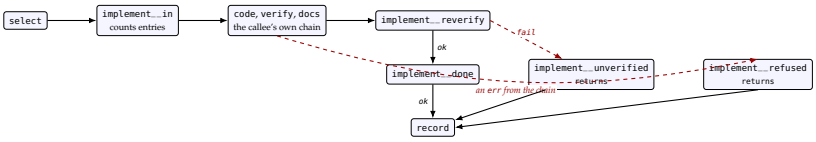


Figure 5.3: After the load: the callee's nodes under the call's name space, the transform that returns on `ok`, and the two refusal nodes marked `returns` so the host's own edges can carry their verdict to `record`.

Five details of the expansion are worth knowing because each of them has produced a misunderstanding at least once:

- **The name space is a string prefix.** A callee node `code` becomes `implement__code`. There is no nesting structure and no scope; the id is flat, and that is why the load refuses an expansion whose ids collide with the host's (`graph.call.id`).
- **The head is not decoration.** `<call>__in` exists so that a call can be counted: it carries `max_entries`, and the run counts completed entries against it. It also gives the call a single place to receive its input.
- **Input is one transform per key.** A transform evaluates one top-level expression, so a call's `[node.in]` table becomes a chain of transforms named `<call>__in__<key>`, each after the head. One node could not hold a whole object of expressions.
- **The callee's reads are rewritten.** Inside the callee, `$.input.x` refers to the call's input, and `$.nodes.m` refers to the callee's own copies of its nodes. A callee cannot read the host's input by accident.
- **The host can read the result by the call's id.** `$.nodes.implement` is rewritten to `$.nodes.implement__done`, so a host expression does not need to know which inner node happens to be the exit.

That last rewrite has a boundary rule that is easy to get wrong and is tested both ways: the prefix must end at a dot or at the end of the string. `$.nodes.sdd` is rewritten; `$.nodes.sddx` is a different node and is left alone. Node ids and kinds are never rewritten — only values — and a host that never mentions the call id comes out byte-identical.

5.4 The exit rule

Now the rule the chapter exists for. When a piece of a graph is spliced into another, the splicer has to answer one question: which of these nodes hands control back? The answer is computed, not declared:

1. For each node in the piece, ask its factory for the ports it declares with this configuration.
2. The node qualifies if that set contains `ok`.
3. The node is *wired* if the piece has any outgoing edge from it on port `ok`.
4. The piece's exits are the nodes that qualify and are not wired.

The rule is applied twice in the loader, at two scales. Per piece, it decides which nodes feed the next piece's entry (or the tail the caller supplied). At the host level, `run.end` uses the same predicate to decide which host nodes the end fragment follows: every host node with no outgoing `ok` edge becomes a sink, and the end chain is hung off all of them with order edges. A graph with three natural endings therefore reaches the end chain from three places, which is the point — a fragment attached at `run.end` runs however the graph finishes.

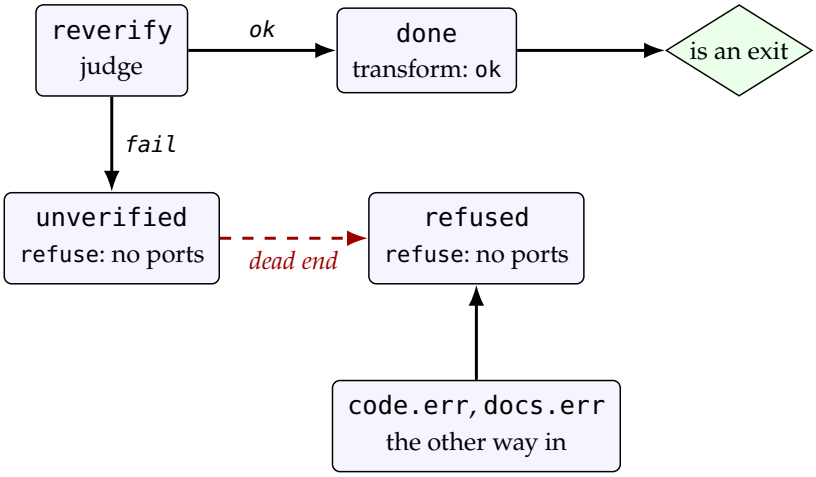


Figure 5.4: The tail of `graphs/spec.toml` under the exit rule: `done` returns because it declares `ok` and has no `ok` successor, while `unverified` and `refused` declare nothing, so a refusal leaves no edge out of the splice.

Two consequences follow, and both are load-bearing.

A piece with no exits is not an error at load. It is a piece that never returns. The loader accepts it because a fragment attached at `run.end` legitimately ends there, and because a callee's refusal is a legitimate way for the whole run to fail. The failure only becomes visible at run time, as a run that stops — which is exactly the class of failure this system tries to make impossible.

The declared `exit` is a different thing. A graph's `exit` names its terminal for a direct run, and the call mechanism uses it to re-point host edges: an edge that left the call now leaves `<call>__<exit>`. The wider rule exists so that a fragment with several natural endings still hands control back. Both facts are true at once, which is why reading only one of them produces a wrong mental model: the declared exit decides where the *host's* edges land, and the computed exit set decides where a *piece* hands control to what follows it.

5.5 Why a refusal dead-ended

The refusal shape is worth following to the bottom, because it is the one place where a node's port set is empty rather than merely unused.

Fact	Where it is decided
A `refuse` node declares no ports	`RefuseFactory::ports` returns an empty vector unless the configuration carries `returns`
Its `run` fails the run by name	`RefuseNode::run` returns a failure whose text is `run.refused{hook=..., reason=...}`
An empty port set cannot contain `ok`	so the exit computation never lists it
No exit means no edge to the tail	so nothing after the callee runs

On 2026-11-04 that combination left a queue row `Running` for a night. The callee refused, the run stopped, and the row's claiming run was never recorded as finished — so the row blocked the queue, and an operator looking at the queue saw work in progress that had in fact already been refused. The symptom and the cause were three files apart: the queue's status in `pii-store/src/queue.rs`, the edge in `graphs/queue.toml`, and the port set in `pii-graph/src/node.rs`.

There were two honest ways out, and the spec said so explicitly rather than letting the implementer choose silently:

1. **Make every verdict return on `ok`.** Give the refusal nodes an `ok` port — or make them transforms carrying a verdict value — so that all three tail nodes are exits. The host must then distinguish them by the value that arrived, because all three come back on the same port.
2. **Collapse the tail into one verdict node** with ports `ok`, `unverified` and `refused`, and let the host wire each to the outcome it wants. This makes the callee's outcome vocabulary explicit but changes the callee's tail and every host that calls it.

5.6 The fix: a wired refusal returns

The chosen fix is a third shape, and it is the smallest one that keeps both properties: the callee's tail does not change, and the host gets a distinct port per refusal. It works by letting the *host* opt in, edge by edge.

The load walks the host's edges. For each edge whose `from` is the call's id, it takes the port name and looks for an added node called `<call>__<port>` whose kind is `refuse`. If it finds one, it does two things: it sets `returns = "<port>"` on that node, and it re-points the edge to leave that node on `ok` instead of leaving the call. The node's factory now reports one port, the exit computation includes it, and its `run` returns a verdict instead of failing the run.

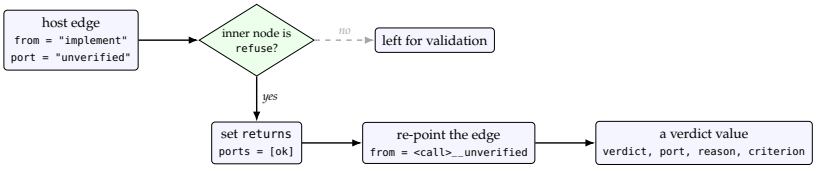


Figure 5.5: The opt-in, edge by edge; a refusal no host wires is not marked, still declares no ports, and still fails the run — which is the control that proves the mechanism is wired and not merely present.

The verdict that comes back is a small object, and every field of it is used by a host:

- `verdict` is the literal `"refused"`;
- `port` is the name the host wired (`unverified`, `refused`), so a host with one `record` node can tell them apart;
- `reason` is `"<port>: <declared reason>"`, extended with the arm's own reason when the arriving value carried one;
- `criterion` is the first failing criterion from the arriving value, when there is one — which is what lets a parked row name the criterion that failed.

The controls that came with the fix are as important as the fix, because a mechanism that fires on everything is indistinguishable from one that fires correctly. Three near misses are asserted: a refusal the host does not wire still fails the run; a port that names no refusal node is left on the call and points at the exit, where validation names it; and the same callee run directly still fails, because nothing in the direct path sets `returns`. The callee's own behaviour is unchanged — which is the requirement that makes the fix acceptable at all.

The host side is four lines of TOML, and their brevity is the point: the mechanism is visible where a reviewer reads the graph.

```toml
[[edge]]
from = "implement"
port = "unverified"
to   = "record"

[[edge]]
from = "implement"
port = "refused"
to   = "record"
```

5.7 Pins

A pin is the sha256 of the included file's bytes, written into the file that includes it. A call carries `sha256`; an attach carries `pin`. The two produce different messages because they are different code paths — `fragment.pin.mismatch` from the call expansion, `fragment.pin.drift` from the fragment loader — but they mean the same thing: the file changed since someone reviewed this inclusion.

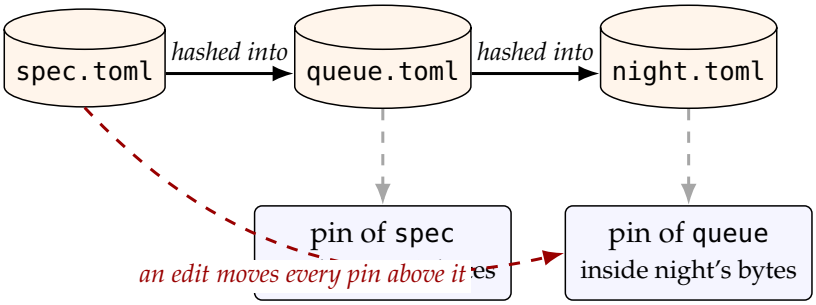


Figure 5.6: Pins are transitive by construction rather than by policy: a callee's text contains its own callee's hash, so editing a leaf changes the middle file's bytes, which changes the top file's pin.

That transitivity is not a feature anyone wrote; it falls out of hashing text that contains hashes. It has one practical consequence worth stating:

re-pinning after a leaf change is a mechanical edit of every file above it, in the same commit, and the diff shows one hash per level with one sentence of justification. The rule the repository follows is that a pin moves only with a reason in the commit, because the pin is the only thing that makes "the night cannot run a graph nobody reviewed" true.

Row 373 is the cautionary case, and it is not a pin defect at all. `graphs/night.toml` had a node whose id was `sdd`, whose kind was `call`, and whose `graph` was `queue`. The pin was honest: it pinned the queue. The defect was that the name described something the composition did not do. The fix was to create the graph the name promised and re-pin the node to it. A pin protects against a file changing under you; it cannot protect against a file that was never the right one.

5.8 Fragments

A fragment is a graph file with a `[fragment]` table instead of a graph's header. It is read once, at load, and spliced into the host at one of four surfaces.

Surface	Where the chain goes	What the splice does
`run.start`	in front of the host's entry	the chain's exits feed the old entry; the host's entry becomes the chain's head
`node.before`	between a node's inbound edges and the node	inbound edges are re-pointed at the chain; the chain's exits feed the node
`node.after`	on a node's `ok` edge	the node's `ok` edges are lifted and re-hung after the chain; the node now points at the chain's head
`run.end`	after every host node with no `ok` successor	each sink gets an order edge to the chain's head; the chain's exits feed nothing

The header is checked before any of that happens. It must declare `name`, `inputs`, `outputs` and a boolean `may_block`; a fragment that omits one is refused with `fragment.header.missing`. The body must not contain a top-level key that would give it autonomy — `timer`, `store`, `budget`, `limits`, `schedule`, `cron` — nor a node configuration key that would do the same (`budget`, `store`, `limits`, `cron`, `timer_ms`, `in_ms`). The message is `fragment.autonomy.forbidden`, and its text states the principle: a fragment is work, never a scheduler.

The block rule is the one that reads as arbitrary until you see the failure it prevents. A fragment attached at `run.start` or `node.before` runs *before* the thing it guards, so a fragment that can block there can hold the host at a point where nothing else has run. Only a fragment whose header says `may_block = true` may attach at those two surfaces; anything else is refused with `fragment.block.invalid`. A fragment at `node.after` or `run.end` runs after work has happened and has no such rule.

```
[fragment]
name      = "record-run"
inputs    = ["run"]
outputs   = ["ok"]
may_block = false
```

Naming follows the same flat logic as a call, with one addition. A fragment attached to a single target is namespaced `<attach>.<node>`. An attach with `on = ["a", "b"]` produces one copy per target, and each copy is namespaced `<attach>@<target>.<node>`, because two copies with the

same ids would collide. The attach's order is `order`, then id, and that ordering is what decides which of two fragments at the same surface runs first. Each inlined node also carries two stamps in its configuration: `woven-from`, of the form `<fragment>#<node>`, and `hook`, the attach id. Those stamps are how a trace of a woven graph can name the file a node came from, long after the file has changed.

5.9 Bounds: depth, cycles, entries

Three limits keep composition from becoming recursion.

- **Depth 4.** A chain of nested calls deeper than four is refused with `graph.call.depth`, naming the call and the chain. The limit is a constant in the expander, not a configuration key, because the right answer for a graph language this small is "shallow".
- **No cycles.** A call that would re-enter a graph already in the chain is refused with `graph.call.cycle`, and the message prints the chain in order. This is a load-time check, not a run-time one: two graphs that call each other cannot be inlined at all.
- **Eight entries.** A call's head counts completed entries, default eight, and the run fails once a call is entered more than `max_entries` times with `call 'X' entered more than max_entries (N) times`. The count is cumulative for the run and is reset by no enclosing loop, so an inner call inside a looped outer call reaches its own limit after that many entries across all the outer passes. An author who needs more raises the inner `max_entries` explicitly, in the file, where the number is reviewable.

The entry counter exists because inlining is not looping. A call is textually one copy of the callee; if the host's shape can reach it repeatedly, the callee runs repeatedly with the same ids, and without a counter a graph that loops would accumulate state indefinitely. The counter turns that into a named failure at a stated number.

5.10 Every refusal composition can produce

The table is the chapter in one page: each row is a rule code, the condition that raises it, and the thing a reader has to look at to fix it. All of them are load-time refusals — composition has no run-time errors of its own.

Code	Raised when	Look at
`graph.call.key`	a call node carries a key outside `id`, `kind`, `graph`, `sha256`, `caps`, `in`, `max_entries`	the call's table
`graph.call.missing`	the call has no `graph`, `sha256` or `caps`	the call node
`graph.call.name`	the graph name is empty, starts with a dot, or contains a path separator	the `graph` value
`graph.call.not_found`	no graph directory holds `<name>.toml`	the graph directories given to the load
`graph.call.read`	the callee exists but cannot be read, or is not UTF-8	the file

Code	Raised when	Look at
`fragment.pin.mismatch`	the callee's sha256 differs from the call's	both hashes in the message
`graph.call.exit`	the callee declares no `exit`, or it names no node	the callee's tail
`graph.call.widen`	a call grants caps the host does not hold, or a callee node needs caps the call does not grant	the caps on both sides
`graph.call.cycle`	the callee is already in the chain	the chain printed in the message
`graph.call.depth`	the chain would nest past four	which call to flatten
`graph.call.id`	an expanded id collides with a host node	the id named in the message
`graph.call.in`	`in` is not a table	the call's input block
`graph.call.max_entries`	`max_entries` is not an integer of at least one	the value
`graph.call.needs`	the callee's `needs` is malformed	the callee's header
`graph.call.undeclared`	the callee reads an input its holder does not declare	the `needs` list and the `in` map
`graph.call.leak`	a callee reads a host node directly instead of through `in`	the expression named in the message
`graph.call.no_dir`	a call exists but the load was given no graph directory	the caller's setup
`fragment.pin.missing`	an attach carries no `pin`	the attach table
`fragment.pin.drift`	the fragment's bytes differ from the attach's pin	both hashes
`fragment.surface.unknown`	the attach names a surface outside the four	the `at` value
`fragment.target.unknown`	`on` names a host node that does not exist	the `on` list
`fragment.surface.unreachable`	a non-blocking attach has no sink to follow	the host's last nodes
`fragment.header.missing`	the fragment lacks `name`, `inputs`, `outputs` or a boolean `may_block`	the `[fragment]` table
`fragment.block.invalid`	a non-blocking fragment attaches at a pre-surface	its `may_block`
`fragment.autonomy.forbidden`	the fragment would own a timer, store, schedule or budget	the key named in the message
`fragment.nesting.forbidden`	the fragment contains an `attach` or a `call`	the fragment body
`fragment.path.invalid`	the fragment name is empty or contains a path component	the `fragment` value

Two of these are worth recognising on sight because they read as something else. `graph.call.undeclared` is not about capabilities: it means the callee reads `$.input.x` for a key nobody mapped, which is the shape of a callee whose contract grew without its callers being told. And `graph.call.leak` is the one that protects composition from becoming

implicit coupling — a callee reading a host node by name would work until the host renamed it, and would make the pin meaningless as a description of what can change.

5.11 What composition cannot do

Two limits are not bugs and are worth knowing before designing a graph.

There is no iteration primitive. Composition is static, so a graph cannot say "do this call once per criterion". Everything that needs repetition is one node looping internally — a judge running four commands, a coder working through a list — which is precisely why the judge has to know how to run criteria at all. A `foreach` or `map` over a list would be a new mechanism, and the repository's rule is that a second inclusion mechanism is never built.

Models are per-node strings. Composition is fine with that, but each piece can carry a different — possibly dead — model id, and a graph that composes three pieces inherits three chances to name a model that is not served. The model-tier work exists to make that a resolved mapping rather than a literal per node, and until a graph uses it, the literal is what runs.

5.12 A checklist for a new graph

The list is short and every item on it corresponds to a refusal in this chapter.

1. Does a graph already do this? If yes, call it; do not copy it. A copy is a second implementation and the two will drift.
2. Does your graph need to be composed? If so, declare `exit`. A graph without one is a run, not a part.
3. Grant caps for what your nodes need, and no more. The ladder will catch a mismatch, loudly, with the node and the capability named.
4. Pin every call by sha256 and every attach by pin, and say in the commit why a pin moved.
5. One responsibility per graph. A loop is a graph; a process is a graph; a schedule is a graph.
6. If a step needs no model and no process, it is a `transform`, not a coder.
7. Every failure path gets a port whose name says what happened. If the callee refuses, wire the refusal — otherwise the refusal is invisible to the host.

The last item is the one this chapter was written for. A refusal that is not wired is not an error the host can see; it is a run that stops.

6 Stores

This chapter answers one question: where does pii keep what it knows, and what stops two parts of the program from disagreeing about it? The answer is a single crate over a single embedded database, a small number of tables with exactly one writer each, a counter record that moves in the same transaction as the events it counts, and a retention sweep whose guards matter more than its numbers. The chapter also states what a store is *not* allowed to become: a place to keep an index nothing reads, or a place to delete the one row an operator is looking for.

6.1 One crate knows the database format

`crates/pii-store/src/lib.rs` opens with the rule it exists to enforce: this is the only crate that knows redb. A write transaction is opened and committed inside one method body. It is never stored in a struct, never returned to a caller, and never held across a node dispatch. That single sentence is what makes the journal's one-transaction-per-append property true, and it is why a node that runs for three minutes cannot be holding a write lock for three minutes.

Everything above the store sees a handle: `Store` for runs, `QueueStore` for the work queue, `ChannelStore` and `TimerWheel` for the two durable stores, `MemStore` for memory. The handle carries a database, a path, and test instrumentation that is explicitly not public API. The instrumentation is worth naming because it is how several claims in this manual are measured rather than asserted: `write_txn_count` counts committed write transactions, `session_bytes_written` counts bytes the session path wrote, and four fault-injection flags make a write or a read fail on demand so the failure-handling path can be exercised without an operating-system trick that does not work for root.

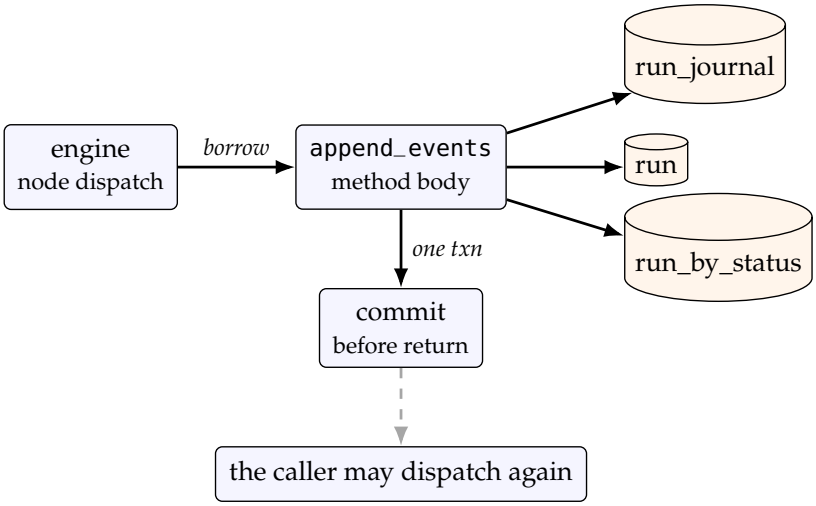


Figure 6.1: One transaction moves three tables and commits before the caller sees a return value, so no node can be dispatched while a write is open.

The cost of that shape is that a caller cannot batch across calls; the benefit is that no code path exists in which a lock is held across an await. The batching that mattered was inside one append, and it is there: `append_events` takes a slice, writes every event at consecutive sequence numbers, rewrites the header once, and repoints the status index once, in one transaction. The single-event `append_event` is a thin wrapper over it, kept so the observable behaviour of the pre-batching path is byte-identical.

6.2 The table inventory

The definitions live in `crates/pii-store/src/tables.rs`, one constant per table, with the key and value types written down. Values are `serde_json` bytes rather than a compact encoding: postcard would be smaller, and it would add an encoding failure mode that a generated change does not need.

Table	Key	Value	Owner
`meta`	`&str`	`u64`	run counter, format version
`graph_body`	blake3 hex	bytes	graph bodies, content-addressed
`run`	run id	header	one row per run
`run_journal`	`(run, seq)`	event	append-only event log
`run_snapshot`	`(run, seq)`	bytes	blackboard snapshots
`run_by_status`	`(status, run)`	`()`	secondary index for listing
`sys_status`	`"totals"`	record	health totals (spec 370)
`sys_status_run`	run id	per-run row	contribution the totals sum
`session`	session id	meta blob	cwd, tools, graph, status, run
`session_tokens`	session id	`(u64,u64)`	cumulative input/output
`session_msg`	`(session, seq)`	message	one row per message
`session_snapshot`	`(session, seq)`	full session	replay tail only
`mem_doc`	doc id	doc	memory documents
`mem_term`	`(term, doc)`	`u32`	postings, redb-resident
`mem_df`	term	`u64`	document frequency
`mem_vec`	doc id	packed	quantised vectors
`mem_edge`	`(concept, doc)`	`u64`	co-occurrence weight
`mem_meta`	`&str`	`u64`	`n_docs`, `sum_dl`, dims
`mem_archive`	doc id	doc	archived, never deleted

Table	Key	Value	Owner
mem_layer	doc id	u8	layer tag, absent means L1
mem_evidence	doc id	bytes	skill-edit evidence (spec 210)
timer_wheel	(fire_at, run)	resume key	durable timers
approval	approval id	record	pending approvals
heal_attempt	key	record	heal-attempt ledger
channel_cursor	channel	cursor+run	channel position

Two tables are outside this file and deserve to be named here because they are opened by the same daemon: queue_item and queue_event live in pii-store/src/queue.rs on their own redb file with a 4 MiB cache, and the durable pair channel.redb and timer.redb live in a directory the daemon resolves once (chapter 7). A run store, a queue store and two durable stores are four files with four handles and no shared transaction — which is exactly why the durable pair is claimed before anything is bound.

6.3 One write path per fact

Each table has exactly one writer. When two subsystems write one table the counters drift and no single place can explain a number, so the question to ask of any new field is not "where would it be convenient" but "which method owns it".

- run, run_journal, run_snapshot and run_by_status are written only by the append path and by retention. No node kind opens them.
- graph_body is written only by put_graph, which hashes the body and returns early when the hash is already stored. A body is addressed by its own hash and never changes, so committing one that is already there only rewrites pages. Every run used to store its graph and its input afresh; after the change, identical bodies cost nothing.
- meta holds the run counter, moved only by reserve_run_ids, and the format version, moved only at open.
- sys_status and sys_status_run are written only by the append observer, inside the append transaction (section below).

The run counter is the clearest example of a counter whose cost was measured before it was redesigned. Ids are reserved in blocks of 32: one durable transaction moves the persisted counter to the end of a block, and the ids inside the block are handed out from memory. A crash or restart skips the unissued rest of the block, so ids may have gaps but are never reused. The alternative — one commit per id — was about 20% of the bytes a run wrote. The test that pins the new behaviour counts transactions rather than trusting the intention: 100 ids cost 4 commits, where one-per-id would cost 100.

```rust
// crates/pii-store/src/lib.rs
pub fn next_run_id(&self) -> Result<RunId, PiiError> {
    let n = {
        let mut ids = self.run_ids.lock();
        if ids.0 >= ids.1 {
            *ids = self.reserve_run_ids()?;
        }
        ids.0 += 1;
        ids.0
    };
    Ok(RunId::from(format!("r{n:012}")))
}
```

6.4 Counters that stay current

The health record is the part of the store most likely to rot, because it is derived: it says how many agent calls happened, and every one of those numbers could be recomputed from the journal. A derived number kept in a table has two possible failure modes — it can lag the journal, or it can disagree with it — and both destroy an operator's trust in the whole record.

The design answers both with one mechanism. `set_append_observer` installs a function that every append runs *inside its transaction*, after the header and events have been written to the transaction but before the commit. The observer receives the store, the open write transaction, the advanced header and the events just appended. An error from the observer aborts the append. So the record moves with the runs it counts, or neither moves.

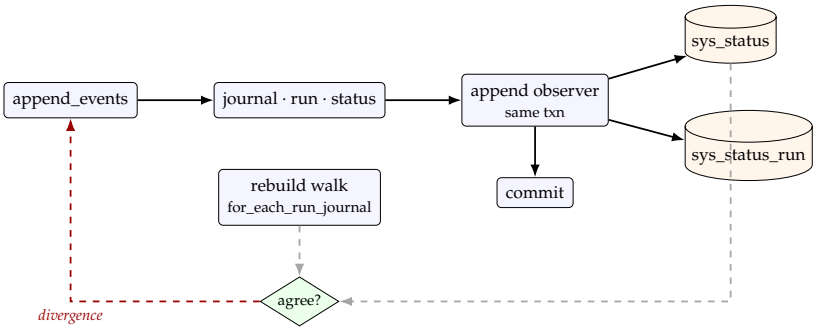


Figure 6.2: The record and the journals can be compared only because the rebuild walk exists, since health itself never replays on a request path.

The rebuild is deliberately not on any request path. `for_each_run_journal` walks every run and its whole journal inside one write transaction, oldest key first, handing each pair to a closure. It exists so a test can rebuild the record from first principles and compare. `sys_clear` deletes both record tables in one transaction, and the next write re-creates them — the cheapest possible way to prove that nothing else was quietly reading them. `runs_up_to` counts runs with a limit, so a health probe can answer "at least N" without walking a large store.

Three properties follow, and each is worth stating because each is easy to lose in a refactor:

1. A record row is never written outside a transaction that also wrote the event it counts.
2. A record row can be deleted and rebuilt; nothing may depend on it as the only copy.
3. A read of the record is a single key lookup, so health cost does not grow with history.

6.5 Retention

`crates/pii-store/src/retention.rs` is short and its comments carry the measurements. Three numbers, all defaults in `RetentionPolicy`:

Field	Default	Unit	What it means
`run_age_ms`	30 days	ms	a finished run's header, journal and snapshots go
`node_event_age_ms`	7 days	ms	a finished run's node-call events go
`max_rows`	1 000	rows	most rows one sweep removes

A sweep is one write transaction. It collects the finished runs, filters out rows it cannot date, then removes node-call events for runs past the shorter age, then whole runs past the longer age, all against one shared budget. The cap is not decoration: without it, a first run on a store that has never been trimmed would hold a write transaction for minutes, and every append behind it would wait.

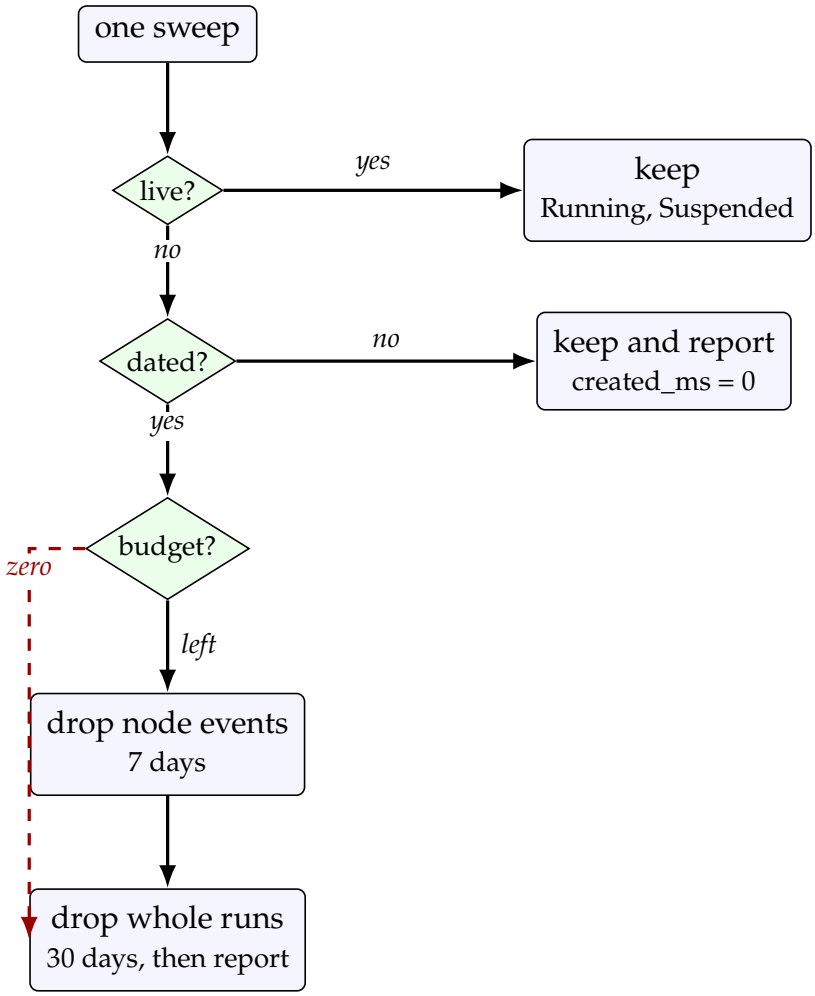


Figure 6.3: Two guards run before the numbers do: a live run is never swept whatever its age, and a row with no clock is kept because it is exactly the row an operator is looking for.

Both guards were found the hard way. The live-run guard is the difference between trimming and losing work in flight: the daemon may still resume a `Running` or `Suspended` run, so deleting it would delete the thing it is about to continue. The undateable-row guard came from a fixture. Rows seeded at epoch 1000 were deleted by the sweep, and the same rule would have deleted every pre-clock row in a real store — a store written before the clock was recorded has `created_ms == 0`, and a filter written as an age comparison treats zero as infinitely old. The code now filters `created_ms > 0` first and says why in a comment.

Order inside a run. The order is not cosmetic. Whole runs are expensive and the budget is shared, so the cheap, targeted deletion happens first. Within a run the header goes last, once nothing else of the run is left, so a capped sweep never leaves a header pointing at rows that no longer exist. Snapshots follow the journal, and the status-index entry goes with the header and is not counted as a row.

Step	Removes	Why in this position
1	`NodeStarted`, `NodeFinished`, `NodeFailed`, `NodeProgress` rows	these are the bulk of a journal; the run stays readable
2	`run_journal` rows of an expired run	a run is only removable once its journal is
3	`run_snapshot` rows	follows the journal, same budget
4	`run` header	last, so a capped sweep cannot orphan rows
5	`run_by_status` entry	goes with the header, not counted

Reporting. A sweep that removed at least one row emits one `store.retention.swept` event carrying the count, through the store's observer. A sweep that removed nothing emits nothing, so an idle host does not fill its log with zeroes. The event is what makes a shrinking store observable rather than quietly smaller.

Boot. `sweep_at_boot` runs one sweep when the daemon builds, and a failure is reported to the caller's sink as `store.retention.failed` rather than refused. The reasoning is stated in the function's own comment: refusing to boot over retention would trade a big store for none. Retention is maintenance, not a precondition.

The operator's entry point. `pii store prune [-dir D]` runs one sweep under the default policy and prints the count. Both `compact` and `prune` require the daemon to be stopped, because both take the store's write lock.

```
# one sweep, default policy, daemon stopped
pii store prune
# pruned 1000 rows
```

The number in that line is usually the cap. A store that has never been trimmed will report 1 000 on the first run and again on the second; that is the bound working, not a failure.

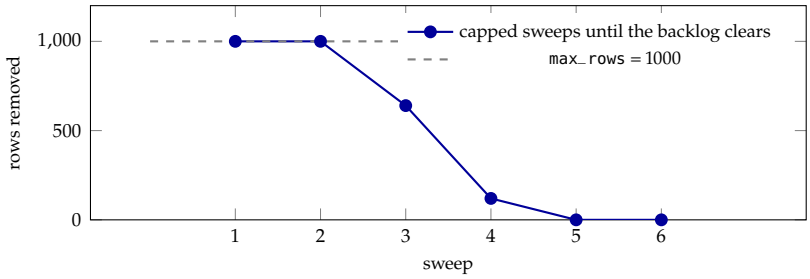


Figure 6.4: The shape an operator should expect, illustrated, is several capped sweeps and then quiet, while a store that reports 1 000 on every sweep is receiving rows faster than the policy expires them.

6.6 The queue store is a second store, with one writer

The work queue is not a table in the run store. It is its own redb file with two tables — `queue_item` keyed by item id and `queue_event` keyed by `(id, seq)` — and a 4 MiB cache, in `crates/pii-store/src/queue.rs`. Keeping it separate means a queue claim never competes for the write lock with a run's journal, and a store rebuilt for one purpose does not have to be rebuilt for the other.

The single-writer rule shows up here in the protocol rather than in the process. The daemon is the only writer; the CLI does not open the file. `pii queue add` sends a request over the control socket, the engine's dispatch handles it, and the insertion and its `queue.item.added` event are one transaction. A CLI that wrote the file directly would be a second writer, and the daemon's in-memory view of the queue would be wrong until restart.

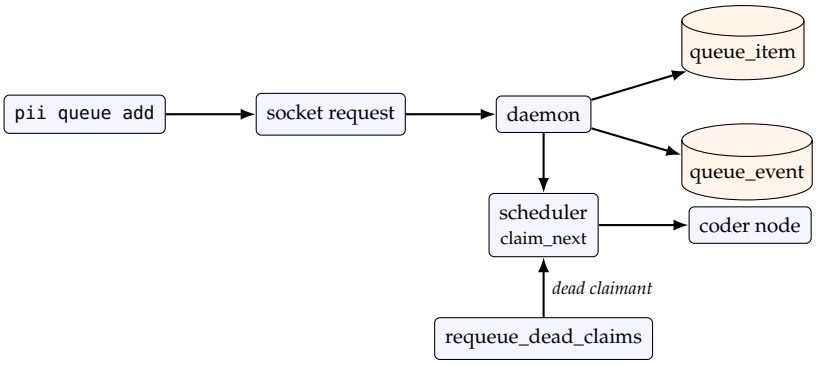


Figure 6.5: One writer and one claim path, where the CLI reaches the store only through the daemon and a claim is a status change plus its event in one transaction.

`claim_next` is the interesting method because it must be atomic against a second caller. It opens one write transaction, reads every item, sorts by queue position, decides, and — only if the decision is a claim — writes the item back as `Running` with its attempt count incremented, its claimant recorded, and an `item.claimed` event appended, all before the commit. A `Running` item short-circuits: the caller is told the queue is already running something rather than being handed a second row.

`set_status` follows the same shape: read the item, change its status, write it back, journal `item.<status>` in the same transaction. That is what makes the queue's event log sufficient to explain any row's state without reconstructing it from logs elsewhere.

`requeue_dead_claims` is the recovery path. It asks a caller-supplied predicate whether the recorded claimant of a `Running` row is dead (an empty string for a row with no recorded claim), returns those rows to `Pending`, and journals each as `requeued_after_restart`. The returned count is what the daemon reports at boot. Two properties matter: the predicate is supplied by the caller, so the store does not need to know how to look up a process, and a row whose claimant is alive is left alone, so a second daemon cannot steal work from a running one.

```
pii queue add --id 376 --spec specs/376-retention-drops-what-the-policy-
    ↪ expires.md \
              --title "retention drops what the policy expires" \
              --criteria "cargo test -q -p pii-store --test
    ↪ retention_drops_what_the_policy_expires" \
              --touches crates/pii-store/src/retention.rs
pii queue ls
pii queue claim --by operator
```

6.7 Bounded artefacts

A store is one of several things a long-lived host fills. `pii disk` prints the others in one table: the warm gate cache, the fresh-gate target, the coverage and tool target dirs, the trash that parks work, and the gate exports outside the repository, each with bytes, age, and *the filesystem it sits on*. The filesystem column exists because a floor was once measured on the wrong volume: two cleans freed 150 GB where nothing was short.

- `_tmp/gate-target` — the warm build cache every gate step shares;
- `_tmp/gate-target-fresh` — the `-fresh` export's separate cache;
- `_tmp/targets` — per-step target dirs (clippy, doctest, tools);
- `_tmp/_trash` — in-repo quarantined work;
- `/projects/_tmp` — `fresh-gate-<pid>` exports;
- `/projects/_trash` — quarantined working trees, one directory per park;
- `/tmp/_trash-podman` — container scratch that outlives a run;
- `/tmp/gate-cache` — tool caches outside the repository.

Each row carries bytes, age and the owning filesystem, because the question an operator asks is not "how big is this" but "which volume is short".

Each has an owner and a ceiling. The warm cache is capped at 40 GiB by `FRESH_GATE_CACHE_MAX_GB`, and is trimmed before a build only when no cargo or rustc is live — a running build would lose its target under it. The trajectory directory is bounded by bytes and file count (512 MiB and 50 000 by default) and prunes the oldest finished runs. The store's own page cache is 8 MiB, which is the subject of the next section.

6.8 The two numbers that were wrong

The page cache. redb defaults to a 1 GiB cache. For a control-plane daemon holding a store of a few megabytes that is not a tuning detail; under soak it appeared as a slow, sublinear RSS climb with a flat thread count, a flat descriptor count, and a store file of a few megabytes. The climb was in no code that owned a connection or a thread. Setting the cache to 8 MiB left ample headroom under the 50 MB budget the RSS gate enforces.

The index nothing read. `RESUME_INDEX` was declared in `tables.rs`, opened three times in `lib.rs`, written by the scheduler's dispatch path, and counted into the identity stats. The code's own comment at the write site said that nothing read it. The cost was real — an index write on every resume — and the number invited an operator to trust a structure with no reader. Spec 375 removed the table, every `open_table`, the write site,

and the stat, while explicitly keeping `RUN_JOURNAL` and `RUN_BY_STATUS`, which are read. The guardrail in that spec is the useful part for a reviewer: do not drop a table because it looks unused; drop it because you found the reader that does not exist.

6.9 Format version and identity

A store records a format version in `meta`. Opening a store whose recorded version is *newer* than the binary understands is refused with both numbers in the message, so an old binary never writes into a layout it does not know. A fresh or pre-versioning store is stamped on open. The auxiliary stores — approvals, heal, memory, channel, timer, queue — are stamped the same way through one shared function, so there is one rule and not six.

`Store::info` reports the path, the file size, the format version, the last write time, and per-table entry counts. It counts six tables, not all of them: `graph_body`, `run`, `run_journal`, `run_snapshot`, `session`, `session_msg`. That is a deliberate short list — the counts an operator needs to answer "is this store growing, and in which of the two big things" — and `pii store stats` surfaces it. The health record carries the same numbers under `entries.<table>`, so a probe can compare two hosts without reading either store.

```
pii store stats
pii store compact      # daemon stopped
pii store prune        # daemon stopped
```

6.10 What a single run writes

A worked example makes the accounting concrete. One run of a four-node graph, with two node calls, writes the following (an illustrative count, not a captured run):

- one row in `run` (rewritten on every append, so the header is the hot row);
- one `run_journal` row per event: started, node started, node finished, node started, node finished, finished — six rows for four nodes, because the run itself emits two;
- one `run_by_status` entry, rewritten as the status moves, and removed from the old status bucket each time;
- one `sys_status_run` row and one rewrite of the `sys_status` totals;
- zero `graph_body` rows on the second and later runs of the same graph, because the body is already stored under its hash.

The header being hot is why the append path rewrites it once per call rather than once per event: batching events into one append is what keeps a chatty node from paying a header write per event.

6.11 Failure modes worth recognising

1. **A second writer.** Two processes on one store interleave writes or refuse; the symptom is a lock error naming the file, usually from

a test that inherited the operator's `PII_STATE_DIR`. The fix is that every test daemon owns its own state directory.

2. **A counter that lags.** A health number that changes only after a restart means the observer is installed after the first writes, or is not installed at all. The rebuild walk is how to tell lag from corruption.
3. **A sweep that deletes what it cannot date.** Symptom: a store that loses its oldest and most interesting runs first, and a fixture that fails only with seeded pre-clock rows.
4. **A sweep that blocks.** Symptom: every append stalls for the duration of one sweep. The cap is the fix, and a sweep that reports exactly 1 000 is the evidence that it is working.
5. **A table with no reader.** Symptom: a statistic nobody can act on, and a write cost on a hot path. The test is to delete the table and see whether any test fails for the right reason.
6. **A cache that grows toward its default.** Symptom: RSS climbing with flat threads and flat descriptors, no leak in any owner. Check the embedded database's cache before rewriting the connection pool.

Each of these was observed here, and each is now pinned by a test that fails if the fix is reverted: the transaction count for id blocks, the transaction count for a duplicate graph body, the undateable-row fixture, the live-run fixture, and the identity report.

7 The daemon

This chapter answers what the daemon is, what it refuses to start without, and where the real security boundary sits. The short answer: one process per state directory, one writer per store, a socket for everything else, and a filesystem confinement applied before the first byte is served. The interesting part is not the wiring diagram; it is the order of the checks, because each check exists to make a specific failure impossible rather than unlikely.

7.1 The production shape

Four binaries ship: `pii`, `pii-mcp`, `pii-web` and `pii-evolve`. The daemon is not a fifth. `pii` embeds it in-process, so `pii daemon` runs the control plane in the foreground and `pii -p`, `pii tui` and the command surface either talk to a running daemon over the control socket or start an engine of their own. There is one engine implementation and no client-side copy of the rules.

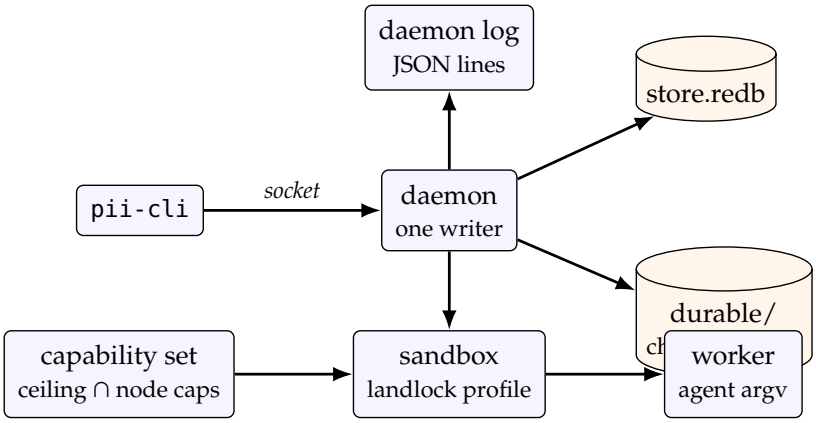


Figure 7.1: One binary, one writer, a socket, and two independent bounds between the engine and any agent: the process-wide sandbox and the per-node capability set.

The two bounds are worth separating at the start because they answer different questions. The capability set answers "may this node do this kind of thing" — read files, write files, reach the network, spawn a process. The sandbox answers "may this process, at all" — and it is enforced by the kernel, after the engine has decided. A node without `fs_write` cannot write whatever the agent believes about itself, and an agent that talks its way past its own instructions still cannot write outside the granted paths.

7.2 Boot order

The order is the chapter's real subject. Each step below can refuse, and each refusal names the thing that failed before anything has been bound, spawned or written.

1. **Flags and config.** `pii daemon` takes `-store`, `-config`, `-workspace`, `-sandbox`, `-log-file`. An unknown key in the config file is a load error, not a silent default: a typo'd grant must not become no grant.
2. **The sandbox.** Under `-sandbox workspace`, confinement is applied and verified before the store is opened. Failure exits `4`.
3. **The measurement switches.** `-observer` and `-audit` accept `on` or `off`; a typo is an error, never "on".
4. **The log and the starting mark.** `daemon.starting` is written before anything that can fail, carrying the pid and the store parent, so a start that never serves is visible in the record rather than only in the supervisor's journal.
5. **The durable claim.** `pii_store::durable::claim` probes `channel.redb` and `timer.redb`; a store another process holds exits `3` with the store's path in the message.
6. **The model.** No model configured exits `5`. The daemon will not start into a state where the first agent dispatch is the thing that discovers the omission.
7. **The store and the engine.** One retention sweep runs here; a failure is reported as `store.retention.failed` and is not a refusal.
8. **Channels, MQTT, cron, workers.** Each entry is validated; a cron entry whose graph does not start at a `timer` node is refused, because it would re-run as fast as it finishes.
9. **The socket.** Bound last, after everything the daemon needs is known good.

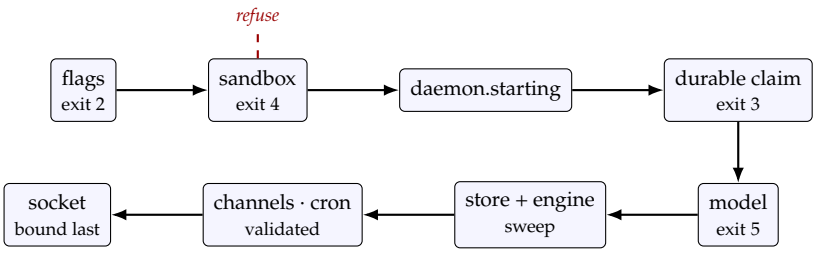


Figure 7.2: Every refusal happens before the socket exists, so a start that fails fails where a person is watching a terminal rather than on the first request from a client.

Exit	Condition	Message shape
2	bad flag or config	`config: <toml error>` or the usage line
3	a durable store is held	`store <path> is locked: another daemon holds it`
4	sandbox not enforceable	`sandbox: enforcement could not be verified`
5	no model configured	`no model configured (set PII_MODEL, or [core].model ...)`

```
pii daemon --store /var/lib/pii/store.redb \
           --workspace /projects/pi-improved \
           --sandbox workspace \
           --config ~/.pii/pii.toml \
           --log-file /var/log/pii/daemon.jsonl
```

The default store path is documented in the flags' own help as `<runtime dir>/store.redb`, and the default workspace is the working directory. The defaults are the thing to check when a daemon "sees the wrong data": the store path decides which history a client is reading, and the workspace decides which files a sandboxed agent may write.

7.3 One writer, enforced three times

The store admits one process. That sentence is true at three levels, and each level catches a different mistake.

The database file. redb takes an exclusive lock on the file. A second process that opens it either blocks or fails; either way two processes never interleave writes. The single-writer rule in `pii-store` is about transactions inside one process; the file lock is about processes.

The durable pair, claimed first. The channel cursor and the timer wheel are separate small databases under the state directory. Before binding anything, the daemon probes both with a non-blocking lock, one attempt per store, no retry, no create. A store that is free is left exactly as it was found; a store that is held is named. The probe deliberately keeps no lock — the owner opens the store immediately after — so the claim is a check, not a reservation.

The state directory. `PII_STATE_DIR` is resolved before `HOME`, so a second instance — a test, a soak, an operator's second daemon — opens its own store, its own durable pair and its own audit log. Without that resolution order, a test started beside a running daemon opens the operator's `channel.redb`, finds it held, and hangs before binding its socket. The symptom is a test that never starts and a daemon log with nothing in it.

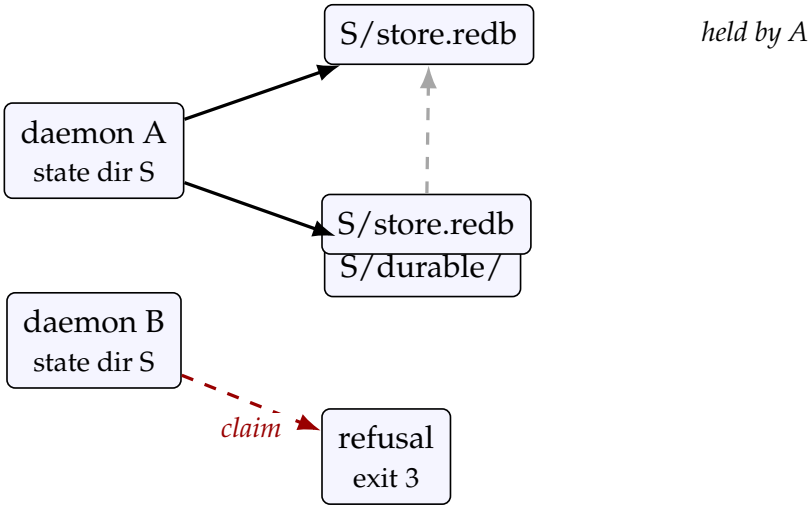


Figure 7.3: The second daemon is refused by name rather than queued behind a lock, because waiting produced a hang with no message that reads as a broken binary.

The rule has a cost the operator must know: two daemons cannot share a state directory, so two concurrent soaks or two test suites need two directories. The failure when they do not is loud now, which is the point; before it was loud, it was a flaky test attributed to the code under test.

7.4 Two directories that mean different things

The runtime directory holds things that are allowed to vanish on reboot: `control.sock`, the graphs a daemon loads, `crashes/`. On this host it is a per-user directory under `/run/user`. The state directory holds things that must survive: the run store, the durable pair, the audit log.

Keeping them apart is a decision with a visible consequence. A store under the runtime directory would be emptied by a reboot, and a reboot would therefore silently delete every run history and every pending timer. The durable directory is resolved as `[core].durable_dir` when set, else `<state_dir>/durable`, and never defaults to the runtime directory. The comment in `daemon/mod.rs` says why in one clause: the runtime dir is a tmpfs.

7.5 The sandbox is the real bound

`crates/pii-tools/src/sandbox.rs` implements two profiles and one rule about them.

Profile	What is in force
`workspace`	WRITES confined to the workspace, the store directory, the runtime directory, `$TMPDIR`, the container runtime state dirs and `/dev/shm`, plus the device files `/dev/null`, `/dev/zero`, `/dev/urandom`, `/dev/tty`. READS are not confined: read-only access to `/` is granted, so `/etc`, `HOME` and every other readable path stay readable.
`off`	no filesystem confinement in force; the explicit opt-out. Reported loudly at start, in the log, in the audit trail and by `pii health`.

The asymmetry — writes confined, reads not — is stated in the same words in four places: the profile's grant text, the startup event, `pii health`, and the deploy runbook. One text, so a reader cannot get a more generous impression from one surface than another. The test suite pins the wording, which is unusual and deliberate: a permission model that is described differently in two places is two permission models.

Applying the profile is `apply_workspace`, and it is fail-closed in a way worth reading line by line.

1. The ruleset handles every filesystem access the ABI knows, and is created before any rule is added. A failure here is an error, not a partial ruleset.
2. Every read-only path (`/` and `/dev`) is added as read access. A missing path is skipped rather than fatal, because a container may not have it.

3. Every writable path is added with full directory rights — except device nodes, which get file-content rights only. Granting directory rights on a device file makes the kernel downgrade the whole ruleset to partially enforced, which would silently weaken everything else.
4. The ruleset is applied to the process. If the resulting status is not fully enforced, or `no_new_privs` is not set, the function returns an error. The caller exits `4`.
5. An *out-of-grant* probe writes a file in a directory that must not be writable. If the write succeeds, confinement is not real and the function fails.
6. An *in-grant* probe writes a file in a directory that must be writable. If that write fails, the grant is inert — every rule was skipped or downgraded — and the function fails.

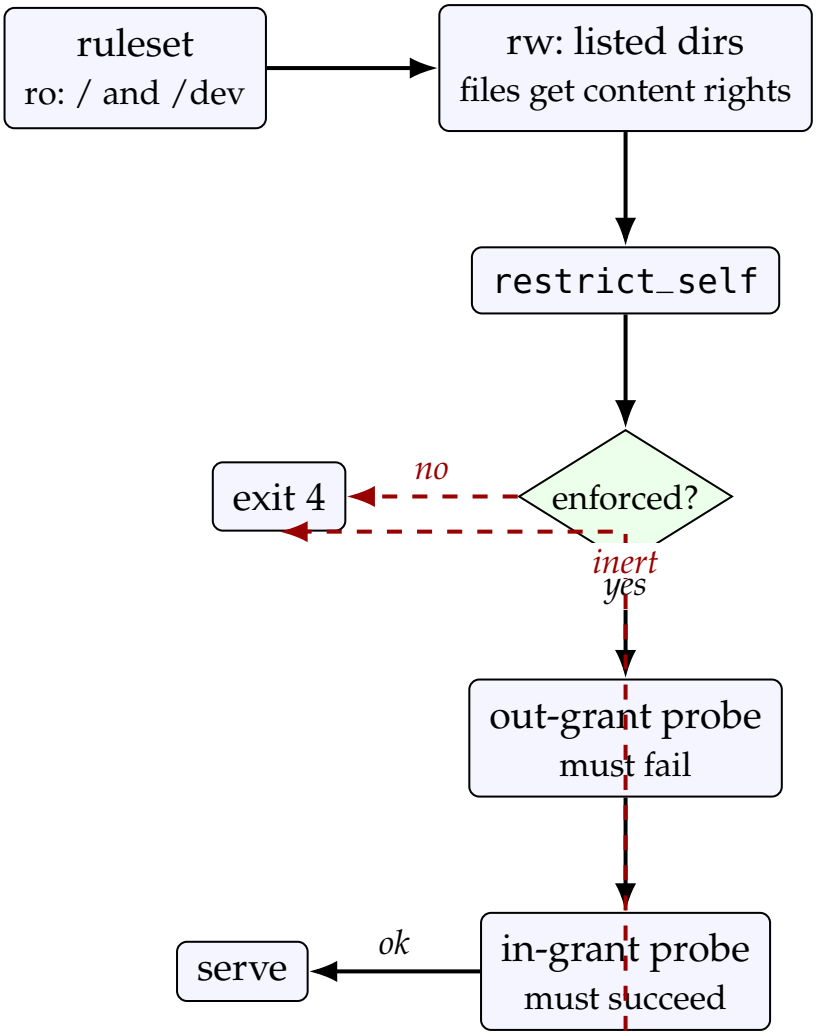


Figure 7.4: Six checks, two of them probes in opposite directions, because without the in-grant probe a ruleset whose every rule was skipped would pass the out-grant test while granting nothing, and every later refusal becomes a coincidence.

The two probes are the part that survives review. A single out-grant probe proves that something is denied; it cannot distinguish "confined correctly" from "confined to nothing". The in-grant probe is the control that makes the first probe meaningful, and it is the reason a report of a green sandbox on a new host can be trusted.

Two grants that are not obvious. Podman keeps its graph root under `HOME/.local/share/containers` and its runroot and locks under `XDG_RUNTIME_DIR/containers` and `XDG_RUNTIME_DIR/libpod`. A sandboxed daemon without those could not inspect a present image: the measured symptom was `EACCES` on `bolt_state.db` and `alive.lck`. The fourth entry is `/dev/shm`, because podman's rootless lock file lives there and the profile grants `/dev` read-only. The podman *socket* is deliberately not a grant: a process reaches it by `connect()`, which landlock does not mediate, so adding it would change the grant text without changing what is allowed. A test asserts the text does not mention it, which keeps the documentation and the mechanism from drifting apart.

The forced-failure switch. `PII_SANDBOX_FORCE_UNSUPPORTED=1` makes `apply_workspace` return the unverifiable error immediately. It exists so the exit-4 path can be exercised on a host where landlock *does* work — a refusal path that is never run is a refusal path that does not work.

7.6 The capability ceiling

The sandbox is process-wide and coarse. The capability set is per-run and per-node, and it is the bound an agent's own instructions cannot argue with.

A capability set is twelve bits plus five scopes. The bits are stable forever, because a worker or a plugin references them without depending on the engine:

Kind	Bit	What it permits
`fs_read`	1	read files within an allow path scope
`fs_write`	2	write files within an allow path scope
`net`	4	reach the network
`proc`	8	spawn or talk to processes
`hw`	16	touch hardware
`secrets`	32	access secrets
`memory_write`	64	write into the agent's memory
`spawn`	128	spawn subprocesses
`memory_read`	256	read from the agent's memory
`post`	512	post into a configured channel
`mqtt_read`	1024	list and subscribe to a house's devices
`mqtt_write`	2048	command a device through its allowlist

The only operation on two sets is intersection. There is deliberately no union: a capability set can be narrowed at every hop and widened at none. `CapSet::intersect` ANDs the bits and intersects every scope; `narrow_kinds` lowers the bits while keeping the inherited path and host scopes, which is what a node that declares fewer capabilities than its run gets.

On top of the per-run set sits a ceiling from configuration: `[grant_ceiling]` names the capability kinds no graph or node may request beyond. An empty list means no ceiling; a name the engine does not

know is an invalid-config error rather than a silently ignored entry. The check runs where the node is built, so a graph asking for more than the ceiling is refused before it runs, and the refusal names the capability.

Path scopes are deny-wins. `PathScope::permits` checks the deny list first, so a path inside an allow prefix but inside a deny prefix is refused. The heal path uses that to keep a patch away from `specs`, `xtask`, `quality-gate.sh`, `graphs`, `clippy.toml`, `rust-toolchain.toml` and `.github`: a heal worktree is a full checkout, and without the deny list a patch could delete the assertions that prove it worked.

```
[core]
model = "llamacpp-server:granite-4.2-3b"

# the ceiling: no graph or node may request beyond these kinds
grant_ceiling = ["fs_read", "fs_write", "proc", "spawn", "net"]

[net]
grants = ["10.10.10.217:8082"]
allow_private = true

[[model_tier]]
name = "small"
model = "llamacpp-server:granite-4.2-3b"
tools = ["read_file", "grep"]
max_tokens = 2048
```

Why the two bounds are both needed. A capability set with no sandbox is advice to a process that has already been spawned by an engine with the operator's rights. A sandbox with no capability set confines a process but gives every node the same power inside the fence. The pairing is what makes "this node may write, that node may not, and neither may write outside the workspace" expressible.

7.7 The socket, and why clients are thin

Every client reaches the daemon through one unix socket, `control.sock`, under the runtime directory. The socket is the boundary that keeps a second copy of the rules from existing: `pii run ls`, `pii health`, `pii queue add` and `pii skill show` all become a request with a method and a payload, and the engine answers. A client cannot be more permissive than the daemon, because it has no engine of its own to be permissive with.

The socket's location is computed once, from the runtime directory, and `pii-uds` owns that computation — one default socket, so a CLI started from a plain shell with no environment finds the daemon that an operator started from a session with one. Before that, `pii health` answered for a daemon that did not exist or failed to find one that did, and the difference was an unset variable nobody could see.

This is also why a long-running command does not hold a store lock. The client sends a request and waits; the work happens in the daemon, in the daemon's process, under the daemon's sandbox and capability ceiling. The `-store` flag on a one-shot command exists for the case where no daemon is running, and then the process really does open the file, which is why a second one-shot beside a running daemon can collide where a socket request would not.

7.8 What the daemon runs

Beyond runs, the daemon owns a small set of long-lived things, each with its own failure mode:

- **Channels** — one runtime per configured channel, with a poll interval, a poll backoff and a graph to start per inbound message. The default poll interval is 1500 ms because Discord answers a message read at roughly five requests per five seconds per route; at 500 ms the daemon collected 173 `channel.poll.failed 429` events in a few hours and never read a message.
- **MQTT connections** — one broker connection each, subscribing to topic filters and starting a graph per message, with a bounded inbound queue that drops and counts beyond its cap. A name used by both a channel and an MQTT connection is a config error, not a last-writer-wins.
- **Cron entries** — display-only schedules; the actual firing comes from the graph's entry `timer` node. The entry kind is checked at start, because a non-timer entry would re-run the instant it finishes.
- **Workers** — declared in config as a program, its argv, extra environment and an in-flight cap, so a bounded number of external processes exist at once.
- **Timers** — the durable wheel, rearmed from the store at boot.
- **Claim recovery** — `requeue_dead_claims` at boot returns rows whose claimant is gone to `Pending`, so a killed daemon does not leave work claimed forever.

7.9 Refusals that name the thing

A refusal is only useful if it names what failed and where. The daemon's refusals follow one shape: the object, the reason, and the identifier.

```
store /root/.local/state/pii/durable/channel.redb is locked: another
    ↪ daemon holds it
cron 'nightly': graph 'night' must start at a timer node (entry is gate);
    ↪ it would re-run
  as fast as it finishes
config: [core].ollama_host is retired (spec 129) - set '[llama] host' for
    ↪ the
  llama.cpp server instead
config: duplicate [[model_tier]] name 'small'
sandbox: enforcement could not be verified
```

Two of those deserve comment. The retired `ollama_host` key is refused *by name* rather than as an unknown field, because the generic message would send an operator looking for a typo. And the duplicate tier name is refused at parse time, so a node asking for the tier gets one provider rather than whichever entry happened to win.

7.10 What an operator should check first

When a daemon is suspected rather than proven wrong, four checks answer most questions without reading code:

1. `pii health -json`: the state of each subsystem, the store's path and per-table counts, and the mounts the process actually has.

2. The log's first line: `daemon.starting` carries the pid and the store parent, so the process and the directory it opened are in the record.
3. The sandbox line in `health`: `sandbox=workspace (landlock)` with the grants spelled out, or the loud `off` text.
4. The store's parent directory, compared with `PII_STATE_DIR` and `HOME`: a daemon reading the wrong store looks exactly like a daemon with amnesia.

7.11 Failure modes worth recognising

1. **A test daemon that hangs on start.** Symptom: no socket, no log line after `daemon.starting`, a test that times out. Cause: it inherited the operator's `PII_STATE_DIR` and is waiting on a held `channel.redb`. Fix: give the test its own state directory.
2. **A gate that fails on a channel lock.** Symptom: `channel.redb is locked: another daemon holds it` inside a gate run. Cause: the gate inherited a state dir from the shell that started it. The gate now removes `PII_STATE_DIR` and `PII_RUNTIME_DIR` from the child environment.
3. **A sandbox that refuses every start.** Symptom: exit `4` on a host where landlock should work. Read the two probe outcomes: an out-grant write that succeeded means the ruleset did not apply; an in-grant write that failed means the grant list is wrong, usually a missing directory the process cannot create.
4. **A podman run that cannot see its own image.** Symptom: `EACCES` on `bolt_state.db`. Cause: the container state dirs are not in the grant list.
5. **A node that "has no write tool".** Symptom: the agent stops and reports that no write or execute tool is available. Cause: the node declared too few capabilities — a coder without `fs_write` cannot implement anything. The fix is in the graph, not in the sandbox.
6. **A cron entry that spins.** Symptom: a graph that starts again immediately after it finishes. Cause: the entry node is not a `timer`. The daemon refuses that at start now.

8 Agents and models

This chapter answers two questions that look like one: which model answers, and which program is asked. They are decided in different places and by different mechanisms. The model comes from a five-level precedence that ends in the daemon's configuration; the *agent* — `pi` or `claude` — comes from the graph, and its command line is assembled explicitly, flag by flag, because every flag in it was added after its absence produced a silent failure.

The chapter also states a limit plainly. The role table that would let a graph say `model = "@workhorse"` is designed and specified (spec 362) and is *not* in the tree at this revision; what exists is a node's explicit model, a node's `tier`, the run's model, `$PII_MODEL`, and `[core].model`. A reader who needs the role table should read the last section of this chapter as a design, not as behaviour.

8.1 The five levels that decide a model

`crates/pii-server/src/tiers.rs` holds the only written-down statement of the order:

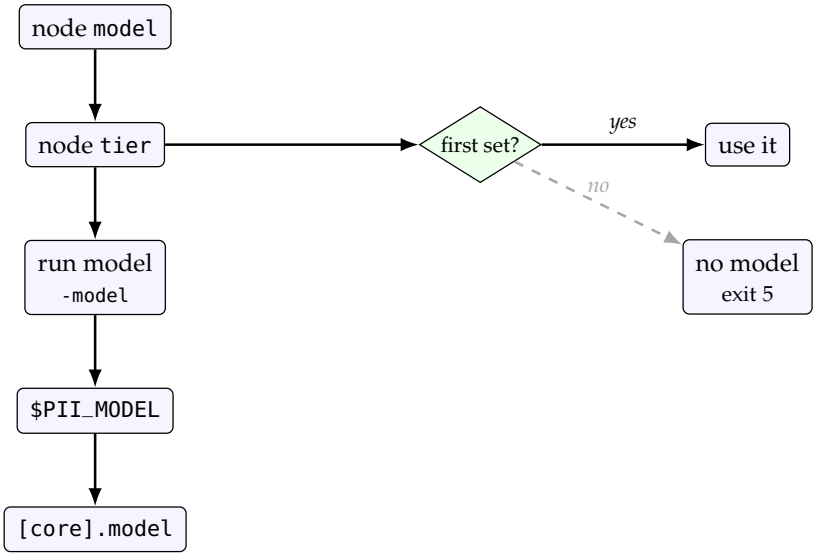


Figure 8.1: Node model beats node tier beats run model beats environment beats config, and the order lives in one function so a caller cannot invent a sixth level in its own file.

Two consequences follow. A graph that names a model explicitly keeps working exactly as it did before tiers existed — which is why the precedence has node model at the top rather than the bottom. And a daemon with no model anywhere refuses to start with exit `5` rather than starting and failing at the first dispatch: the failure belongs where the operator is watching.

Tiers. A `[[model_tier]]` entry names a model class a node can ask for by name. Each entry has a name, a provider spec, an optional tool allowlist, and an optional `max_tokens` cap. The name must be non-empty and unique, and both are checked at parse time, because a duplicate would make a node's `tier` resolve to whichever entry won and the operator would have no way to know which.

The tool allowlist is enforced by the capability layer, not by the prompt. That distinction is the whole point of the field: a tier that may call three tools cannot be talked into calling a fourth, and no instruction in a brief can widen it, because the check happens where the tool is dispatched. The allowlist only ever shrinks: intersecting two sets takes the names present in both.

8.2 Agent argv, assembled by hand

The worker node's spec is read from graph data: `agent`, `model`, `spec`, `workdir`, and optionally `provider`, `sid`, `session`, `run`, `max_budget_usd`. `WorkerSpec::from_cfg` validates each optional field before anything is spawned: `max_budget_usd` must be a positive finite number (a typo can never become "no cap" or a flag the CLI rejects after the dispatch started), a claude `sid` must be a lowercase UUID v4, and a `run` must match `[A-Za-z0-9._-]+` because it names a file.

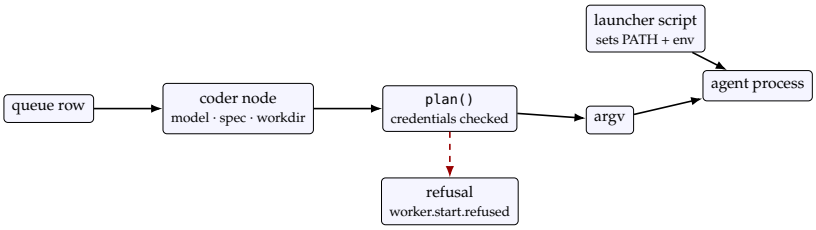


Figure 8.2: Planning is pure and never spawns, and the spawn goes through a named launcher script that sets the environment inside itself rather than inheriting the caller's.

For `pi`, the argv is built in this order and for these reasons:

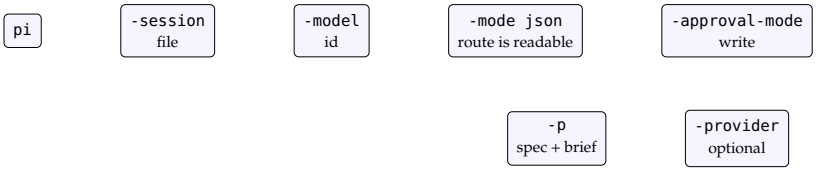


Figure 8.3: Every element is a decision that has a failure behind it, not a default that was copied from a shell history.

Flag	Value	Why it is there
`-session`	`<workdir>/.pii-sessions/<node>-<run>.jsonl` or the node's `session`	one session file, and its path is what makes a resume possible
`-model`	the resolved model id	the only place the model is stated

Flag	Value	Why it is there
`-mode json`	`json`	the event stream names the provider and model per message, so the run's real route can be read back
`-approval-mode`	`write`	headless runs have no approval surface; see below
`-provider`	the node's provider, when set	pins the route; the key is *not* on argv
`-p`	`Spec: <path>` and the brief	the brief says "the spec named above", so the spec must be named

```
# what a pi dispatch looks like, assembled by plan()
pi --session /repo/.pii-sessions/376-retention-r000000000412.jsonl \
   --model deepseek-v4.1-flash:cloud \
   --mode json \
   --approval-mode write \
   --provider ollama-cloud \
   -p "Spec: specs/376-retention-drops-what-the-policy-expires.md

Implement the spec named above. Follow it as the contract. Do not commit."
```

For `claude` the argv is shorter and different in kind:

```
claude --session-id 4f1c2a9e-8b7d-4c31-9a02-6d5e0f7b1c88 \
       --model claude-haiku-5-5 \
       --permission-mode bypassPermissions \
       --output-format stream-json --verbose \
       --max-budget-usd 0.25 \
       -p "<brief>"
```

Four decisions are visible in that line. `-output-format stream-json` with `-verbose` is what makes the session id, the cost and the model readable after the fact; partial messages are *not* requested, because one measured run produced 3779 `stream_event` lines and passed the capture bound. A resumed run carries `-resume <sid>`; a fresh run carries `-session-id <new>` when the caller chose one, so the handle exists before the spawn. The two flags are never both emitted. And no provider flag and no key appear at all: claude reads its own credential.

8.3 Approval modes, and what they are not

Approval modes separate *writing* from *judging*. They are not the security boundary, and the chapter states that first because it is the mistake to make.

`pi`'s default is `always-ask`. Headless, there is no surface to ask on, so every mutating tool is denied. The measured result was a coder that ran for 122 seconds, exited 3, and reported:

```
approval required for ‘bash‘ but this session has no approval surface
```

A coder in that state has written nothing and has reported a plausible reason. The fix was `-approval-mode write`, which approves file edits and still refuses to run commands. That is the right split for this architecture: the coder writes, the judge runs the exit criteria, and the daemon's capability set stays the outer bound. A coder that could run arbitrary

commands would be running the very checks that are supposed to judge it.

`claude` has the same problem with a different flag. Headless `-p` cannot ask for permission, so without a mode it cannot write at all; the dispatch passes `-permission-mode bypassPermissions`. The name is alarming and the reason it is acceptable is the same: the process is inside the sandbox, under a capability set that decided what it may do before it started, and its work is judged by criteria it cannot edit.

Agent	Mode used	Can write	What still stops it
`pi`	`write`	files only	the capability set, the sandbox, and the judge
`pi`	`always-ask` (default)	nothing	nothing needs to: the run does nothing and exits 3
`claude`	`bypassPermissions`	yes	the capability set, the sandbox, the judge, and `-max-budget-usd`

8.4 Credentials are checked before the spawn

A dispatch that names a provider must have that provider's key in its environment. The table is two entries — `ollama-cloud` reads `OLLAMA_API_KEY`, `deepseek` reads `DEEPSEEK_API_KEY` — and a provider not in the table has no key variable, so the check fails and the dispatch is refused. That is deliberate: an unknown provider is refused as a missing credential rather than attempted, because attempting it costs a spawn and produces a provider error instead of a configuration error.

The refusal is journalled as `worker.start.refused` with the agent, the model and the reason, and the node fails without a process. This is the difference between a run that fails in one second with a nameable cause and a run that fails in ninety seconds with a stack trace from a subprocess.

The key itself never reaches argv. For `pi`, the value is resolved at runtime from an environment reference in a pii-owned `models.json` (`apiKey = "$ENV:<NAME>"`), with `PI_CODING_AGENT_DIR` pointing at that directory. The measured proof is a run that exited 0 through ollama-cloud with no `-api-key` on the command line. An argv key would be visible in `/proc/<pid>/cmdline` to every process on the host.

For `claude`, the credential check accepts either an environment token (`ANTHROPIC_API_KEY` or `CLAUDE_CODE_OAUTH_TOKEN`) or the host's login file with an OAuth block. Only the block's *presence* is checked; its values are never read into the journal.

8.5 The brief must name the spec

The brief is the text handed to the agent, and the code prefixes it with the spec path:

```
let prompt = if spec.spec.is_empty() {
    brief.to_owned()
} else {
    format!("Spec: {}\n\n{brief}", spec.spec)
};
```

That two-line prefix exists because of a measured failure. A coder was spawned with the spec in its session *name* and nothing about the spec in its prompt, and it implemented a different spec's files: the row said one thing, the tree changed another, and the criteria that failed belonged to work nobody had asked for. The failure is invisible in the argv log unless the prompt is logged, which is why the prefix is in the code and pinned by a test rather than left to a brief author's memory.

An empty brief is refused. A whitespace-only brief runs the agent with no prompt; it exits 0 at once, and a criterion that already passes makes the node succeed having done nothing. The coder refuses such a brief before it spawns. This is the general shape of a guard that exists because a passing check can be worse than a failing one: success-with-no-work is the hardest failure to notice.

8.6 Two verdicts, and only one of them counts

The agent's report is a self-report. A coder can exit 0 having written nothing, or having written the wrong thing, or having weakened a test until it passed. So the node has two inputs and one rule.

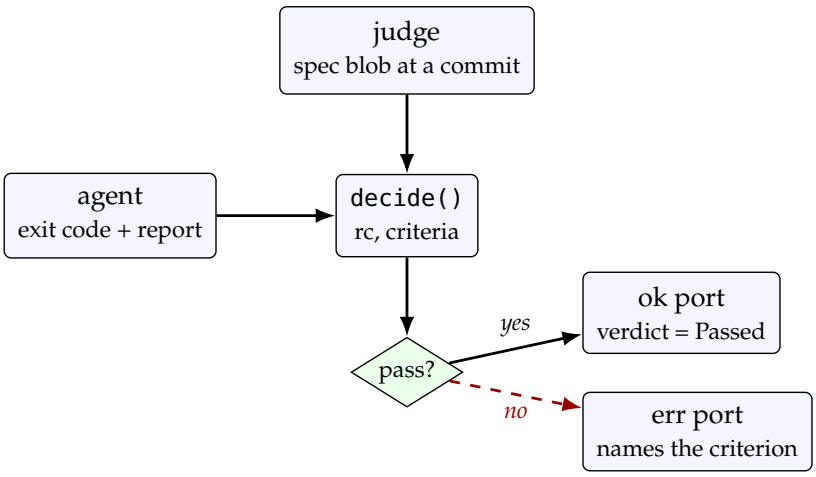


Figure 8.4: The worker decides nothing: its exit code is one input, the criteria runs are the other, and any non-zero criterion fails the node even when the worker exited cleanly.

`decide` has three rules and no exceptions:

1. A non-zero worker exit is a failure, named `worker exited <n>`.
2. Zero criteria run is a failure, named `no criteria were run`. This is the rule that catches a node whose criteria list was silently empty.
3. The first criterion with a non-zero exit is a failure, and the message carries the criterion command. Every criterion still runs, so the verdict can name more than one problem in the log even though the message names the first.

Where the criteria come from. The spec-mode judge reads them from the spec's blob *at a commit*, not from the working tree, the run input, or the agent. A worker that edits its own spec gains nothing: the judge asks git for the document that was reviewed, parses the `## Exit Criteria` section, takes the first backticked span of every checkbox line, and runs each command in the node's workdir. A base that is not exactly forty lowercase hex digits is refused before git is asked, so a malformed handle is a refusal and not a mystery.

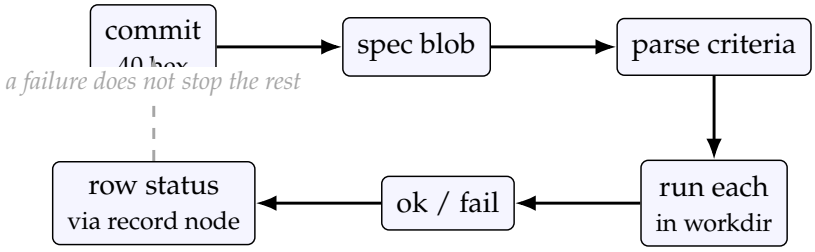


Figure 8.5: The document is read at a commit, so the verdict is about the spec that was reviewed rather than the tree the agent left.

8.7 The ladder, and when to climb it

A heal ladder is an ordered list of rungs, cheapest first, each a pair of agent and model. `escalate` returns rung zero with no history, and otherwise the next rung only when *every* previous attempt failed *with evidence*. An attempt that passed, or that failed without naming a failing criterion, ends the ladder.

The rule sounds fussy and is the difference between escalation and retrying. Evidence is a failing criterion: a specific command that returned non-zero. Without it there is nothing to escalate *about* — the next model would be handed the same problem statement and no new information, and the spend would buy a second opinion on an unknown question. With it, the next rung starts from the command that failed.

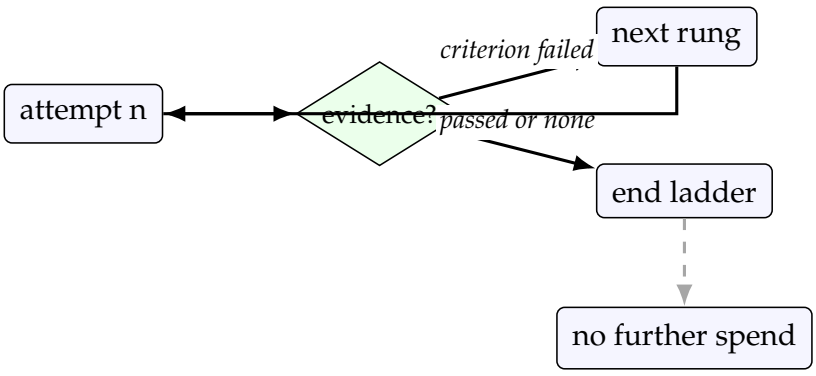


Figure 8.6: Escalation is gated on evidence, so a ladder cannot become a slot machine and a ladder whose rungs all lack evidence never climbs at all.

8.8 The session is the resume handle

A dispatch records a session before it can need one. For `pi`, that means writing a session header first, because an empty or missing session file is rejected before any model is called. The header is the shape the installed `pi` emits — `type`, `version`, `id`, `timestamp`, `cwd` — and the timestamp comes from the caller's clock, so a replay sees the same value. An existing file is left untouched: a resumed session keeps its identity.

For `claude`, a fresh run names its session on the command line (`-session-id <uuid>`), so a crash after the spawn still journals the handle. A resumed run carries `-resume <sid>` instead.

The session id is the only handle that survives a turn cap or a killed process, and it is unrecoverable once the process exits. That is why the dispatch scripts write it to a file at start, and why the queue's recovery path reads the worker's own log when the dispatcher never got to write it (chapter **??**).

What the run reports about itself. `reported_routes` reads the agent's `-mode json` output and collects the distinct `(provider, model)` pairs it named. The distinction matters: a node asked for one route and the agent may have used another, and an empty result means the output named no route at all — which is reported as unknown, never as a match. The same output yields the cost, the turn count and claude's own error flag, all optional, because a killed run has an `init` line and no `result` line.

8.9 The role table that is specified and not landed

Spec 362 exists because seven graphs hardcoded a model and three of them named an id that no longer existed. The measured cost was concrete: the first real run of `graphs/queue.toml` died in 1.6 seconds with

```
{"code":"internal","exit_code":1,"message":"Model deepseek-v4-flash not found"}
```

and eight rows would have parked for one dead id. Two graphs already took the model from the run input, which is why a stale id can hide for months: nothing validates it until an agent is spawned.

The design is one config table per role and a role reference in a node:

```
[models.workhorse]        # implements a spec
model = "deepseek-v4.1-flash"
provider = "ollama-cloud"

[models.planner]          # writes a spec
model = "glm-5.3"
provider = "ollama-cloud"
```

with R2 allowing `model = "@workhorse"` in a node, resolved when the node is built; R3 refusing an unknown role *by name* and listing the configured roles, before any agent is spawned; and R4 refusing an entry whose model is not on the provider's list when the provider is reachable, while saying so and continuing when it is not — an offline start must not depend on the network.

R4 is the half that makes it real, and the spec says so in its own words: a worker can add the table, resolve the role in one node kind, and leave every other call site hardcoded, so the config exists, nothing uses it, and the next stale id hides in a graph nobody reads.

Requirement	What it says	State at HEAD	Criterion
R1	one table per role, no model id in a graph	not landed	`grep workhorse` in `config.rs` fails
R2	node may name `@workhorse`	not landed	`grep '@workhorse' graphs/queue.toml` fails
R3	unknown role refused by name	not landed	`models_from_config` test file absent
R4	dead model refused at config time	not landed	same test file
—	no graph names a dead id	partially	`deepseek-v4-flash` still in four graph files

The parked-backlog review of 2026-11-04 recorded the verdict as `needs-decision`, with one open question for the operator: whether to raise the 450-effective-line budget for `config.rs` or split the file further. The lesson recorded with it is the general one — the file-size guard counts effective lines, so a clean extraction that moves code without deleting it stays over budget.

Until that is answered, the working rule is the one the tree enforces: a graph names its model explicitly, and the daemon's `[core].model` is the fallback for graphs that take it from input. A reader who wants a role table should treat the section above as the design and check `graphs/queue.toml` for whether it has landed.

8.10 Failure modes worth recognising

1. **A coder that reports success and wrote nothing.** Cause: no approval mode, every mutating tool denied, exit 3 with a plausible sentence. Check the argv before the report.
2. **A coder that implemented another spec.** Cause: the spec path never reached the prompt. The prompt prefix is the fix; the symptom is a tree that matches the row's title and not its criteria.
3. **A dispatch that dies in about a second with "model not found".** Cause: a stale id in a graph. This is the failure the role table is meant to move from spawn time to config time.
4. **A dispatch refused with "no credential for provider X".** Cause: the provider is not in the two-entry key table, or its key is unset or empty in the daemon's environment. The message names the provider, so the fix is visible without reading code.
5. **A run whose route is unknown.** Cause: text output, or output not captured. Unknown is reported as unknown; a missing route never reads as a match.
6. **An escalation that never climbs.** Cause: the previous attempt failed without a failing criterion, so there is no evidence to escalate on. The ladder is doing what it says.

9 Gates, controls and tests

This chapter answers what the gate actually runs, what a green result does and does not prove, and why so much of the surrounding machinery is about the *shape* of a check rather than its subject. The gate is one list of twenty steps, executed in two different trees, with the environment sanitised so that a result cannot depend on who started it. Around it sit the rules that decide whether a check is worth having: a criterion that passes before the work measures nothing, a rule that suppresses needs a control that must still take the normal path, and a check must consume what the live path produces rather than a copy of it.

9.1 One list, read twice

`gate-steps.txt` is the gate. Its header says so, and the format is one line per step:

```
# Format: <label>|<shell command>   (one line per step; # lines and blank lines are ignored)
# A step passes only if its command exits 0; every step runs even after an earlier failure.
fmt|cargo fmt --all --check
clippy|CARGO_TARGET_DIR=".../clippy-gate" cargo clippy --workspace --all-targets -- -D warnings
build|CARGO_TARGET_DIR=".../build-gate" cargo build --workspace && cp ... target/debug/pii
lint-files|CARGO_TARGET_DIR="_tmp/tools-gate" cargo run -q -p xtask -- lint-files
```

`quality-gate.sh` reads that file for both modes; nothing else lists the steps. The local run iterates the file, calls `run_step` for each line, and keeps a status that any failure sets.

```
while IFS= read -r line || [ -n "$line" ]; do
    case "$line" in '' | '#'*) continue ;; esac
    run_step "${line%%|*}" "${line#*|}" || status=1
    steps=$((steps + 1))
done <"$GATE_STEPS"
# An empty or unreadable list must never read as green.
if [ "$steps" -eq 0 ]; then
    echo "FAIL: gate-steps.txt lists no steps"
    status=1
fi
```

Two properties are in that excerpt. Every step runs even after an earlier failure, so one run produces the whole list of problems rather than the first one. And an empty list is a failure, not a pass: a gate that runs nothing and exits zero is the most dangerous possible configuration, because it looks exactly like a gate that ran everything.

9.2 The twenty steps

The list has grown by addition, and each addition has a reason recorded next to it in the file. The table below is the inventory: what each step runs, and the class of defect it exists to catch.

Step	Runs	Why it exists
`fmt`	`cargo fmt -all -check`	zero-tolerance formatting
`clippy`	`clippy -workspace -all-targets -D warnings`, own target dir	one `-all-targets` rlib hash can poison the doctest linker (E0463)
`clippy (hal-gpiod)`	`clippy -p pii-hal -features hal-gpiod`, own target dir	the real GPIO backend is feature-gated off everywhere else; compile and lint only, never run
`build`	`cargo build -workspace`, then publish `target/debug/pii`	the worker's supervisor tests spawn the real binary at a fixed path
`test (lib/bins/tests)`	`cargo test -workspace -lib -bins -tests`	the suite
`test (doc)`	`cargo test -workspace -doc`, own target dir	doctests build libs without `cfg(test)`; sharing a target dir picks a stale rlib
`coverage-floor`	`cargo llvm-cov` then `xtask coverage-floor`	line coverage must stay at or above `coverage-floor.txt`
`lint-files`	`xtask lint-files`	450 effective lines per file, plus spec/doc hygiene
`lint-scratch`	`xtask lint-scratch`	a test that uses a crate-local `_tmp` it never creates
`lint-observability`	`xtask lint-observability`	every crate uses the `pii-observe` seam, no new raw prints, registry names in `ALL`
`reachability-check`	`xtask reachability-check`	a declared capability seam has a real non-test caller
`deps-check`	`xtask deps-check`	five dependency invariants, from `cargo metadata`
`net-check`	`xtask net-check`	sockets and name resolution only in allowlisted files
`doc-drift-check`	`xtask doc-drift-check`	every backticked repo path in `HANDOFF.md` exists
`autonomy (shell tests)`	`bash scripts/autonomy/run-tests.sh`	tests `cargo test` cannot see, without this the gate cannot go red on them
`plan-verify`	`sync-verify.sh -check`	an open row's verify column must still be its spec's criteria
`item-ids`	`item-ids.sh -check`	a flow item's id is its work's spec number
`devbox-check`	`modules/devbox/check.sh`	the devbox module's offline claims hold

Step	Runs	Why it exists
`install-config`	`deploy/install-config-keeper.test.sh`	keeps the operator's config
`daemon-guard`	`scripts/checks/daemon-test-guard.sh`	test that guarded the daemon owns it through `DaemonGuard`

The autonomy step's history is the clearest argument for the list being a file rather than a habit. `tick-message-worker-rc.test.sh` was red for two days after a change to `tick.sh`'s phone line, and every gate in that window was green, because `cargo test` never reached a shell test. A check that is not in the list does not exist.

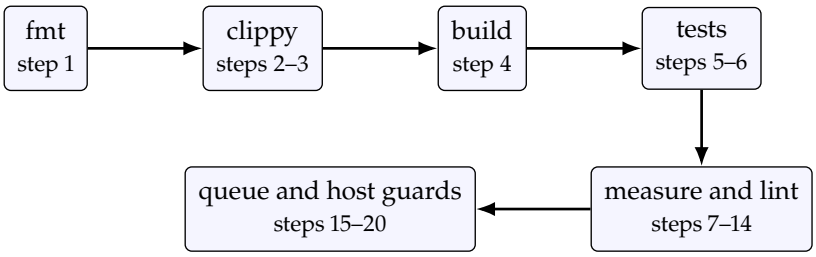


Figure 9.1: The twenty steps as five stages and a tail of guards, where the numbered groups are the ones a reader will otherwise have to count in the file.

9.3 Two trees

The same list runs in two places, and the difference is what the result proves.

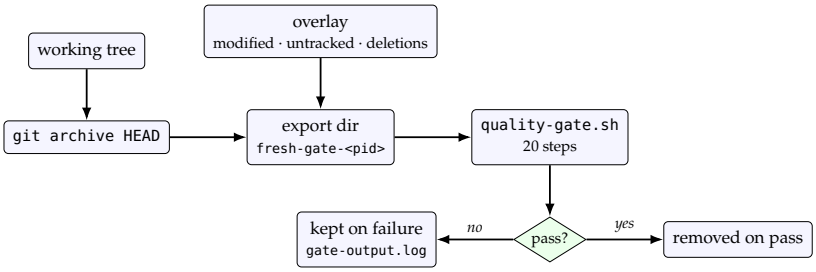


Figure 9.2: When the two trees disagree the export is the one to believe, because tests that pass only because an earlier run left a directory behind are the class the second tree catches.

`quality-gate.sh` has three entry points: `-routine` (default, this tree, warm cache, minutes), `-fresh` (the export, the release boundary), and `-summary` (prints what the gate cannot exercise and runs nothing). The fresh mode is a wrapper around `cargo xtask fresh-gate -worktree`, with a separate target dir, because an export's artifacts must not land in the warm cache: a test binary bakes its manifest directory at compile time, and a routine run that reused an artifact from a deleted export failed three tests on a tree where the package's own suite passed 5/5.

Overlay, including deletion. `overlay_worktree` copies every modified and untracked path from the working tree into the export, and then *removes* from the export every path git reports as deleted. The second half was added after a change that removes a test file could not be gated at all: the export still held the deleted file while the overlaid manifest no longer declared its dependency, so the gate failed on a tree that does not exist. A gate that cannot express a deletion is not testing the change.

Single-file substitution. `-overlay src=dest` copies one file over the export. `dest` must be relative and must not contain `..`; both are refused at argument parsing, before anything is created. This is the way to test a deliberately mutated file without touching the tree.

Which list ran. `check_step_list` reads `GATE_STEPS`, counts the non-comment lines in the *export*, and refuses a reduced list unless `-quick` was asked for:

```
GATE_STEPS=gate-steps-quick.txt cargo xtask fresh-gate --worktree
# REFUSED: GATE_STEPS=gate-steps-quick.txt is the ITEM list (5 steps), not
    ↪ the gate that decides
# shipping. Unset GATE_STEPS (or set it to gate-steps.txt) for the full
    ↪ list, or pass --quick if that
# reduced run IS what you intend - it will say so in its own output.
```

That refusal exists because of a measured evening: an autonomy script exports the quick list for the item boundary, a hand run that sourced it reported "gated" five times, and the commit gate — which runs the full list — had three real failures waiting. The quick list is a legitimate tool. It must not be the accidental default, so it is allowed only when asked for, and the run always prints which list it used and how many steps it holds.

9.4 The environment must not be inherited

A gate that depends on the caller's environment is not a gate. Two variables are removed from the child and one is redirected:

- `PII_STATE_DIR` and `PII_RUNTIME_DIR` are removed. Measured: a lane daemon holding the real store made a daemon test fail with `channel.redb is locked: another daemon holds it`, because the test inherited the state dir. The test's own store parent was not enough — the durable stores follow the variable.
- `CARGO_TARGET_DIR` points at the warm cache, so the export's tree is fresh and its compiler output is not. Pinning it inside the export made every attempt pay a full cold workspace build: a spec that took minutes took 40–60 minutes after the export entered the loop.
- `TEST_GATE_TARGET` points *inside* the export for the test step only, because a test binary bakes `CARGO_MANIFEST_DIR` and a binary left in the shared cache would be reused by the next run.

The third is the subtle one, and it has its own test with a control: the test target must start with the export's path, must not equal the shared cache's test directory, and every other step must still get the warm cache. Without the control, a change that redirected *every* step would pass.

9.5 Disk, retention and the two ceilings

A fresh export is about 8 GB before it builds anything, and a coverage-measuring export needs at least 11.4 GiB free. Both numbers are measurements, not estimates: a run with 11 GB free drove the volume to zero for about three minutes and left an 11.4 GB failed export behind. The floor is `FRESH_GATE_MIN_FREE_MB`, default 12 288, checked before the export directory is created, and the refusal names the free megabytes and the override.

The warm cache has the opposite problem: a bigger cache builds faster, so nothing bounded it and it reached 105.8 GiB. It is now capped by `FRESH_GATE_CACHE_MAX_GB` (default 40) and trimmed before the build — but never while a cargo or rustc is live, because a running build would lose its target underneath it. When a build is live the tool says so and skips the trim rather than guessing.

Retention by liveness. Failed exports are removed before a new one is created, but the predicate is not age. Each export directory is named for the pid that created it (`fresh-gate-<pid>`), and retention asks whether that pid still exists. The measurement behind this is worth repeating: retention once removed a *running* gate's export, and that gate then failed sixteen steps on `target/gate-steps/<step>.log: No such file or directory` with `failing tests: []` — a red verdict blaming the tree for a directory another process deleted.

Two tests pin the pair: a running owner's export is not reaped, and — as the control — a dead owner's export is. The control matters because "never remove anything" would also pass the first test.

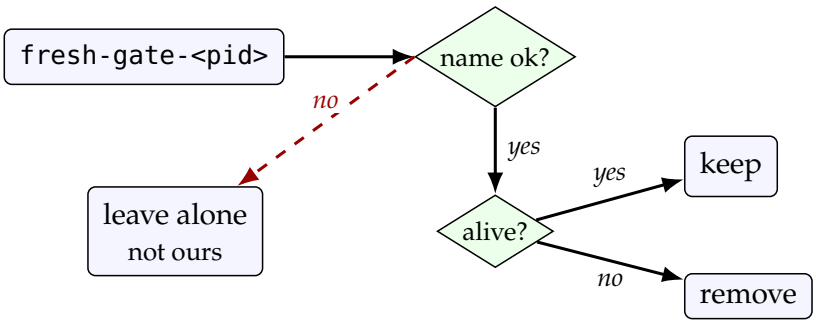


Figure 9.3: The name guard runs first and fails safe: an empty base, a foreign directory or a bare prefix is refused, so a guard around a deletion cannot pass on an unset variable.

The export can still disappear. After the gate returns, the runner checks that the export directory is still there. If it is not, the result is reported as its own condition — "export lost: another gate's retention removed it while this gate ran; the step failures above are not a verdict on the tree" — rather than as a failure of the code. A gate that reports the wrong cause is worse than a gate that reports nothing.

9.6 What a step is allowed to say

`scripts/gate-lib.sh` runs each step in its own session, streams its output, and prints a heartbeat at least every 30 seconds with the step's last output line. Three verdicts exist:

- `PASS: <label>` — the command exited 0 and printed no `SKIPPED:` line.
- `SKIPPED: <reason>` — the command exited 0 and said it skipped. There is no PASS line; the skip is the record. `QG_SKIP_COVERAGE=1` uses this.
- `FAIL: <label>` — non-zero, or killed by a signal, or hung. A step whose output has not changed for `QG_STALL_SECS` (default 1 800) is killed and reported as `HUNG` with the seconds since its last heartbeat and its last line.

The heartbeat is a progress note only. The exit code is the sole verdict, which is why the stall detector exists: a step that prints nothing for thirty minutes is indistinguishable from a step that is working, and the difference has to be decided by a bound.

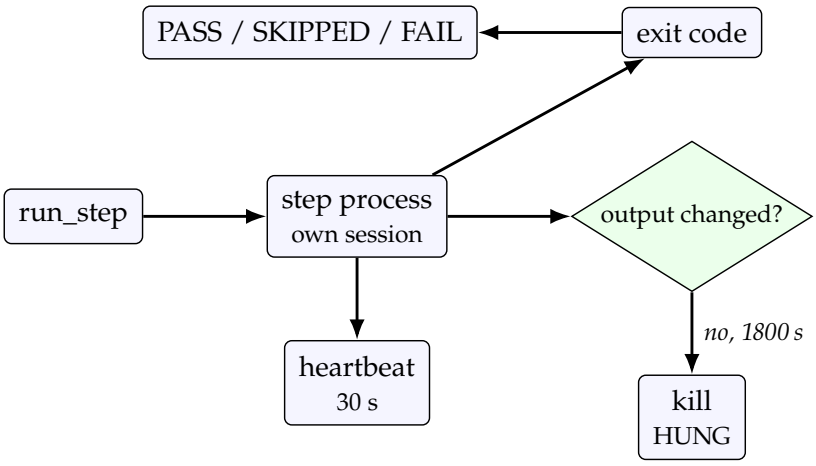


Figure 9.4: Liveness and verdict are separate signals, so a step can be alive and wrong or dead and therefore not a verdict at all.

9.7 Controls

The single most useful rule in this repository is that every rule which suppresses, skips or special-cases something needs a control that must still take the normal path. A suppression without a control cannot be distinguished from a broken check.

The rule	The near-miss that must still happen	Where it is pinned
a running gate's export is not reaped	a dead owner's export *is* reaped	`fresh_gate.rs` tests
the test step's target lies inside the export	every other step keeps the warm cache	`gate_env` test

The rule	The near-miss that must still happen	Where it is pinned
the sandbox refuses an out-of-grant write	an in-grant write must *succeed*	`sandbox.rs` probes
a failure report echoes the panic block	a build error with no test failure yields nothing	`failure_report` test
a graph refuses an unreachable callee	the reachable callee is still called	splice tests
a dirty tree stops dispatch	a row that owns its own dirty tree may resume	lane tests

The sandbox pair is the clearest illustration of why the rule exists. A ruleset whose every rule was skipped passes an out-of-grant probe while granting nothing; every later refusal is then a coincidence that looks like policy. The in-grant probe is what turns "something is denied" into "this is granted and that is not".

9.8 A criterion that is already green measures nothing

An exit criterion is a shell command that fails today. If it passes at `HEAD`, before any work, it is not a criterion; it is a wish, and the work it describes may already have landed or may be described by something else entirely. The rule that follows is mechanical: every new check is run against `HEAD` first, and if it passes it is rewritten or withdrawn.

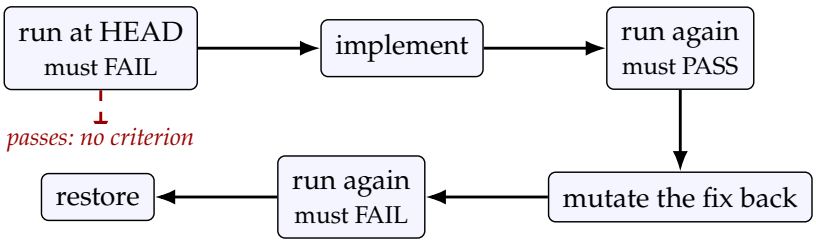


Figure 9.5: Four runs, not two: the middle pair proves the work did something and the last pair proves the test is about the work rather than about the tree being green that day.

The fourth step is the one that gets skipped, and it is the one that catches the most. A test can pass for a reason unrelated to the fix — another change in the same commit, a fixture that happened to be left behind, a guard that fires on its own control. Mutating the fix back and watching the test go red is the only evidence that the test is about the fix. It is also how a partly-guarded fix is discovered: mutate one of three guards and see whether anything notices.

Where this shows up in the store. The retention work is the clearest example in this manual. The criterion was a test that seeds an expired run and asserts it is gone, plus its opposite: a seeded `Running` run that must

survive. At `HEAD` the first failed because there was no retention code at all — the search for `retention|prune` in `pii-store/src` returned nothing. After the implementation both passed. The mutation that matters is deleting the live-run filter: the first test still passes, the second goes red. A test suite with only the first would have certified a sweep that can delete work in flight.

9.9 The three lints that encode a convention

Three gate steps exist because a convention that lives only in a document is a convention that will be broken. Each encodes a rule the compiler cannot express, and each is a step only because it can fail.

- **`lint-files`** enforces 450 *effective* lines per file, where effective means non-blank and non-comment. The distinction is not pedantry: a clean extraction that moves code into a new file without deleting anything leaves both files over budget, which is exactly the decision that parked the model-role work. The step also runs spec and doc hygiene checks.
- **`lint-scratch`** fails on a test that uses a crate-local `_tmp` directory it never creates. The sanctioned helper is `pii_store::testing::tmp_scratch`, which creates the crate-local directory *and* the temp dir inside it, so a test cannot depend on a directory a previous run happened to leave. This class cost four separate fixes, because leftover state made the repository gate look green while a clean export failed.
- **`reachability-check`** proves that a declared capability seam still has a real, non-test, non-comment caller. It was added after a resolver existed, was tested, and was called by nothing: the test exercised the function directly, so it passed, and the feature did not work.

The three share a shape: the rule is stated in prose somewhere, the prose is not loaded by the compiler, and so the rule is restated as a step that exits non-zero. A rule that lives only in a reference document is a rule that will be broken again — a sentence this repository has had to learn twice, once about dispatch scripts and once about commit identity.

9.10 The coverage floor

`coverage-floor.txt` holds one number, `74.0`, with the measurement that justified it recorded in its comments: 77.02% (21 741 of 28 229 lines) on 2026-10-10, with cargo-llvm-cov 0.9.1 over lib, bins and tests, no doctests. The floor is enforced, not reported:

- A missing `cargo-llvm-cov` or the llvm-tools component is a FAIL, not a skip. A tool that is absent cannot measure, and an unmeasured floor is not a floor.
- `QG_SKIP_COVERAGE=1` skips only that step and prints a `SKIPPED:` line naming the variable. It is never implied.
- A queue commit refuses a gate that passed without measuring: the cycle reads the log for the skip line and parks the row as `gate-coverage-not-measured`. The distinction between "the floor held" and "the floor was not measured" is the whole point.

- The floor is raised when coverage rises and never lowered to make a run green.

9.11 What the gate cannot see

A gate's limits belong next to its verdict, or the verdict will be over-read. Four are known and written down:

1. **The `hal-gpiod` steps are unexercised.** They compile and lint 117 lines that have never run on hardware, because this host has no `/dev/gpiochip*`. The gate prints that sentence at the end of every run, so a PASS there cannot read as hardware coverage.
2. **`net-check` is a text scan.** It misses a symbol built by a macro or `concat!`, an alias, and any non-`.rs` source; and its allowlist is per-file, so anything inside an allowlisted file is unchecked.
3. **`deps-check` reads metadata.** It cannot see a dependency that arrives through a `#[path]` include or a `use` of a crate already in the graph.
4. **`doc-drift-check` reads one file.** It checks that backticked paths in `HANDOFF.md` exist, and nothing about whether the prose is true.

The largest limit is not in that list: a green gate is not a correct change. It says the checks passed, not that the change did what the spec asked. For a landed fix the check is to mutate it back and watch the test go red, which is the only evidence that the test is about the fix.

9.12 Gates that fired on the case they allowed

The recurring defect in this codebase is not a missing guard. It is a guard that fires on the case it exists to permit, while reporting a reason that is not the real one. Six are recorded in the manuals' own margin, and each one cost hours:

- a **park** that left its working tree dirty, so the dispatcher refused every later row for a night;
- a **dirty-tree guard** that stopped the row whose own work it was (row 368 was parked by the guard it was written to permit);
- **prose in a phase field** read as a marker, so a dispatch was skipped;
- a **resolver nothing called**, certified as working by a test that hand-wrote the input;
- a **sweep** that deleted rows it could not date;
- a **gate** that inherited the caller's environment and failed on a lock the caller held.

The defence is the same every time: make the check consume what the live path produces, and keep a control that must still take the normal path.

9.13 One document that had drifted

`docs/02-guides/quality-gate-and-fresh-tree.md` describes the gate as fourteen steps and the free-space floor as 2 048 MB. At this revision the list has twenty steps and the floor is 12 288 MB. The document is still

the best explanation of *why* each early step exists, and it is wrong about two numbers a reader would act on.

That is not an argument against documentation; it is an argument for the rule that the script is the source of truth for the step list and the document explains it. When they disagree, the script wins — which is exactly why the gate prints its list and its step count at the start of every fresh run.

Development workflow 10

This chapter answers how a finding becomes a commit here, and why the answer is a queue of rows rather than a conversation. The loop is: a finding, a spec that a cheaper model can implement, a row that carries the spec and its criteria, a lane that dispatches a worker, a verify step, the gate, a commit — and, when any of those fail, a park that keeps the work and moves the queue on. The parts that matter are the ones that keep the loop honest: the row cannot drift from its spec, a park cannot wedge the queue, a resume cannot become a retry, and a commit cannot name a tool.

10.1 The loop

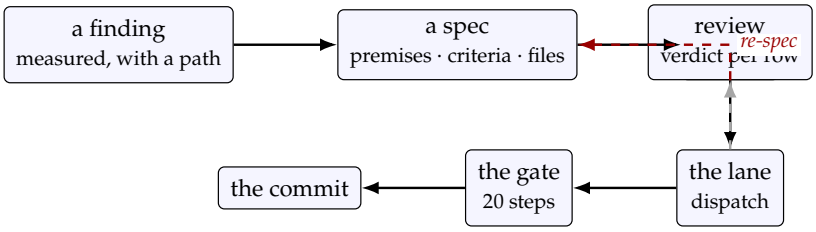


Figure 10.1: The red edge is what keeps the queue honest, because a parked row is reviewed and returns as a better spec rather than as a retry.

Each hop has a file behind it. The finding is a paragraph with a measured path and a command that reproduces it. The spec is a numbered document under `specs/`. The row is a line in `scripts/autonomy/plan.tsv`. The lane is a dispatch script. The gate is `quality-gate.sh`. The commit is one commit with a neutral identity. Nothing in the loop depends on a session's memory, which is what lets a cycle be killed and resumed without losing the thread.

10.2 The spec is written for a model that cannot guess

A spec here has a fixed shape, and the shape is the point: a cheaper model implements it, so every fact the implementer would otherwise have to guess is written down.

- **Why this exists** — the failure, measured, with the number that makes it a failure.
- **Verified premises** — facts measured on a date, each with the path or command that produced it.
- **Requirements** — behavioural, numbered, each checkable.
- **Files** — the paths the work may touch.
- **Exit criteria** — shell commands that fail today.
- **Guardrails** — what must not be done, including the tempting wrong fix.

Spec 376 is a good example of the first two: it states that `pii-store` has no retention code at all, and it proves it with a search whose count is zero. Spec 362 does the same for the model ids: seven graphs hardcode a model, three name an id that no longer exists, and the first real run died in 1.6 seconds with the provider's own message quoted.

```
## Exit Criteria

- [ ] `cargo test -q -p pii-store --test
    ↪ retention_drops_what_the_policy_expires`
- [ ] `grep -q 'retention' crates/pii-store/src/retention.rs`
- [ ] `grep -q 'prune' crates/pii-cli/src/cmd_store.rs`
- [ ] `cargo test -q -p pii-store --test retention_keeps_a_live_run`
```

The criteria are commands, not descriptions, because three separate consumers parse them: the judge runs them, the queue copies them into the row's verify column, and the gate's `plan-verify` step checks that the copy still matches. A criterion written as prose is a criterion no machine can hold a worker to.

The requirement a plausible implementation satisfies in appearance only. Every spec here carries one, and it is the most useful paragraph in the document. Spec 362's is that a worker can add the config table, resolve the role in one node kind, and leave every other call site hardcoded: the config exists, nothing uses it, and the next stale id hides. Writing that paragraph forces the author to name the check that distinguishes the real implementation from the plausible one.

10.3 The row is the unit of dispatch

A row is one line, seven tab-separated fields, no header:

Field	Name	What it is
1	`id`	the row's key in the state file; the spec number where one exists
2	`spec`	the spec's file name under `specs/`
3	`brief`	a path to the brief, or `-`
4	`title`	one line, used in the commit message
5	`verify`	the spec's exit criteria, joined with `;`
6	`paths`	the paths this row may change, joined with `;`
7	`gate`	the gate to run, or an explicit skip

Column 5 is a *cache* of column 2's document, not an independent copy. For an open row, `state.py` re-derives it from the spec's `## Exit Criteria` section every time the plan is read; for a finished row the text is frozen, because regenerating it from a spec amended later would claim the work was held to criteria that did not exist when it ran. The reason is recorded in the code: 30 of 183 rows had dropped a criterion outright, and the drift is invisible until the gate runs the wrong list.

Column 6 makes "this work belongs to this row" checkable. `row-paths.judge` compares it with the spec's `## Files` section, and `row-verify.judge` compares column 5 with the spec's criteria, command for command. Both refusals happen at pick time, before a worker is bought.

Duplicate ids are refused, loudly. The id is the key the state is stored under, so two rows sharing one is not cosmetic: the second inherits the first's status and is never picked. Measured once: two rows both said `A20`, and every cycle for about five hours reported "queue empty" with a queued row sitting in the plan. The refusal names the duplicates and says why.

A spec number must resolve to one file. The dispatcher resolves `specs/<N>-*.md` and the session recovery keys the resume handle by the same number. Two files sharing a number send the row to the wrong document and resume another item's finished session. Measured: a row was dispatched against a spec that was already committed, resumed a ten-turn session, spent $0.37, changed nothing, and then failed verify on its own criterion.

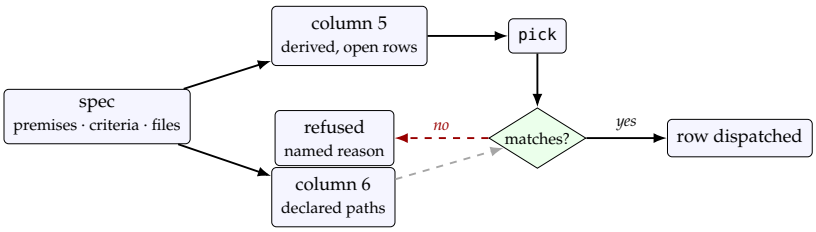


Figure 10.2: Two comparisons stand between the plan and a paid dispatch, and a row that disagrees with its spec is refused with the field named while the state is left untouched.

10.4 The state file

`scripts/autonomy/state.py` holds one JSON file, one object per row, keyed by id. Every write goes to a temp file and is renamed into place, so a cycle killed mid-write cannot leave a truncated state — which is the whole reason it is not an append-only log.

Fields: `status`, `attempts`, `session`, `last`, `evidence`, and the two annotation fields `spec` and `commit`. Statuses in use are `pending`, `in-progress`, `needs-review`, `done`, `skipped` and `parked`.

What `last` is for. It is the reason a row is not offered again, written as a short marker: `window-closed`, `worker-rc=3`, `gate-failed: <step>`, `verify-failed`, `precommit-broken`, `park-cause-repaired: <note>`. A row that is parked or needs review waits for a person *unless* its marker is one a session can act on.

`last` marker	Resumable	Who acts next
`window-closed`	yes, with the session	the next cycle inside the window
`worker-rc=N`	yes	the same session, resumed
`verify-failed`	yes	the same session, with a corrective brief
`gate-failed: <step>`	yes	the same session, with the failing step named

`last` marker	Resumable	Who acts next
`precommit-broken`, `precommit-timeout`	yes	the same session; attempts not spent
`park-cause-repaired: <note>`	yes	the repaired row goes back on the queue
anything else (e.g. `3-strikes`)	no	a person reads it

Three strikes. A row is parked when it reaches three attempts: at dispatch if the count is already three, or at verify if the failure would make it three. The reason is written as prose that states the rule — the same mechanical failure repeated, so a fourth dispatch would spend without new information. Attempts are never incremented by a timeout or a broken pre-commit build, because neither is a verdict on the worker.

Repair, not reset. `state.py repair <id> <note>` moves a parked row back to `needs-review` (or `pending` without a session) and writes `park-cause-repaired: <note>`. It deliberately does not touch attempts: the cap is a safety property, so a repaired row that fails again parks again on its own merits.

Annotations are proven. `state.py annotate <id> -spec <path> -commit <sha>` refuses to write a spec that does not exist, a spec that is not the row's own, or a commit git does not know. The field that can hold a lie is worse than the missing field: a row naming a commit that does not contain its work is evidence an operator would act on.

```
python3 scripts/autonomy/state.py pick | cut -f1          # the top dispatchable id, or NONE
python3 scripts/autonomy/state.py get 376                 # the row as JSON
python3 scripts/autonomy/state.py set 376 status=needs-review attempts=1 last=verify-failed
python3 scripts/autonomy/state.py annotate 376 --spec specs/376-retention-drops-what-the-policy-expires.md \
                                              --commit 7cea0b3
```

10.5 The lane

A dispatch goes through a named script, never an inline command. There are two lanes:

- **coder** — a one-node graph that runs a coder node on the spec. The default.
- **spec** — the tracked graph `graphs/spec.toml`, which is the same implementation expressed as composition: `code` (a coder with its own wall and spend bounds), `verify` (a judge reading the spec at a commit), `docs` (a second coder whose brief is documentation only), `reverify` (the same criteria again, so a docs step that broke them is refused), then `done`, with `unverified` and `refused` as the two named refusals.

`PII_LANE_GRAPH=spec` selects the second; anything else keeps the first. The script prints `LANE-GRAPH: <lane>` before anything is spent and again into the worker log, and the cycle reads that line to record which lane

a tree-changing run used. The commit carries it as a `Lane:` trailer, so a commit says which of the two produced it. A run that changed nothing leaves the previous record standing, because a commit must never be attributed to a run that produced no tree.

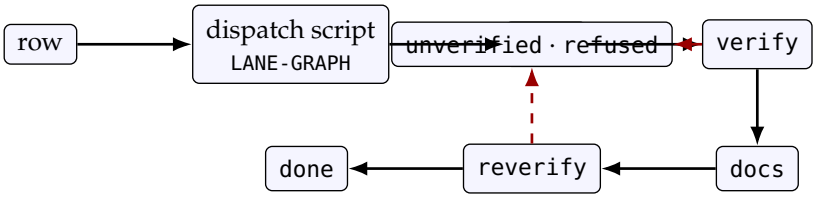


Figure 10.3: Two judges, one coder and one documentation pass in the spec lane, with refusals wired by name so a failed implementation parks its row instead of ending the run with the row still claimed.

10.6 One cycle, in order

A cycle is deterministic on purpose: the reasoning belongs to the agent that calls it, and the mechanics belong in a script that can be tested. `scripts/autonomy/cycle.sh` does the following, in this order, and the order is load-bearing:

1. **The tree hold and the lock.** A background cycle yields if an interactive turn holds the tree, and reports `OUTCOME=tree-held`. The lock spans the whole cycle; `-o` is passed to `flock` so a daemonized grandchild cannot inherit the descriptor and hold the lock after the cycle is gone.
2. **The operator's switch.** `PAUSED` in the control file stops before spending.
3. **The window probe.** A closed account window exits 3 with nothing spent and the row not advanced. An undecided probe exits 4 rather than guessing.
4. **The pick.** `state.py pick` returns the top dispatchable row, or `NONE`, or a refusal. A plan that cannot be read is a loud failure, never an empty queue: "nothing pending" and "the queue is broken" are different outcomes and only one is safe to sleep on.
5. **The re-read.** Between the pick and the dispatch, the state is read again. A row that another cycle closed is not dispatched; a row whose marker says the work is already `worker-ok`, `verified` or `gated` resumes at verify with no worker bought.
6. **The tree.** A dirty tree is handled in three moves: save, attribute, clean. A tree the loop's own last row left is stashed so this gate measures only this change; a tree no row owns is a person's, and the cycle stops with `OUTCOME=dirty-tree`.
7. **The dispatch.** A session id recovered from the worker's own log is used as a resume handle if one exists; otherwise the worker is dispatched with the lane script, and the session id is recorded.
8. **The verify.** The row's own commands, run in a login shell that sources the loop's environment first. Measured: a login shell resets `PATH` for root and loses `/usr/local/cargo/bin`, so two cargo criteria were recorded as failures on a tree where all four passed.
9. **The gate.** Only an explicit `0`, `-` or `none` skips it. A green verify is not a green gate.

10. **The pre-commit build.** The door builds `HEAD` plus the index before the commit exists, so a tree that does not build is never committed.
11. **The commit, then the head build.** The commit is made with an explicit neutral identity; then `head-builds.sh` builds the commit that was just made, because the gate measured the working tree and this reads the commit.
12. **The annotation and the chain.** The row records its spec and its commit; the next row starts as soon as the lock is free.

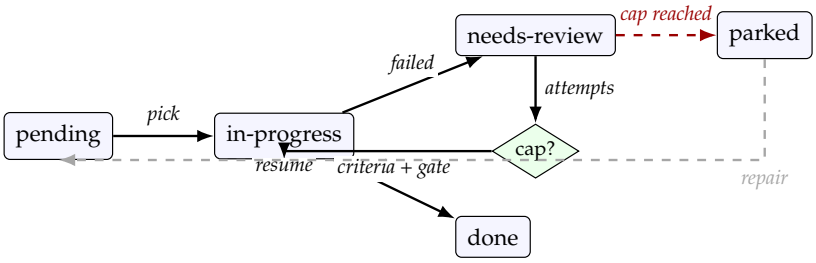


Figure 10.4: Four statuses and one decision: a row that fails goes to needs review with a marker, resumes in the same session while attempts remain, and parks for a person at three.

10.7 Resume before retry

The rule is old and it is the cheapest in the system: continue the existing session. A fresh start re-derives context the previous run already established and already billed for.

Mechanically, a resume needs a session id, and the session id is unrecoverable once the process exits. So it is written at dispatch time, and the recovery path is layered:

- The dispatch script records the id in `_tmp/claude-batchC/session-<N>-*.txt`.
- If the process was cut off before that, `session.sh recover <N>` reads the worker's own log, and the cycle records the id with `last=resumed-from-worker-log`.
- A worker cut off by a closed account window records its session before exiting, so the next cycle resumes rather than restarting from scratch. Without that, a dirty tree with no session was kept as a leftover and the spec was dispatched again from zero.

A resume carries a diagnosis. When the previous attempt ended `gate-failed`, `verify-failed` or `worker-rc`, the resume is sent with a corrective brief that names what failed and includes the failing output. The worker cannot see the gate's verdict, and its own criteria may well have passed — which is exactly how one first pass went: criteria 4 of 4, gate red on a tree the export had not included. Naming the failure is the difference between a fix and a retry.

The brief's evidence block has its own history. It was built from the newest cycle log, which was usually the accumulator file being appended to while the brief was assembled — so a brief for one row quoted rows from a previous month. It now takes the newest log that is neither this cycle's nor the accumulator, and falls back to the log's tail so it can never print nothing.

What is not a row fault. Three failure classes are separated from the row on purpose:

- **Quota.** A provider limit puts the row back on the queue with its attempts and session intact, and the reset time the CLI printed is recorded. Never a blind retry, never a scope cut.
- **Authentication.** A dead login refuses every row, so counting it as an attempt would park three rows that were never tried. The row keeps its attempts and the queue is paused for a person.
- **A killed gate.** A timeout exits with its own outcome; attempts are not incremented and the marker keeps the next cycle at verify, so a slow host costs a retry and never a re-dispatch.

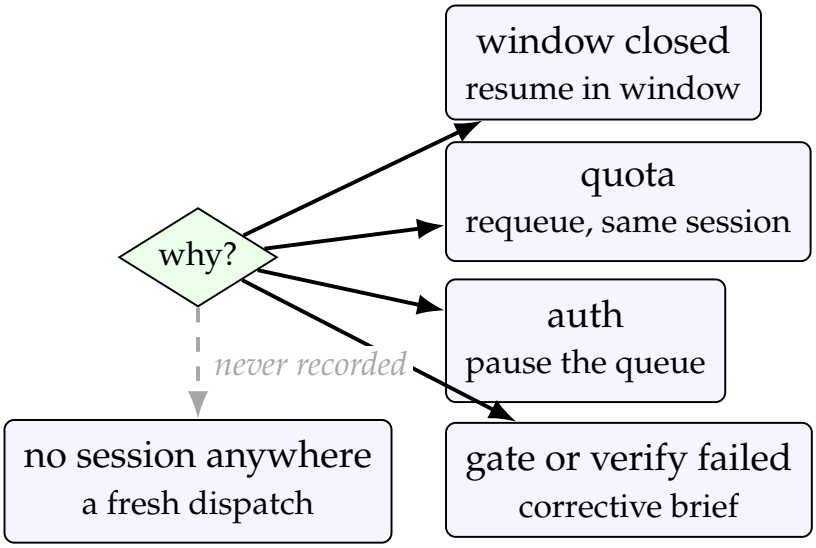


Figure 10.5: Four resumable reasons and one that is not, and only the last one justifies paying for context twice.

10.8 Quarantine: a park must not wedge the queue

Parking used to leave the row's working-tree changes in place, and the dispatcher refuses to dispatch on a dirty tree, so one parked row stopped every later row for a whole night. Measured: a row parked with nine files and the next row never ran — and the parked work was good, five tests green.

`scripts/autonomy/quarantine-wip.sh <row> <reason>` now puts the changes aside without deleting anything:

1. Every changed path is *copied* to `/projects/_trash/<row>-<reason>-<ts>/` with the tree layout preserved, and `git status -porcelain` is saved as `status.txt`.
2. A modified tracked file is restored to its `HEAD` content with `git show HEAD:<path>` — explicitly *not* `git checkout -`, which is banned in this workspace because it destroys uncommitted work.
3. An untracked file is *moved* out of the tree, never removed.
4. The directory is recorded in `ASIDE.txt`, and the path is printed on stdout so the cycle can put it in the row's `last` note.

If anything is still dirty afterwards the script exits 3 with `STILL-DIRTY` and the count, and the cycle says so: the dispatcher will refuse until the tree is clean, and a warning is better than a silent night.

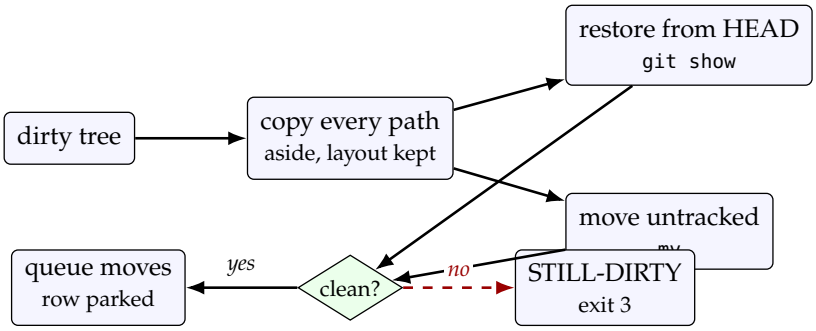


Figure 10.6: Copy first and restore second, because a restore that loses the work it was preserving is the one unrecoverable path.

```
$ bash scripts/autonomy/quarantine-wip.sh 352 parked
QUARANTINED 9
/projects/_trash/352-parked-<UTC timestamp>
$ git status --porcelain | wc -l
0
```

10.9 Review rows

A review is work with a brief, a deliverable and a verifier, so it is a row. The user directive behind it was that an analysis must also be put in the queue, positioned before the items that matter.

A review spec carries two extra lines, `BRIEF: <path>` and `DELIVERABLE: <path>`. `review-row.sh` reads both, refuses if the brief is missing or the deliverable line is absent, dispatches the read-only reviewer in plan mode, then copies the newest `.md` in the brief's directory to the deliverable and exits 5 if the deliverable is missing. The check is on the artefact, not on the reviewer's claim to have produced one.

10.10 Commits

A commit is one message saying what the change does and why the alternative was rejected. Three rules apply to every commit made by the loop:

1. **An explicit neutral identity.** The repository's own git configuration may carry a tool name, and then every plain `git commit` breaches the rule silently. The cycle passes `-c user.name` and `-c user.email` on the command line, so the identity is a property of the commit and not of the machine.
2. **Only the row's paths.** `git add` is given the row's declared paths, and `unlisted-changes.sh` refuses the commit if the index would include a file the row does not list. The tree is left as the worker made it, because the next cycle resumes at verify.

3. **The gate's verdict is in the message, and the lane is a trailer.** The message states that the criteria are in the spec and the gate is green, and a `Lane: spec|coder` trailer names the lane when one was recorded.

Nothing staged is not nothing to do. If the add leaves the index empty, the work may already have landed in an earlier commit. The cycle closes the row on exactly the evidence the normal path uses — the row's own criterion plus a green gate — and names the commit that carries the work. A leftover commit is still a commit; re-dispatching it spends the same spec twice.

```
git add -- crates/pii-store/src/retention.rs crates/pii-store/tests/...
UNLISTED=$(bash scripts/autonomy/unlisted-changes.sh "$PATHS") || { git
    ↪ reset -q; exit 17; }
git -c user.name="Marcelo Correa" -c user.email="mvcorrea@gmail.com"
    ↪ commit -q \
  -m "376-retention: automated pass - retention drops what the policy
    ↪ expires (gate green)

Lane: spec"
```

The identity is on the command line rather than in the repository's configuration for the reason recorded in the workspace rules: a repository whose own `git config` holds a tool name breaches the rule on every plain commit, silently, because nobody re-reads the config. That happened for thirty-five commits in one repository here before anyone looked.

10.11 The review that gave the backlog verdicts

The parked backlog was reviewed on 2026-11-04 under spec 377, and the method is worth copying: every criterion of every parked row was *executed*, from the plan's column 5, split on `;` the same way the runner splits it. Nothing was read from a helper's summary.

The review produced four kinds of verdict, and the distinctions are the useful part:

- `proceed` — the row's work is not on disk, so it is a clean dispatch from `HEAD`.
- `close` — the row was superseded by later work, and its criteria pass or are stale. Closing it is a state write, not code.
- `needs-decision` — the row cannot be judged without an answer only the operator can give (a budget question, or which of two commits answers the row).
- missing evidence — a WIP directory the spec names does not exist, so any verdict depending on it is unverified rather than done.

It also named four failure classes, each of which recurs in this manual: a guard that fires on the case it was built to permit; criteria that name files which no longer exist; a sandbox in which the criteria cannot run as written (a read-only `/tmp`, no `uv` cache, no `python3`); and quarantined work that is not where the spec says it is.

10.12 Failure modes worth recognising

1. **A queue that reports itself empty while work is queued.** Cause: two rows sharing an id, so the second inherits the first's status. The plan refuses duplicates now.
2. **A night with one row parked and nothing else done.** Cause: a park that left the tree dirty. Quarantine is the fix, and the row's `last` note carries the aside path.
3. **The same spec bought twice.** Cause: a resume that did not happen, or a number that resolved to the wrong file. Check the session file and the spec number before the next dispatch.
4. **A criterion that passes on the worker's tree and fails in the gate.** Cause: the export did not include the change (a deletion, or a file the overlay did not see). The gate's export-lost and overlay-deletion handling exist for this.
5. **A commit that names a tool.** Cause: the repository's git configuration. Read it, and pass the identity explicitly.
6. **A worker that fixes the symptom.** Cause: a corrective brief that named the failing command but not the rule. The brief says: fix the cause, do not weaken or delete an assertion, do not raise a timeout, do not add a sleep, do not touch anything outside the spec's scope.

The habit that produced this manual

Every mechanism described here was first understood by running it and then disbelieving its explanation. The recurring failure in this codebase is not a missing feature; it is a *guard that fires on the case it exists to permit*, while reporting a reason that is not the real one. Four instances appear in these chapters: a park that wedged the queue, a dirty tree that refused its own work, a resolver nothing called, and a disk check that was an unset variable.

The defence is the same each time: make the check consume what the live path actually produces, and keep a control that must still take the normal path.